

Лабораторна робота №3. Використання СУБД MongoDB при створенні вебзастосунку мовою Python

Завдання. Для проєкта з першої лабораторної роботи провести міграцію даних на СУБД MongoDB зі збереженням базового функціоналу вебзастосунка та додаванням нового за рахунок застосування MongoDB в цілому, та бібліотеки pymongo зокрема.

ЗМІСТ

Робота з базою даних MongoDB	4
1 Пристрій бази даних. Документи	4
1.1 Пара «ключ-значення»	4
1.2 Ідентифікатор документа	7
2 Встановлення та адміністрування бази даних.....	8
2.2 Встановлення БД поточною	8
2.3 Отримання статистики	10
Додавання даних	11
3.1 Додавання документів.....	11
3.2 Завантаження даних із файлу	16
4 Вибірка та фільтрація.....	17
4.1 Вибірка даних.....	17
4.2 Фільтрація даних.....	18
4.3 Фільтрація за відсутніми властивостями.....	20
4.4 Фільтрація за елементами масиву	21
4.5 Проекція.....	25
4.6 Запит до вкладених об'єктів.....	29
4.7 Використання JavaScript	30
4.8 Використання регулярних виразів	32
4.9 Пошук одиночного документа	32
4.10 Курсори.....	33
5 Пагінація та сортування.....	35
5.1 Функції limit(), skip(), sort()	35
5.2 Оператор \$slice.....	37
6 Індеси.....	40
6.1 Створення індексу	40
6.2 Видалення індексів	42
6.3 Налаштування індексів	42
6.4 Управління індексами	43
7 Агрегатні функції	44
7.1 Кількість елементів у колекції.....	44
7.2 Функція distinct().....	46
7.2 Функції min() і max()	46
8 Оператори вибірки	48

8.1 Умовні оператори	48
8.2 Логічні оператори	52
8.3 Пошук за масивами	56
9 Оновлення даних	62
9.1 Функція replaceOne()	62
9.2 Функції updateOne() і updateMany()	64
9.3 Видалення поля	67
9.4 Оновлення масивів	68
10 Видалення елемента з масиву	71
11 Видалення даних	73
11.1 Видалення документа	73
11.2 Видалення колекцій і баз даних	75
12 Встановлення посилань у БД	75
12.1 Ручне встановлення посилань	75
12.2 Автоматичне зв'язування	77
13 Керування колекцією	79
13.1 Явне створення колекції	79
13.2 Перейменування колекції	80
13.3 Обмежені колекції	81
13.4 Підколекції	83
14 Робота з БД MongoDB у Python	83
14.1 Бази даних NoSQL	84
14.2 MongoDB і коли його використовувати	84
14.3 Взаємодія з MongoDB через Mongo Shell	85
14.4 Інтеграція Python з MongoDB	91
14.5 Підключення до MongoDB	92
14.6 Створення документів	93
14.7 Отримання документів	95
14.8 Оновлення документів	96
14.9 Видалення документів	97

Робота з базою даних MongoDB

1 Організація бази даних. Документи

Всю модель влаштування бази даних у MongoDB можна представити таким чином:

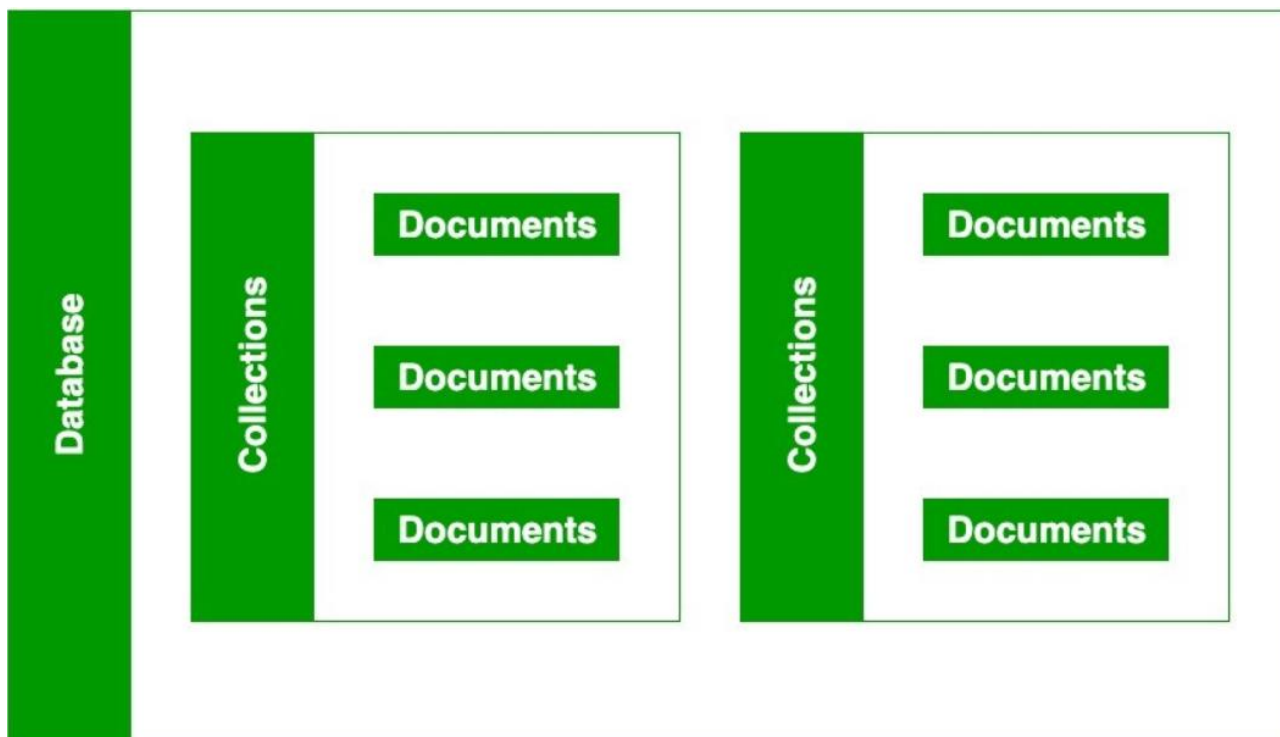


Рисунок 1 – Організація бази даних у MongoDB (<https://media.geeksforgeeks.org/wp-content/uploads/20200219180521/MongoDB-database-colection.png>)

Якщо в реляційних БД вміст складають таблиці, то в *MongoDB* база даних складається з колекцій.

Кожна колекція має своє унікальне ім'я - довільний ідентифікатор, що складається з не більше ніж 128 різних алфавітно-цифрових символів і знака підкреслення.

На відміну від реляційних баз даних *MongoDB* не використовує табличний пристрій із чітко заданою кількістю стовпців і типів даних. *MongoDB* є документо-орієнтованою системою, в якій центральним поняттям є документ.

1.1 Пара «ключ-значення»

Документ можна уявити як об'єкт, що зберігає деяку інформацію. У певному сенсі він подібний до рядків у реляційних СУБД, де рядки зберігають інформацію про окремий елемент. Наприклад, типовий документ:

```
{
  "firstname": "Timothy",
  "surname": "Donald",
  "lastname": "Cook"
  "age": "63",
  "company": {
    "name" : "Apple",
    "salary" : "99999"
  }
}
```

Документ представляє набір пар ключ-значення. Наприклад, у виразі *"firstname": "Timothy"* *"firstname"* представляє ключ, а *"Timothy"* - значення.

Ключі представляють рядки. Значення ж можуть відрізнятися за типом даних. У даному випадку у нас майже всі значення також представляють строковий тип, і лише один ключ (*company*) посилається на окремий об'єкт. Усього є такі типи значень:

- *String*: строковий тип даних, (для рядків використовується кодування UTF-8)
- *Array* (масив): тип даних для зберігання масивів елементів
- *Binary data* (двійкові дані): тип для зберігання даних у бінарному форматі
- *Boolean*: булевий тип даних, що зберігає логічні значення *TRUE* або *FALSE*, наприклад, *{"married": FALSE}*
- *Date*: зберігає дату у форматі часу *Unix*
- *Double*: числовий тип даних для зберігання чисел із плаваючою крапкою
- *Integer*: використовується для зберігання цілочисельних значень розміром 32 біти, наприклад, *{"age": 29}*

- *Long*: використовується для зберігання цілочисельних значень розміром 64 біти
- *JavaScript*: тип даних для зберігання коду *javascript*
- *Min key/Max key*: використовуються для порівняння значень з найменшим/найбільшим елементів *BSON*
- *Null*: тип даних для зберігання значення *Null*
- *Object*: об'єкт, який містить набір властивостей
- *ObjectId*: тип даних для зберігання id документа
- *Regular expression*: застосовується для зберігання регулярних виразів
- *Decimal128*: тип даних для зберігання десяткових дробових чисел розміром 128 біт, які дають змогу розв'язати проблеми з проблемою точності обчислень під час використання дробових чисел, що становлять тип *Double*.
- *Timestamp*: застосовується для зберігання часу

На відміну від рядків документи можуть містити різномірну інформацію. Так, поруч із документом, описаним вище, в одній колекції може перебувати інший об'єкт, наприклад:

```
{
  "name": "Jerom",
  "birthday": "1977.01.01",
  "place" : "London",
  "languages" :[
    "english",
    "german",
    "spanish"
  ]
}
```

Здавалося б різні об'єкти за винятком окремих властивостей, але всі вони можуть перебувати в одній колекції.

Ще пара важливих зауважень: у *MongoDB* запити мають регістрозалежність і сувору типізацію. Тобто наступні два документи не будуть ідентичні:

```
{"age" : "55"}  
{"age" : 55}
```

Якщо в першому випадку для ключа *age* визначено як значення рядок, то в другому випадку значенням є число.

1.2 Ідентифікатор документа

Для кожного документа в *MongoDB* визначено унікальний ідентифікатор, який називається *_id*. Під час додавання документа в колекцію цей ідентифікатор створюється автоматично. Однак розробник може сам явно задати ідентифікатор, а не покладатися на автоматично генеровані, вказавши відповідний ключ і його значення в документі.

Це поле повинно мати унікальне значення в межах колекції. І якщо ми спробуємо додати в колекцію два документи з однаковим ідентифікатором, то додасться тільки один із них, а під час додавання другого ми отримаємо помилку.

Якщо ідентифікатор не задано явно, то *MongoDB* створює спеціальне бінарне значення розміром 12 байт. Це значення складається з декількох сегментів: значення типу *timestamp* розміром 4 байти (яке представляє кількість секунд з моменту початку епохи *Unix*), випадкове число з 5 байт і лічильник з 3 байт, який ініціалізовано випадковим числом. Така модель побудови ідентифікатора гарантує з високою часткою ймовірності, що він матиме унікальне значення.

2 Встановлення та адміністрування бази даних

У цьому та подальших пунктах ми розглянемо базові операції з даними в *MongoDB* із застосуванням як консольної оболонки *mongosh*, так і графічного клієнта *MongoDB Compass*. Однак у будь-якому разі під час початку роботи із сервером слід не забувати запускати сам сервер - тобто застосунок *mongod*.

2.2 Встановлення БД поточною

Починаючи працювати з *MongoDB* у консольній оболонці *mongosh*, насамперед треба встановити потрібну нам базу даних як поточну, щоб потім її використовувати. Для цього треба використовувати команду *use*, після якої йде назва бази даних. При цьому не важливо, чи існує така БД чи ні. Якщо її немає, то *MongoDB* автоматично створить її під час додавання в неї даних.

Отже, запустимо консольну оболонку *mongosh* і введемо там таку команду:

```
use usersdb
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1:27020/  x  +  v
Using MongoDB: 7.0.5
Using Mongosh: 2.1.1

For mongosh info see: https://docs.mongodb.com/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

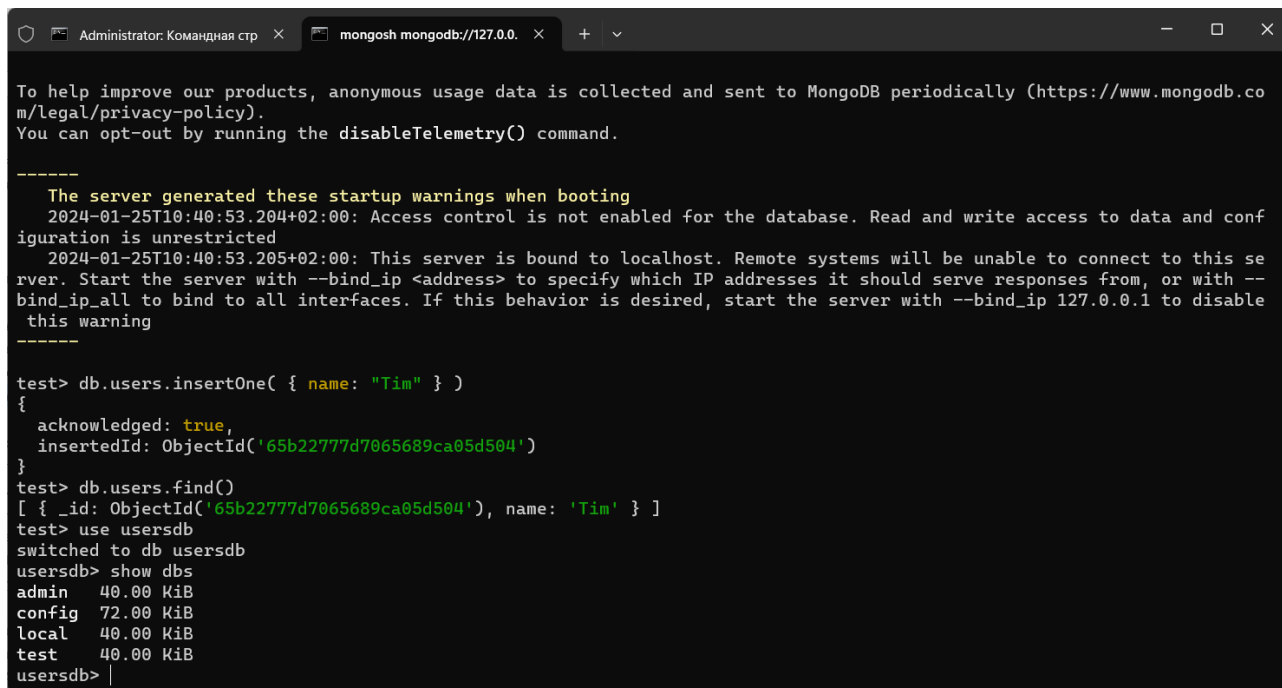
-----
The server generated these startup warnings when booting
2024-01-25T10:40:53.204+02:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2024-01-25T10:40:53.205+02:00: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> db.users.insertOne( { name: "Tim" } )
{
  acknowledged: true,
  insertedId: ObjectId('65b22777d7065689ca05d504')
}
test> db.users.find()
[ { _id: ObjectId('65b22777d7065689ca05d504'), name: 'Tim' } ]
test> use usersdb
switched to db usersdb
usersdb>
  
```

Рисунок 2 - Створення бази даних у MongoDB

Тепер як поточну буде встановлено БД *usersdb*. При цьому не важливо, що спочатку такої бази даних може не існувати: якщо її не існує, то під час першої операції вона створюється.

Якщо ви раптом не впевнені, а чи існує вже база даних з такою назвою, то за допомогою команди *show dbs* можна вивести назви всіх наявних БД на консоль:



```

Administrator: Командная строка  mongosh mongodb://127.0.0.1
To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2024-01-25T10:40:53.204+02:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2024-01-25T10:40:53.205+02:00: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> db.users.insertOne( { name: "Tim" } )
{
  acknowledged: true,
  insertedId: ObjectId('65b22777d7065689ca05d504')
}
test> db.users.find()
[ { _id: ObjectId('65b22777d7065689ca05d504'), name: 'Tim' } ]
test> use usersdb
switched to db usersdb
usersdb> show dbs
admin    40.00 KiB
config   72.00 KiB
local    40.00 KiB
test     40.00 KiB
usersdb>
  
```

Рисунок 3 – Результат виконання команди *show dbs*

Зверніть увагу, що в списку баз даних ще немає БД *usersdb*, тому що ми з нею не проводив ще ніяких операцій.

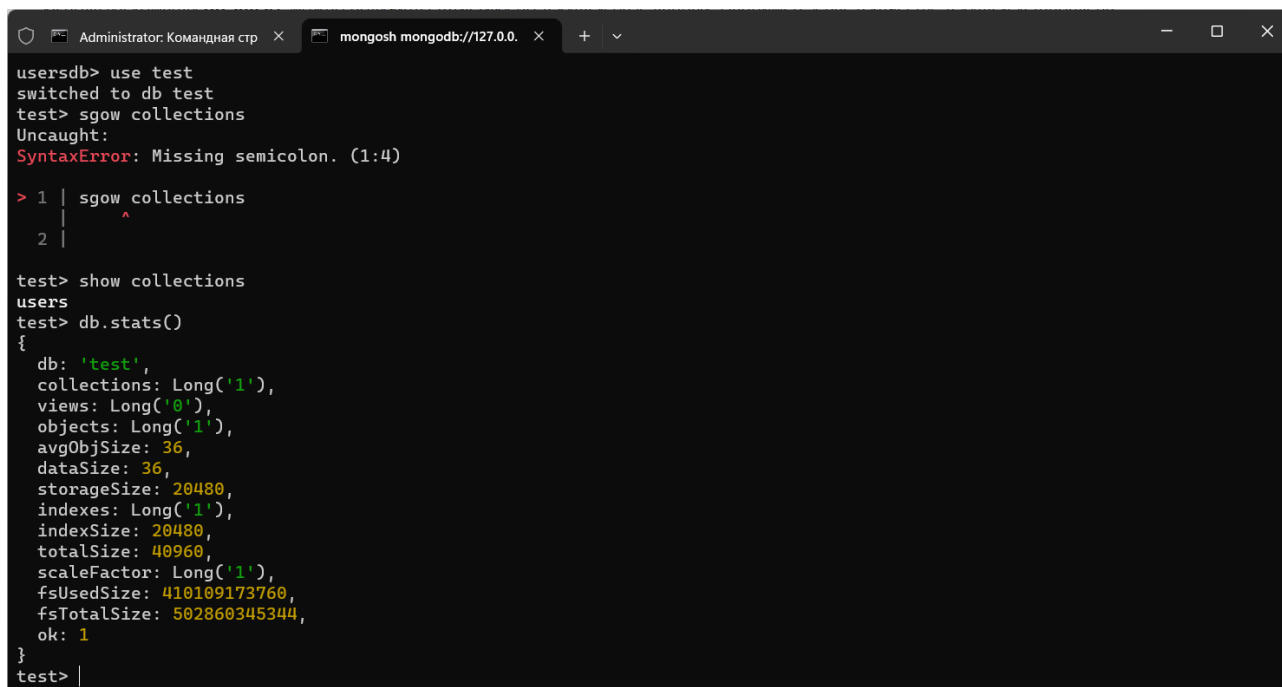
Для бази даних можна задати будь-яке ім'я, проте є деякі обмеження. Наприклад, в імені не повинно бути символів /, \, ., ", *, <, >, :, |, ?, \$. Крім того, імена баз даних обмежені 64 байтами. Також є зарезервовані імена, які не можна використовувати: *local*, *admin*, *config*. Ці імена представляють бази даних, які вже мають за замовчуванням на сервері і призначені для службових цілей.

Крім баз даних ми можемо переглянути список усіх колекцій у поточній БД за допомогою команди

```
show collections
```

2.3 Отримання статистики

Використовуючи команду `db.stats()`, можна отримати статистику за поточною базою даних. Наприклад, у нас як поточну встановлено базу даних `test`:



```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
usersdb> use test
switched to db test
test> sgow collections
Uncaught:
SyntaxError: Missing semicolon. (1:4)

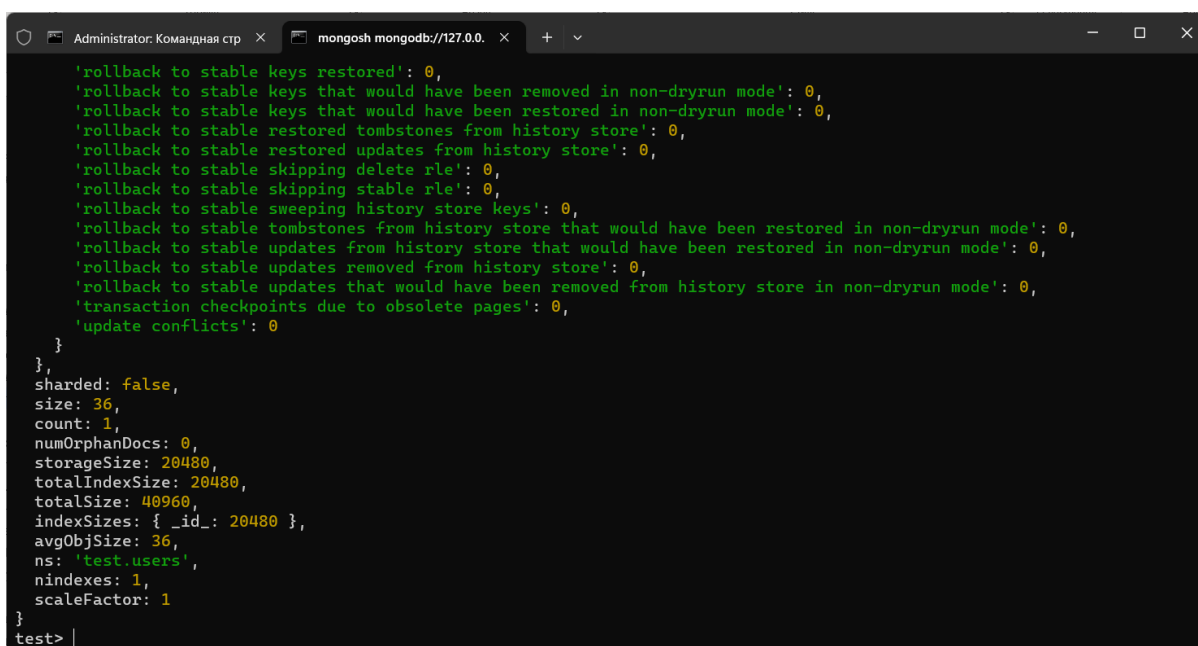
> 1 | sgow collections
    |      ^
    2 |

test> show collections
users
test> db.stats()
{
  db: 'test',
  collections: Long('1'),
  views: Long('0'),
  objects: Long('1'),
  avgObjSize: 36,
  dataSize: 36,
  storageSize: 20480,
  indexes: Long('1'),
  indexSize: 20480,
  totalSize: 40960,
  scaleFactor: Long('1'),
  fsUsedSize: 410109173760,
  fsTotalSize: 502860345344,
  ok: 1
}
test> |

```

Рисунок 4 - Виведення статистики за БД у MongoDB

Схожим чином ми можемо дізнатися всю статистику за окремою колекцією. Наприклад, дізнаємося статистику за колекцією `users`: `db.users.stats()`



```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
'rollback to stable keys restored': 0,
'rollback to stable keys that would have been removed in non-dryrun mode': 0,
'rollback to stable keys that would have been restored in non-dryrun mode': 0,
'rollback to stable restored tombstones from history store': 0,
'rollback to stable restored updates from history store': 0,
'rollback to stable skipping delete rle': 0,
'rollback to stable skipping stable rle': 0,
'rollback to stable sweeping history store keys': 0,
'rollback to stable tombstones from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates removed from history store': 0,
'rollback to stable updates that would have been removed from history store in non-dryrun mode': 0,
'transaction checkpoints due to obsolete pages': 0,
'update conflicts': 0
}
},
sharded: false,
size: 36,
count: 1,
numOrphanDocs: 0,
storageSize: 20480,
totalIndexSize: 20480,
totalSize: 40960,
indexSizes: { _id_: 20480 },
avgObjSize: 36,
ns: 'test.users',
nindexes: 1,
scaleFactor: 1
}
test> |

```

Рисунок 5 – Виведення статистика за колекцією

Додавання даних

Встановивши БД, тепер ми можемо додати в неї дані. Усі дані зберігаються в БД у форматі *BSON*, який близький до *JSON*, тому нам треба також вводити дані в цьому форматі. І хоча у нас, можливо, наразі немає жодної колекції, але під час додавання в неї даних вона автоматично створюється.

Як раніше йшлося, ім'я колекції - довільний ідентифікатор, що складається з не більше ніж 128 різних алфавітно-цифрових символів і знаку підкреслення. Водночас ім'я колекції не повинно починатися з префіксу *system*, оскільки він зарезервований для внутрішніх колекцій (наприклад, колекція *system.users* містить усіх користувачів бази даних). І також ім'я не повинно містити знака долара - \$.

3.1 Додавання документів

Для додавання в колекцію можуть використовуватися два її методи:

- *insertOne()*: додає один документ
- *insertMany()*: додає кілька документів

Припустимо, ми використовуємо базу даних *test*. Додамо в неї один документ:

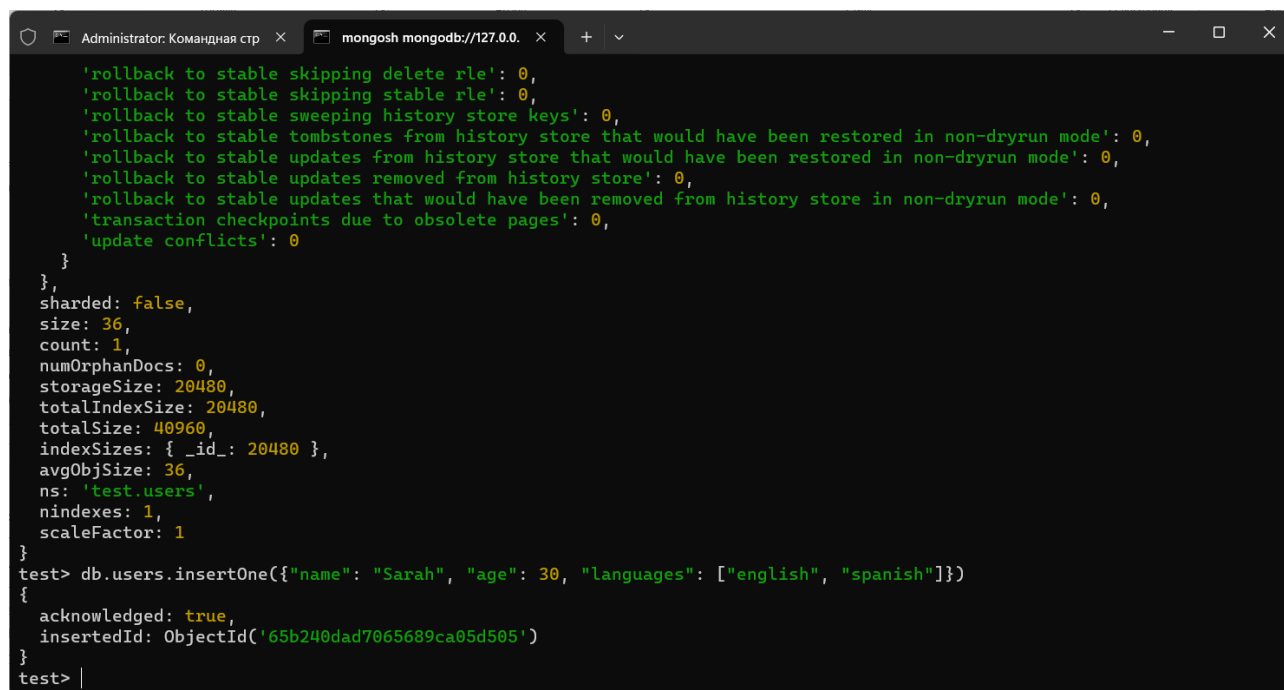
```
test> db.users.insertOne({"name": "Sarah", "age": 30, "languages":
["english", "spanish"]})
```

Документ являє собою набір пар ключ-значення. У цьому випадку документ, що додається, має три ключі: *name*, *age*, *languages*, і кожному з них зіставляє певне значення. Наприклад, ключу *languages* в якості значення зіставляється масив. Варто зазначити, що назви ключів можуть використовуватися в лапках, а можуть і без лапок.

Деякі обмеження при використанні імен ключів:

- символ \$ не може бути першим символом в імені ключа
- ім'я ключа не може містити символ крапки .

Під час додавання даних, якщо ми явно не надали значення для поля `"_id"` (тобто унікального ідентифікатора документа), то воно генерується автоматично. Так, після виконання операції додавання консоль виведе згенерований для доданого документа ідентифікатор:



```

'rollback to stable skipping delete rle': 0,
'rollback to stable skipping stable rle': 0,
'rollback to stable sweeping history store keys': 0,
'rollback to stable tombstones from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates removed from history store': 0,
'rollback to stable updates that would have been removed from history store in non-dryrun mode': 0,
'transaction checkpoints due to obsolete pages': 0,
'update conflicts': 0
}
},
sharded: false,
size: 36,
count: 1,
numOrphanDocs: 0,
storageSize: 20480,
totalIndexSize: 20480,
totalSize: 40960,
indexSizes: { _id_: 20480 },
avgObjSize: 36,
ns: 'test.users',
nindexes: 1,
scaleFactor: 1
}
test> db.users.insertOne({"name": "Sarah", "age": 30, "languages": ["english", "spanish"]})
{
  acknowledged: true,
  insertedId: ObjectId('65b240dad7065689ca05d505')
}
test> |

```

Рисунок 5 – Результат виконання команди `insertOne()`

У відповіді сервера ми отримаємо об'єкт, у кого параметр `insertedId` міститиме ідентифікатор. Але в принципі ми можемо самі встановити цей ідентифікатор під час додавання даних:

```
test> db.users.insertOne({"_id": 987654, "name": "Jessica", "age": 25, "languages": ["english", "ukrainian"]})
```

або використовувати для ідентифікатору тип `ObjectId`

```
test> db.users.insertOne({"_id":
ObjectId("17f56abb34d8e543512da124"), "name": "Parker", "age": 45,
"languages": ["english", "italian", "" ]})
```

Варто враховувати, що якщо для визначення ідентифікатора застосовується тип *ObjectId*, то він має містити рядок 12 байт або рядок із 24 символів.

У разі вдалого додавання на консоль буде виведено ідентифікатор доданого документа.

```

'update conflicts': 0
}
},
sharded: false,
size: 36,
count: 1,
numOrphanDocs: 0,
storageSize: 20480,
totalIndexSize: 20480,
totalSize: 40960,
indexSizes: { _id_: 20480 },
avgObjSize: 36,
ns: 'test.users',
nindexes: 1,
scaleFactor: 1
}
test> db.users.insertOne({"name": "Sarah", "age": 30, "languages": ["english", "spanish"]})
{ acknowledged: true, insertedId: ObjectId('65b240dad7065689ca05d505') }
test> db.users.insertOne({"_id": 987654, "name": "Jessica", "age": 25, "languages": ["english", "ukrainian"]})
{ acknowledged: true, insertedId: 987654 }
test> db.users.insertOne({"_id": ObjectId('17f56abb34d8e543512da124'), "name": "Parker", "age": 45, "languages": ["english", "italian", "" ]})
{ acknowledged: true, insertedId: ObjectId('17f56abb34d8e543512da124') }
test> |

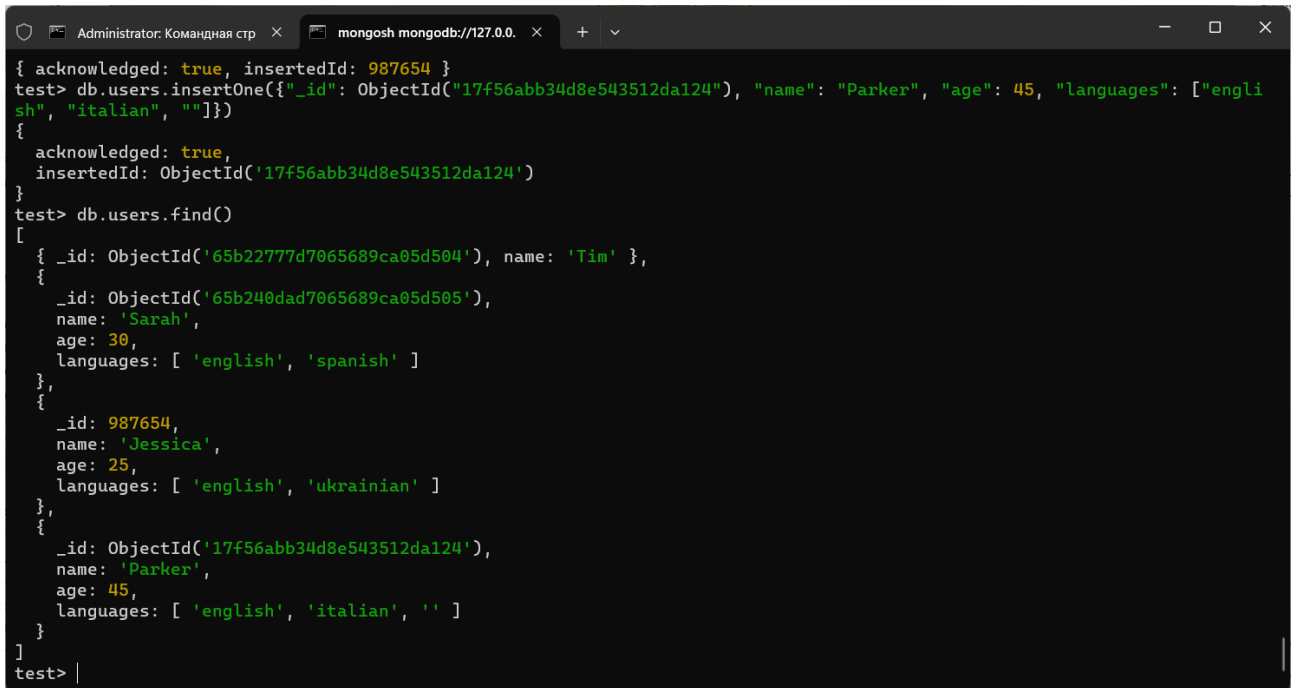
```

Рисунок 6 – Результат додавання даних з власними id

І щоб переконатися, що документ у БД, ми його виводимо функцією `find`.

```
test> db.users.find()
```

За замовчуванням він виводить усі документи колекції:



```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.  ×  +  ▾
{ acknowledged: true, insertedId: 987654 }
test> db.users.insertOne({ "_id": ObjectId("17f56abb34d8e543512da124"), "name": "Parker", "age": 45, "languages": ["english", "italian", ""] })
{
  acknowledged: true,
  insertedId: ObjectId('17f56abb34d8e543512da124')
}
test> db.users.find()
[
  { _id: ObjectId('65b2277d7065689ca05d504'), name: 'Tim' },
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  },
  {
    _id: ObjectId('17f56abb34d8e543512da124'),
    name: 'Parker',
    age: 45,
    languages: [ 'english', 'italian', '' ]
  }
]
test> |

```

Рисунок 7 – Зміст колекції

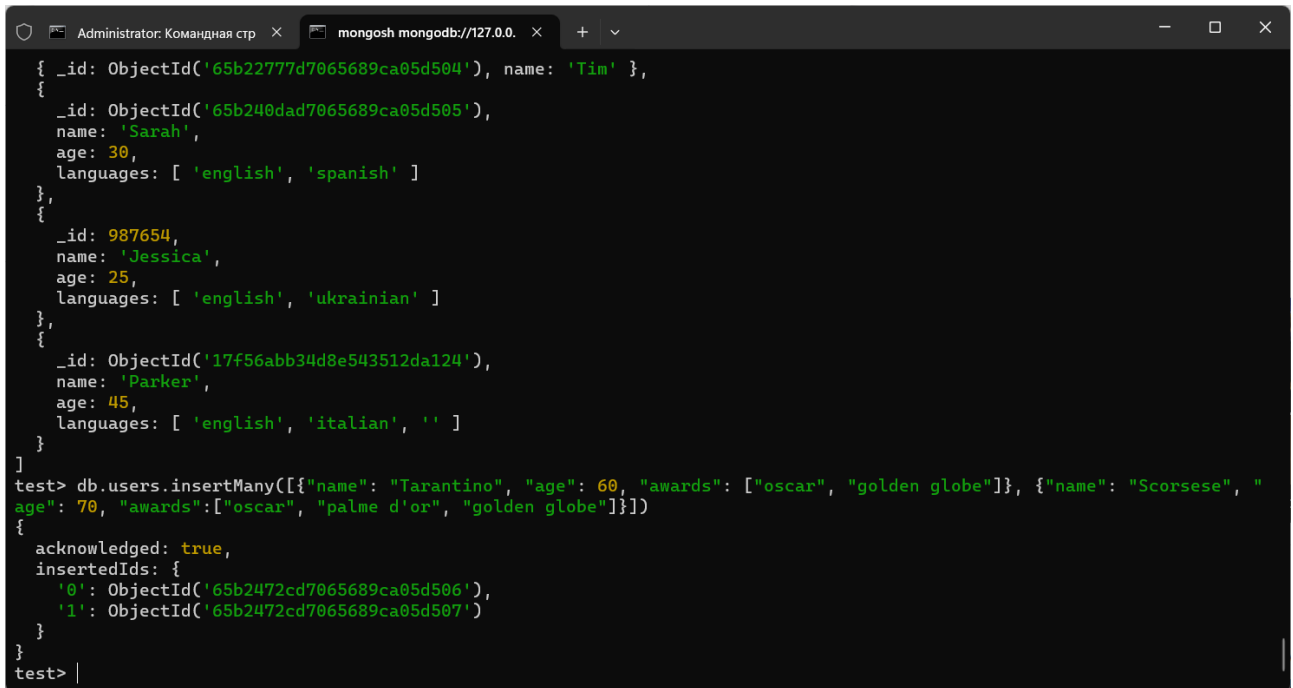
Якщо треба додати низку документів, то ми можемо скористатися методом `insertMany()`, який приймає масив об'єктів:

```

db.users.insertMany([{"name": "Tarantino", "age": 60, "awards":
["oscar", "golden globe"]}, {"name": "Scorsese", "age": 70,
"awards":["oscar", "palme d'or", "golden globe"]}])

```

Після додавання консоль виводить ідентифікатори доданих документів:



```

{ _id: ObjectId('65b22777d7065689ca05d504'), name: 'Tim' },
{
  _id: ObjectId('65b240dad7065689ca05d505'),
  name: 'Sarah',
  age: 30,
  languages: [ 'english', 'spanish' ]
},
{
  _id: 987654,
  name: 'Jessica',
  age: 25,
  languages: [ 'english', 'ukrainian' ]
},
{
  _id: ObjectId('17f56abb34d8e543512da124'),
  name: 'Parker',
  age: 45,
  languages: [ 'english', 'italian', '' ]
}
]
test> db.users.insertMany([{"name": "Tarantino", "age": 60, "awards": ["oscar", "golden globe"]}, {"name": "Scorsese", "age": 70, "awards": ["oscar", "palme d'or", "golden globe"]}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65b2472cd7065689ca05d506'),
    '1': ObjectId('65b2472cd7065689ca05d507')
  }
}
test> |

```

Рисунок 8 – Результат виконання команди insertMany()

Є ще один спосіб додавання в БД документа, який включає два етапи: визначення документа ($document = (\{ \dots \})$) і власне його додавання:

```

document=({"name": "Bruno", "age": 32, "languages": ["spanish",
"french"]})
db.users.insertOne(document)

```

Можливо, не всім буде зручно вводити в один рядок усі пари ключів і властивостей. Але інтелектуальний інтерпретатор *MongoDB* на основі *javascript* дає змогу також вводити і багаторядкові команди. Якщо вираз не закінчено (з точки зору мови *JavaScript*), і ви натискаєте Enter, то введення наступної частини виразу автоматично переноситься на наступний рядок:

```

{
  _id: 987654,
  name: 'Jessica',
  age: 25,
  languages: [ 'english', 'ukrainian' ]
},
{
  _id: ObjectId('17f56abb34d8e543512da124'),
  name: 'Parker',
  age: 45,
  languages: [ 'english', 'italian', '' ]
}
]
test> db.users.insertMany([{"name": "Tarantino", "age": 60, "awards": ["oscar", "golden globe"]}, {"name": "Scorsese", "age": 70, "awards": ["oscar", "palme d'or", "golden globe"]}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65b2472cd7065689ca05d506'),
    '1': ObjectId('65b2472cd7065689ca05d507')
  }
}
test> document={"name": "Bruno", "age": 32, "languages": ["spanish", "french"]}
{ name: 'Bruno', age: 32, languages: [ 'spanish', 'french' ] }
test> db.users.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('65b247f6d7065689ca05d508')
}
test> |

```

Рисунок 9 – Багаторядкове введення в MongoDB

3.2 Завантаження даних із файлу

Дані для бази даних `mongodb` можна визначати у звичайному текстовому файлі, що досить зручно, оскільки ми можемо переносити або пересилати цей файл незалежно від бази даних `mongodb`. Наприклад, визначимо де-небудь на жорсткому диску файл `users.js` з таким вмістом:

```

db.users.insertMany([
{"name": "Ania", "age": 30, "languages": ["english", "polish"]},
{"name": "Astrid", "age": 29, "languages": ["swedish", "finnish"]},
{"name": "Anders", "age": 35, "languages": ["norwegian", "sami"]}
])

```

Тобто тут за допомогою методу `insertMany()` додаються три документи в колекцію `users`. Для завантаження файлу в поточну базу даних застосовується функція `load()`, в яку як параметр передається шлях до файлу:

```
load("C:/webpy/lb_MongoDB/users.js")
```


У цьому випадку передбачається, що файл розташовується за шляхом "C:/webpy/lb_MongoDB/users.js".

```

languages: [ 'english', 'italian', '' ]
}
]
test> db.users.insertMany([{"name": "Tarantino", "age": 60, "awards": ["oscar", "golden globe"]}, {"name": "Scorsese", "age": 70, "awards": ["oscar", "palme d'or", "golden globe"]}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65b2472cd7065689ca05d506'),
    '1': ObjectId('65b2472cd7065689ca05d507')
  }
}
test> document={"name": "Bruno", "age": 32, "languages": ["spanish", "french"]}
{ name: 'Bruno', age: 32, languages: [ 'spanish', 'french' ] }
test> db.users.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('65b247f6d7065689ca05d508')
}
test> load("C:\webpy\lb_MongoDB\users.js")
Uncaught:
SyntaxError: Bad character escape sequence. (1:27)

> 1 | load("C:\webpy\lb_MongoDB\users.js")
    |                                     ^
    2 |

test> load("C:/webpy/lb_MongoDB/users.js")
true
test>

```

Рисунок 10 – Результат завантаження даних з файлу

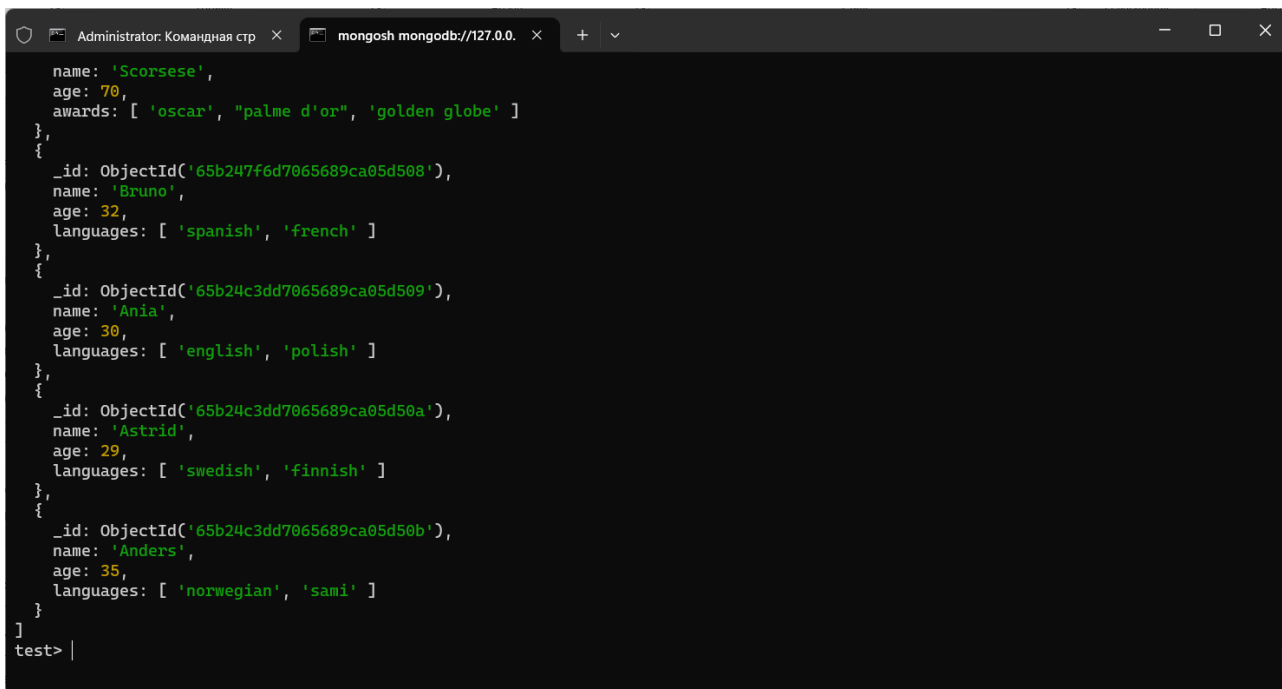
Виконання команди `db.users.find()` дозволяє нам подивитися зміст колекції.

4 Вибірка та фільтрація

4.1 Вибірка даних

Найпростішим способом вибірки документів із колекції є використання функції `find()`. Дія цієї функції багато в чому аналогічна звичайному запиту `SELECT * FROM Table`, який застосовується в `SQL` і який витягує всі рядки. Наприклад, щоб витягти всі документи з колекції `users`, створеної в минулій темі, ми можемо використовувати команду:

```
db.users.find()
```



```

name: 'Scorsese',
age: 70,
awards: [ 'oscar', 'palme d'or', 'golden globe' ]
},
{
  _id: ObjectId('65b247f6d7065689ca05d508'),
  name: 'Bruno',
  age: 32,
  languages: [ 'spanish', 'french' ]
},
{
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Ania',
  age: 30,
  languages: [ 'english', 'polish' ]
},
{
  _id: ObjectId('65b24c3dd7065689ca05d50a'),
  name: 'Astrid',
  age: 29,
  languages: [ 'swedish', 'finnish' ]
},
{
  _id: ObjectId('65b24c3dd7065689ca05d50b'),
  name: 'Anders',
  age: 35,
  languages: [ 'norwegian', 'sami' ]
}
]
test> |

```

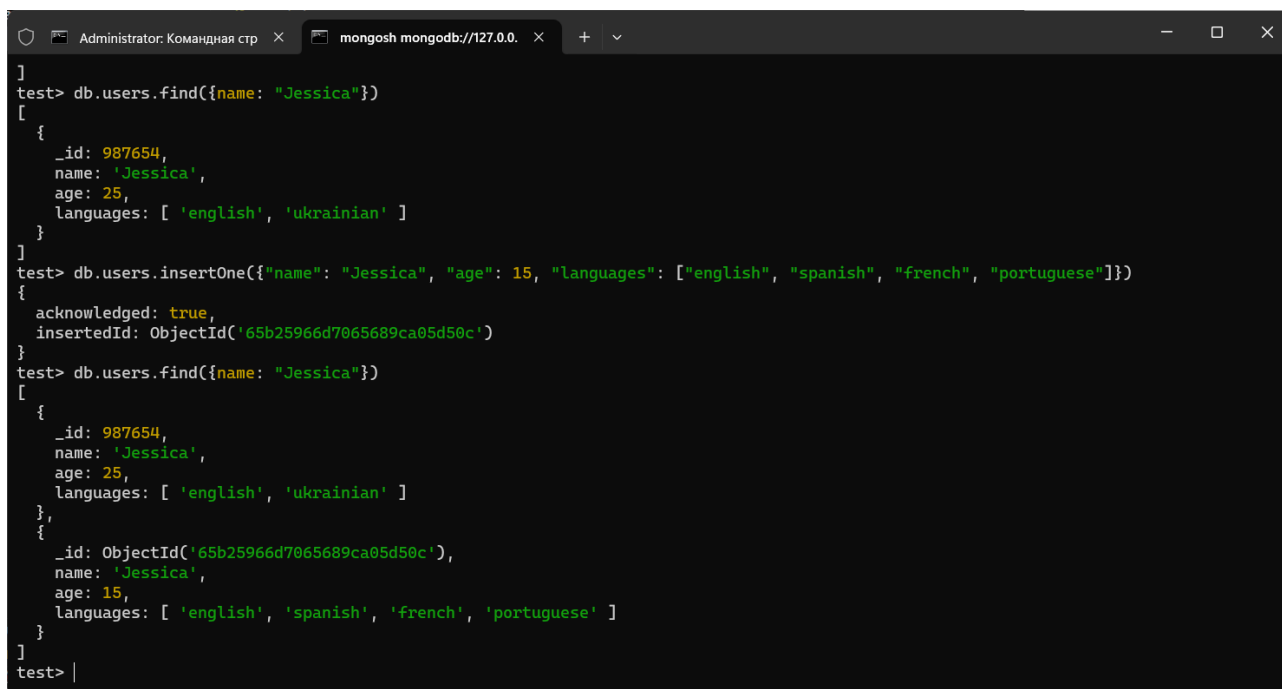
Рисунок 11 - Витяг даних з MongoDB

4.2 Фільтрація даних

Однак що, якщо нам треба отримати не всі документи, а тільки ті, які задовольняють певній вимозі. Наприклад, ми раніше в базу додали різні документи. Виведемо всі документи, у яких ім'я= Jessica:

```
db.users.find({name: "Jessica"})
```

Такий запит виведе нам два документи, у яких name= Jessica.



```

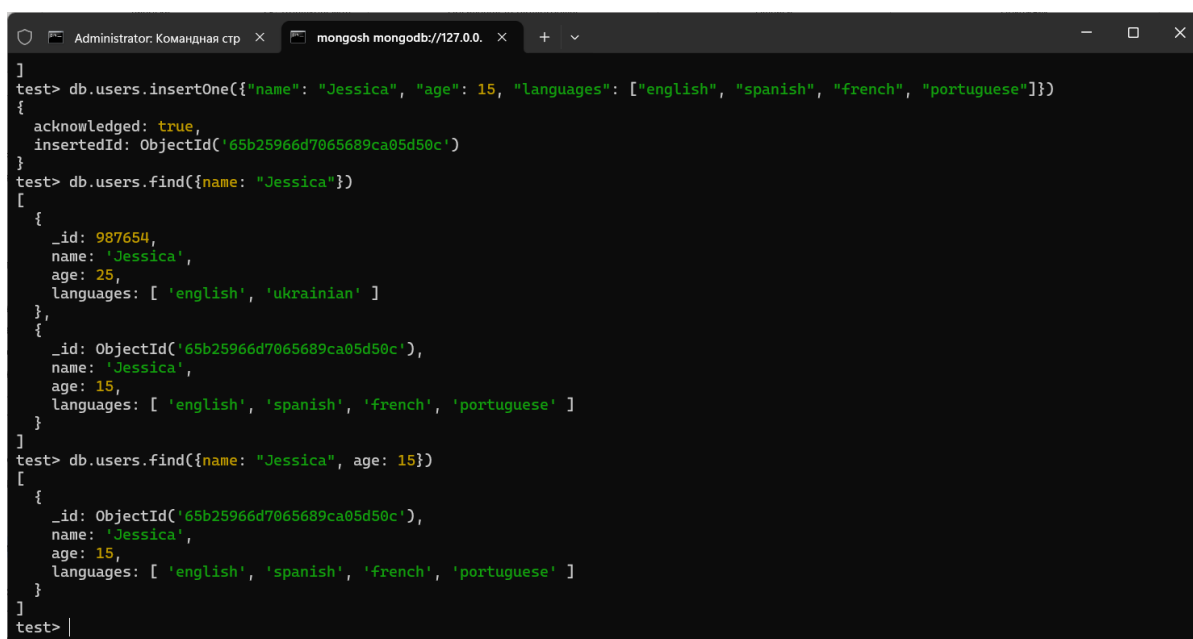
]
test> db.users.find({name: "Jessica"})
[
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  }
]
test> db.users.insertOne({"name": "Jessica", "age": 15, "languages": ["english", "spanish", "french", "portuguese"]})
{
  acknowledged: true,
  insertedId: ObjectId('65b25966d7065689ca05d50c')
}
test> db.users.find({name: "Jessica"})
[
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> |

```

Рисунок 12 – Результат фільтрації даних за умови ім'я= Jessica

Тепер складніший запит: нам треба вивести ті об'єкти, у яких name= Jessica і одночасно age=15. Тобто мовою SQL це могло б виглядати так: `SELECT * FROM Table WHERE Name='Jessica' AND Age=15`. Цьому критерію в нас відповідає останній доданий об'єкт. Тоді ми можемо написати такий запит:

```
db.users.find({name: "Jessica", age: 15})
```



```

]
test> db.users.insertOne({"name": "Jessica", "age": 15, "languages": ["english", "spanish", "french", "portuguese"]})
{
  acknowledged: true,
  insertedId: ObjectId('65b25966d7065689ca05d50c')
}
test> db.users.find({name: "Jessica"})
[
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> db.users.find({name: "Jessica", age: 15})
[
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> |

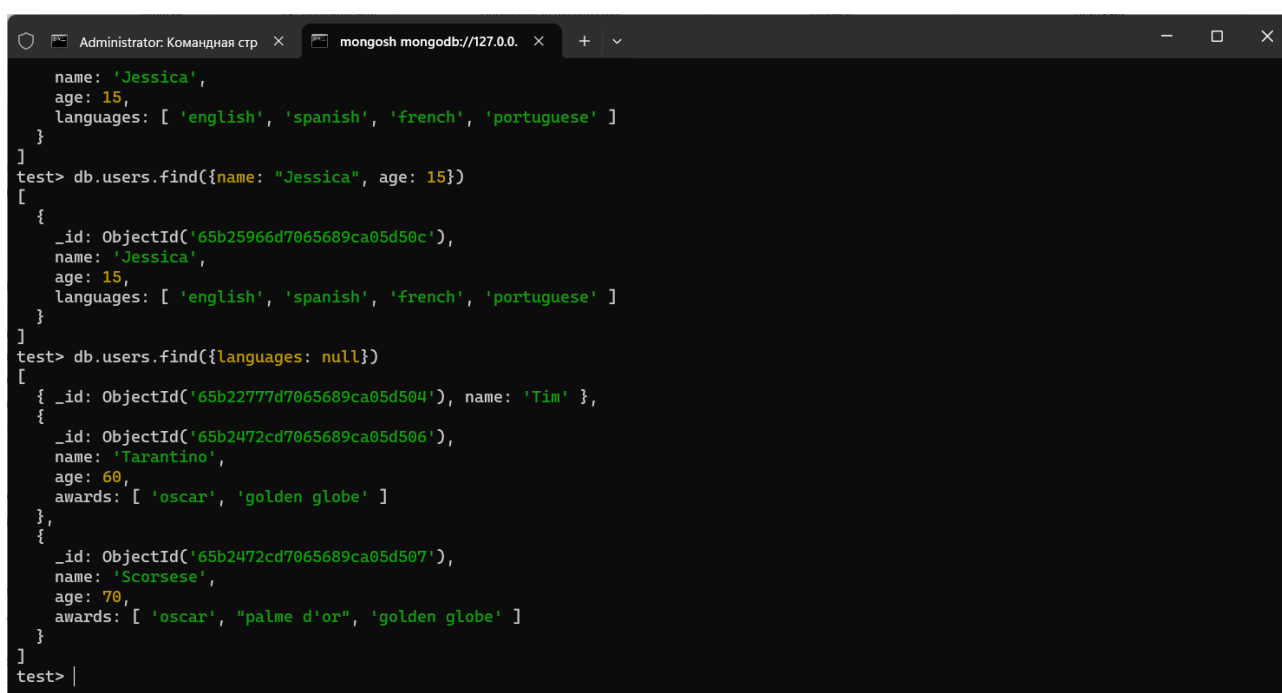
```

Рисунок 13 – Результат фільтрації даних за складною умовою

4.3 Фільтрація за відсутніми властивостями

Якісь документи можуть мати певну властивість, інші можуть її не мати. Що якщо ми хочемо отримати документи, в яких відсутня певна властивість? У цьому випадку для властивості передається значення *null*. Наприклад, знайдемо всі документи, де відсутня властивість *languages*:

```
db.users.find({languages: null})
```



```

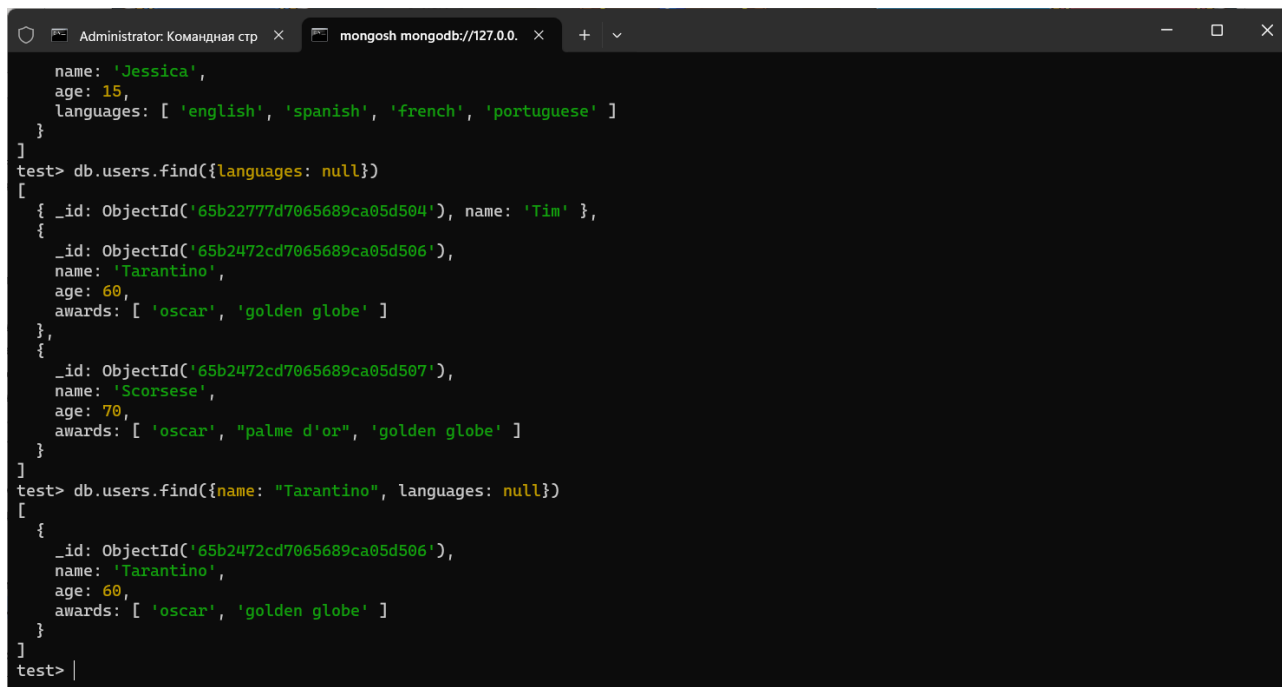
Administrator: Командная строка  mongosh mongodb://127.0.0.1:27017/
> use test
test> db.users.find({name: "Jessica", age: 15})
[
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> db.users.find({languages: null})
[
  {
    _id: ObjectId('65b22777d7065689ca05d504'),
    name: 'Tim'
  },
  {
    _id: ObjectId('65b2472cd7065689ca05d506'),
    name: 'Tarantino',
    age: 60,
    awards: [ 'oscar', 'golden globe' ]
  },
  {
    _id: ObjectId('65b2472cd7065689ca05d507'),
    name: 'Scorsese',
    age: 70,
    awards: [ 'oscar', 'palme d'or', 'golden globe' ]
  }
]
test> |

```

Рисунок 14 – Результати фільтрації за відсутніми властивостями

Або знайдемо всі документи, де name="Tarantino", але властивість languages не визначена.

```
db.users.find({name: "Tarantino", languages: null})
```



```

name: 'Jessica',
age: 15,
languages: [ 'english', 'spanish', 'french', 'portuguese' ]
}
]
test> db.users.find({languages: null})
[
  { _id: ObjectId('65b2277d7065689ca05d504'), name: 'Tim' },
  {
    _id: ObjectId('65b2472cd7065689ca05d506'),
    name: 'Tarantino',
    age: 60,
    awards: [ 'oscar', 'golden globe' ]
  },
  {
    _id: ObjectId('65b2472cd7065689ca05d507'),
    name: 'Scorsese',
    age: 70,
    awards: [ 'oscar', 'palme d'or', 'golden globe' ]
  }
]
test> db.users.find({name: "Tarantino", languages: null})
[
  {
    _id: ObjectId('65b2472cd7065689ca05d506'),
    name: 'Tarantino',
    age: 60,
    awards: [ 'oscar', 'golden globe' ]
  }
]
test> |

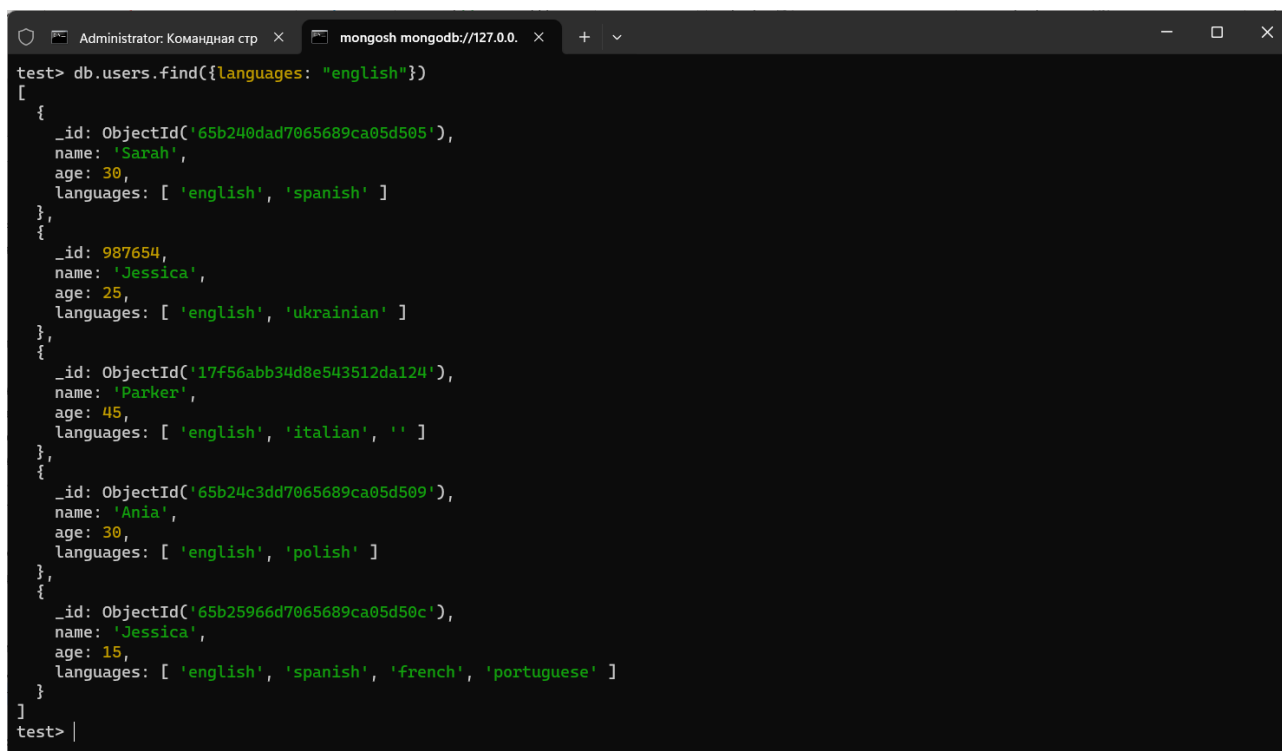
```

Рисунок 15 – Результати складної фільтрації за відсутніми властивостями

4.4 Фільтрація за елементами масиву

Також нескладно відшукати за елементом у масиві. Наприклад, такий запит виводить усі документи, у яких у масиві *languages* є *"english"*:

```
db.users.find({languages: "english"})
```



```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1  x  +  v
test> db.users.find({languages: "english"})
[
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  },
  {
    _id: ObjectId('17f56abb34d8e543512da124'),
    name: 'Parker',
    age: 45,
    languages: [ 'english', 'italian', '' ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> |

```

Рисунок 16 – Результат фільтрації за елементами масиву

Ускладнимо запит і отримаємо ті документи, у яких у масиві *languages* одночасно дві мови: *"english"* і *"german"*:

```
db.users.find({languages: ["english", "german"]})
```

Причому саме в цьому порядку, де *"english"* визначено першим, а *"german"* - другим.

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
{
  "_id": ObjectId('65b240dad7065689ca05d505'),
  "name": 'Sarah',
  "age": 30,
  "languages": [ 'english', 'spanish' ]
},
{
  "_id": 987654,
  "name": 'Jessica',
  "age": 25,
  "languages": [ 'english', 'ukrainian' ]
},
{
  "_id": ObjectId('17f56abb34d8e543512da124'),
  "name": 'Parker',
  "age": 45,
  "languages": [ 'english', 'italian', '' ]
},
{
  "_id": ObjectId('65b24c3dd7065689ca05d509'),
  "name": 'Ania',
  "age": 30,
  "languages": [ 'english', 'polish' ]
},
{
  "_id": ObjectId('65b25966d7065689ca05d50c'),
  "name": 'Jessica',
  "age": 15,
  "languages": [ 'english', 'spanish', 'french', 'portuguese' ]
}
]
test> db.users.find({languages: ["english", "german"]})
test> |

```

Рисунок 17 – Відсутній результат складної фільтрації

Тепер виведемо всі документи, в яких "french" у масиві languages знаходиться на другому місці:

```
db.users.find({"languages.1": "french"})
```

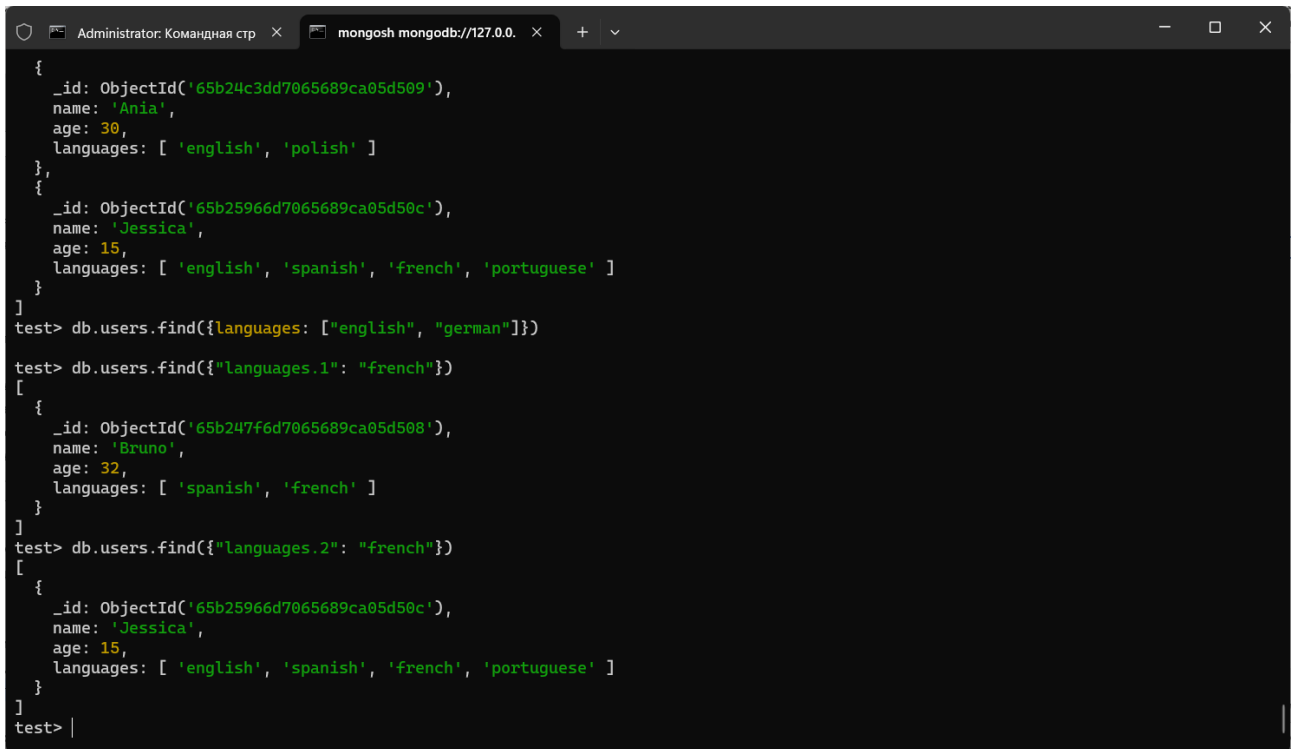
```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
age: 25,
languages: [ 'english', 'ukrainian' ]
},
{
  "_id": ObjectId('17f56abb34d8e543512da124'),
  "name": 'Parker',
  "age": 45,
  "languages": [ 'english', 'italian', '' ]
},
{
  "_id": ObjectId('65b24c3dd7065689ca05d509'),
  "name": 'Ania',
  "age": 30,
  "languages": [ 'english', 'polish' ]
},
{
  "_id": ObjectId('65b25966d7065689ca05d50c'),
  "name": 'Jessica',
  "age": 15,
  "languages": [ 'english', 'spanish', 'french', 'portuguese' ]
}
]
test> db.users.find({languages: ["english", "german"]})
test> db.users.find({"languages.1": "french"})
[
  {
    "_id": ObjectId('65b247f6d7065689ca05d508'),
    "name": 'Bruno',
    "age": 32,
    "languages": [ 'spanish', 'french' ]
  }
]
test> |

```

Риснуок 18 – Результат фільтрації по значенню за позицією у масиві

Зверніть увагу, що "languages.1" надає складну властивість і тому береться в лапки. Відповідно якщо нам треба вивести документи, де french на третьому місці (наприклад, ["german", "english", "french"]), то замість 1 ставимо 2: "languages.2".



```

{
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Ania',
  age: 30,
  languages: [ 'english', 'polish' ]
},
{
  _id: ObjectId('65b25966d7065689ca05d50c'),
  name: 'Jessica',
  age: 15,
  languages: [ 'english', 'spanish', 'french', 'portuguese' ]
}
]
test> db.users.find({languages: ["english", "german"]})
test> db.users.find({"languages.1": "french"})
[
  {
    _id: ObjectId('65b247f6d7065689ca05d508'),
    name: 'Bruno',
    age: 32,
    languages: [ 'spanish', 'french' ]
  }
]
test> db.users.find({"languages.2": "french"})
[
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> |

```

Риснуок 19 – Результат фільтрації по значенню за позицією у масиві

Розглянемо складніший приклад, де елемент масиву представляє складний об'єкт. Для цього додамо у нас у базу даних такі документи:

```

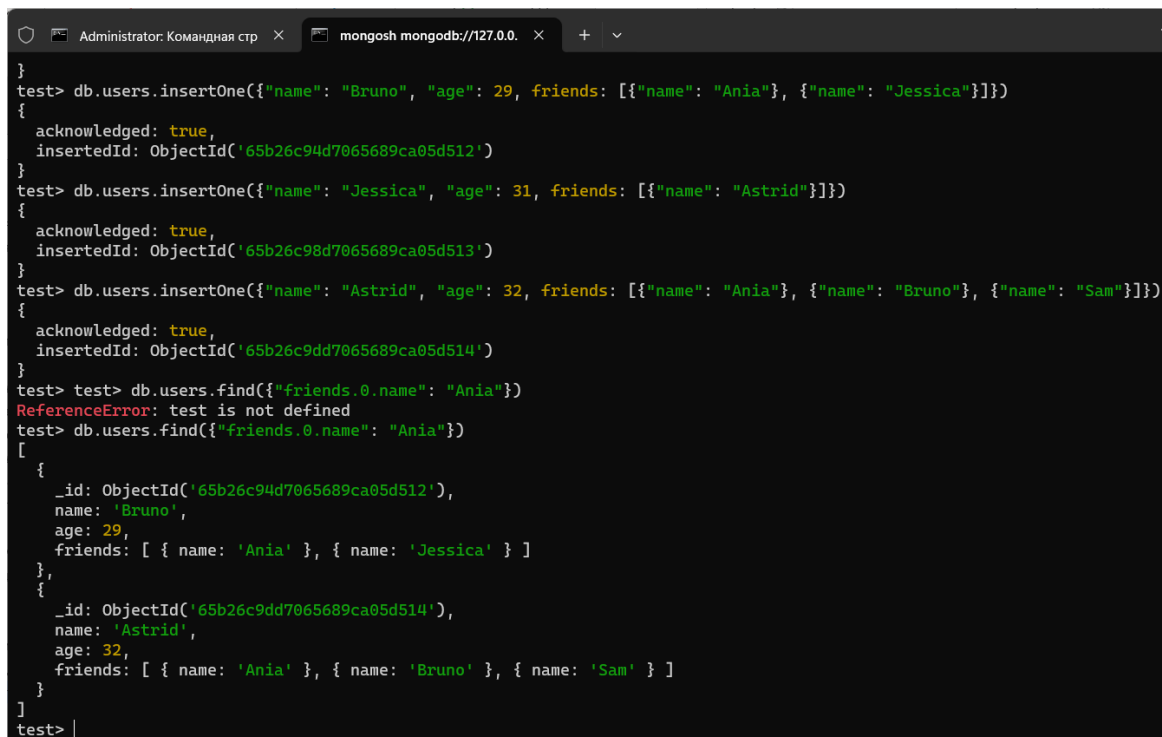
db.users.insertOne({"name": "Ania", "age": 28, friends: [{"name": "Bruno"}, {"name": "Jessica"}]})
db.users.insertOne({"name": "Bruno", "age": 29, friends: [{"name": "Ania"}, {"name": "Jessica"}]})
db.users.insertOne({"name": "Jessica", "age": 31, friends: [{"name": "Astrid"}]})
db.users.insertOne({"name": "Astrid", "age": 32, friends: [{"name": "Ania"}, {"name": "Bruno"}, {"name": "Sam"}]})

```


Виберемо всі документи, де в масиві *friends* властивість *name* першого елемента дорівнює "Ania":

```
test> db.users.find({"friends.0.name": "Ania"})
```

Консольний вивід:



```

Administrator: Командная стр  mongosh mongodb://127.0.0.1:27017
test> db.users.insertOne({"name": "Bruno", "age": 29, friends: [{name: "Ania"}, {name: "Jessica"}]})
{ acknowledged: true, insertedId: ObjectId('65b26c94d7065689ca05d512') }
test> db.users.insertOne({"name": "Jessica", "age": 31, friends: [{name: "Astrid"}]})
{ acknowledged: true, insertedId: ObjectId('65b26c98d7065689ca05d513') }
test> db.users.insertOne({"name": "Astrid", "age": 32, friends: [{name: "Ania"}, {name: "Bruno"}, {name: "Sam"}]})
{ acknowledged: true, insertedId: ObjectId('65b26c9dd7065689ca05d514') }
test> test> db.users.find({"friends.0.name": "Ania"})
ReferenceError: test is not defined
test> db.users.find({"friends.0.name": "Ania"})
[
  {
    _id: ObjectId('65b26c94d7065689ca05d512'),
    name: 'Bruno',
    age: 29,
    friends: [ { name: 'Ania' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b26c9dd7065689ca05d514'),
    name: 'Astrid',
    age: 32,
    friends: [ { name: 'Ania' }, { name: 'Bruno' }, { name: 'Sam' } ]
  }
]
test>

```

4.5 Проекція

Документ може мати безліч полів, але не всі ці поля нам можуть бути потрібні й важливі під час запиту. І в цьому випадку ми можемо включити у вибірку тільки потрібні поля, використавши проекцію. Наприклад, виведемо тільки порцію інформації, наприклад, значення полів "age" у всіх документах, у яких *name=Ania*:

```
db.users.find({name: "Ania"}, {age: 1})
```

Використання одиниці як параметра *{age: 1}* вказує, що запит має повернути тільки зміст властивості *age*.

```

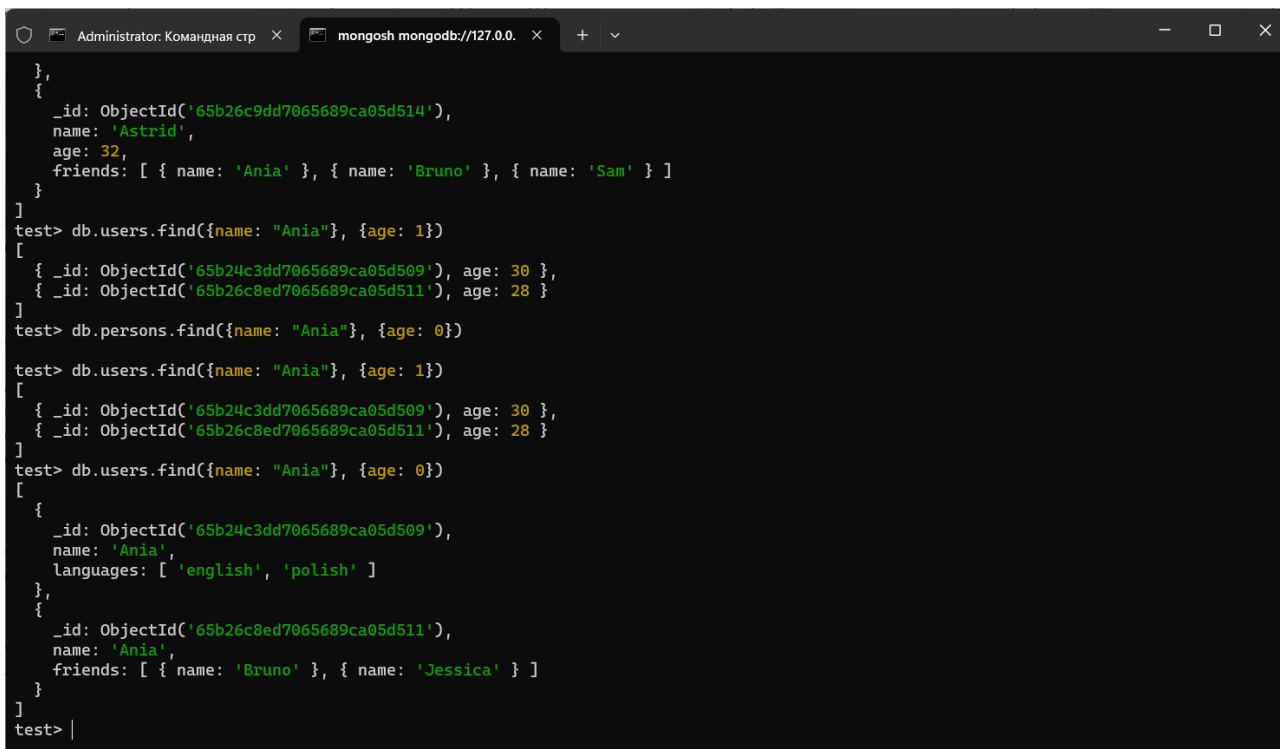
}
test> db.users.insertOne({"name": "Jessica", "age": 31, friends: [{name: "Astrid"}]})
{
  acknowledged: true,
  insertedId: ObjectId('65b26c98d7065689ca05d513')
}
test> db.users.insertOne({"name": "Astrid", "age": 32, friends: [{name: "Ania"}, {"name": "Bruno"}, {"name": "Sam"}]})
{
  acknowledged: true,
  insertedId: ObjectId('65b26c9dd7065689ca05d514')
}
test> test> db.users.find({"friends.0.name": "Ania"})
ReferenceError: test is not defined
test> db.users.find({"friends.0.name": "Ania"})
[
  {
    _id: ObjectId('65b26c94d7065689ca05d512'),
    name: 'Bruno',
    age: 29,
    friends: [ { name: 'Ania' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b26c9dd7065689ca05d514'),
    name: 'Astrid',
    age: 32,
    friends: [ { name: 'Ania' }, { name: 'Bruno' }, { name: 'Sam' } ]
  }
]
test> db.users.find({name: "Ania"}, {age: 1})
[
  { _id: ObjectId('65b24c3dd7065689ca05d509'), age: 30 },
  { _id: ObjectId('65b26c8ed7065689ca05d511'), age: 28 }
]
test> |

```

І зворотна ситуація: ми хочемо знайти всі поля документа крім властивості *age*. У цьому разі як параметр вказуємо 0:

```
db.persons.find({name: "Ania"}, {age: 0})
```

При цьому треба враховувати, що навіть якщо ми зазначимо, що ми хочемо отримати тільки поле *name*, поле *_id* також буде включено в результуючу вибірку.



```

},
{
  _id: ObjectId('65b26c9dd7065689ca05d514'),
  name: 'Astrid',
  age: 32,
  friends: [ { name: 'Ania' }, { name: 'Bruno' }, { name: 'Sam' } ]
}
]
test> db.users.find({name: "Ania"}, {age: 1})
[
  { _id: ObjectId('65b24c3dd7065689ca05d509'), age: 30 },
  { _id: ObjectId('65b26c8ed7065689ca05d511'), age: 28 }
]
test> db.persons.find({name: "Ania"}, {age: 0})

test> db.users.find({name: "Ania"}, {age: 1})
[
  { _id: ObjectId('65b24c3dd7065689ca05d509'), age: 30 },
  { _id: ObjectId('65b26c8ed7065689ca05d511'), age: 28 }
]
test> db.users.find({name: "Ania"}, {age: 0})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  }
]
test> |

```

Тому, якщо ми не хочемо бачити це поле у вибірці, то треба явно вказати:
`{"_id":0}`

Альтернативно замість *1* і *0* можна використовувати *true* і *false*:

```
db.users.find({name: "Ania"}, {age: true, _id: false})
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
  _id: ObjectId('65b26c9dd7065689ca05d514'),
  name: 'Astrid',
  age: 32,
  friends: [ { name: 'Ania' }, { name: 'Bruno' }, { name: 'Sam' } ]
]
test> db.users.find({name: "Ania"}, {age: 1})
[
  { _id: ObjectId('65b24c3dd7065689ca05d509'), age: 30 },
  { _id: ObjectId('65b26c8ed7065689ca05d511'), age: 28 }
]
test> db.persons.find({name: "Ania"}, {age: 0})

test> db.users.find({name: "Ania"}, {age: 1})
[
  { _id: ObjectId('65b24c3dd7065689ca05d509'), age: 30 },
  { _id: ObjectId('65b26c8ed7065689ca05d511'), age: 28 }
]
test> db.users.find({name: "Ania"}, {age: 0})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  }
]
test> db.users.find({name: "Ania"}, {age: true, _id: false})
[ { age: 30 }, { age: 28 } ]
test>

```

Якщо ми не хочемо при цьому конкретизувати вибірку, а хочемо вивести всі документи, то можна залишити перші фігурні дужки порожніми:

```
db.users.find({}, {age: 1, _id: 0})
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
test> db.users.find({name: "Ania"}, {age: 1})
[
  { _id: ObjectId('65b24c3dd7065689ca05d509'), age: 30 },
  { _id: ObjectId('65b26c8ed7065689ca05d511'), age: 28 }
]
test> db.users.find({name: "Ania"}, {age: 0})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  }
]
test> db.users.find({name: "Ania"}, {age: true, _id: false})
[ { age: 30 }, { age: 28 } ]
test> db.users.find({}, {age: 1, _id: 0})
[
  {}, { age: 30 },
  { age: 25 }, { age: 45 },
  { age: 60 }, { age: 70 },
  { age: 32 }, { age: 30 },
  { age: 29 }, { age: 35 },
  { age: 15 }, { age: 28 },
  { age: 29 }, { age: 31 },
  { age: 32 }, { age: 28 },
  { age: 29 }, { age: 31 },
  { age: 32 }
]
test>

```

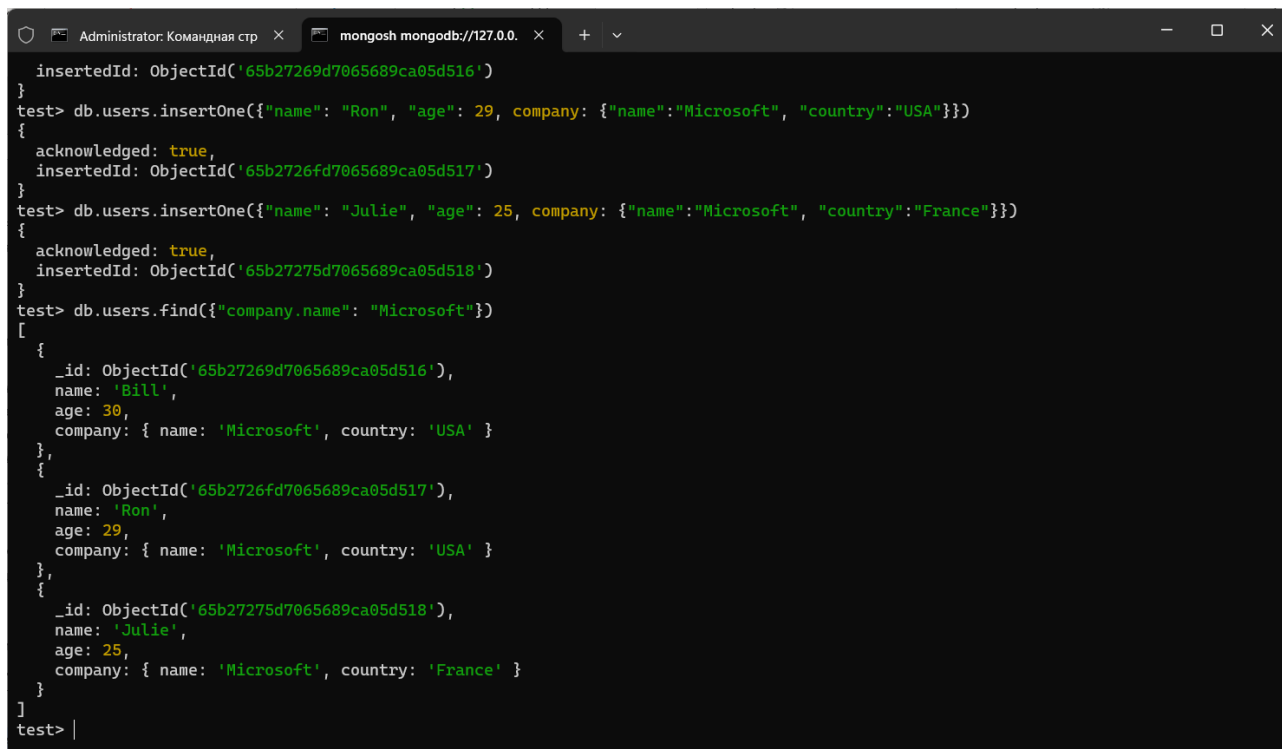
4.6 Запит до вкладених об'єктів

Попередні запити застосовувалися до простих об'єктів. Але документи можуть бути дуже складними за структурою. Наприклад, додамо в колекцію *users* такі документи:

```
db.users.insertOne({"name": "Stive", "age": 28, company:
{"name": "Apple", "country": "USA"}})
db.users.insertOne({"name": "Bill", "age": 30, company:
{"name": "Microsoft", "country": "USA"}})
db.users.insertOne({"name": "Ron", "age": 29, company:
{"name": "Microsoft", "country": "USA"}})
db.users.insertOne({"name": "Julie", "age": 25, company:
{"name": "Microsoft", "country": "France"}})
```

Тут визначається вкладений об'єкт із ключем *company*. І щоб знайти всі документи, у яких у ключі *company* вкладена властивість *name=microsoft*, нам треба використати оператор крапку:

```
db.users.find({"company.name": "Microsoft"})
```



```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.  ×  +  ▾
insertedId: ObjectId('65b27269d7065689ca05d516')
}
test> db.users.insertOne({"name": "Ron", "age": 29, company: {"name":"Microsoft", "country":"USA"}})
{
  acknowledged: true,
  insertedId: ObjectId('65b2726fd7065689ca05d517')
}
test> db.users.insertOne({"name": "Julie", "age": 25, company: {"name":"Microsoft", "country":"France"}})
{
  acknowledged: true,
  insertedId: ObjectId('65b27275d7065689ca05d518')
}
test> db.users.find({"company.name": "Microsoft"})
[
  {
    _id: ObjectId('65b27269d7065689ca05d516'),
    name: 'Bill',
    age: 30,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b2726fd7065689ca05d517'),
    name: 'Ron',
    age: 29,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b27275d7065689ca05d518'),
    name: 'Julie',
    age: 25,
    company: { name: 'Microsoft', country: 'France' }
  }
]
test> |

```

Рисунок 20 – Результат виконання запиту до вкладених даних

4.7 Використання JavaScript

Крім виконання запитів до бази даних ми можемо виконувати вирази *JavaScript*. Наприклад, ми можемо створити якусь функцію і застосовувати її:

```
function sqrt(n) { return n*n; }
sqrt(5)
```

Консольний вивід:

```
Administrator: Командная строка  mongosh mongodb://127.0.0.1:27021/
test> db.users.insertOne({"name": "Ron", "age": 29, company: {"name": "Microsoft", "country": "USA"}})
{
  acknowledged: true,
  insertedId: ObjectId('65b2726fd7065689ca05d517')
}
test> db.users.insertOne({"name": "Julie", "age": 25, company: {"name": "Microsoft", "country": "France"}})
{
  acknowledged: true,
  insertedId: ObjectId('65b27275d7065689ca05d518')
}
test> db.users.find({"company.name": "Microsoft"})
[
  {
    _id: ObjectId('65b27269d7065689ca05d516'),
    name: 'Bill',
    age: 30,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b2726fd7065689ca05d517'),
    name: 'Ron',
    age: 29,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b27275d7065689ca05d518'),
    name: 'Julie',
    age: 25,
    company: { name: 'Microsoft', country: 'France' }
  }
]
test> function sqrt(n) { return n*n; }
[Function: sqrt]
test> |
```

І подібні функції та вирази *JavaScript* ми можемо застосовувати в запитах до БД. Наприклад, знайдемо всі документи, де поле *age* дорівнює $\text{sqrt}(5)+3$:

```
test> db.users.find({age: sqrt(5)+3})
```

```
Administrator: Командная строка  mongosh mongodb://127.0.0.1:27021/
age: 29,
company: { name: 'Microsoft', country: 'USA' }
},
{
  _id: ObjectId('65b27275d7065689ca05d518'),
  name: 'Julie',
  age: 25,
  company: { name: 'Microsoft', country: 'France' }
}
]
test> function sqrt(n) { return n*n; }
[Function: sqrt]
test> db.users.find({age: sqrt(5)+3})
[
  {
    _id: ObjectId('65b269c8d7065689ca05d50d'),
    name: 'Bob',
    age: 28,
    friends: [ { name: 'Tim' }, { name: 'Tom' } ]
  },
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b27256d7065689ca05d515'),
    name: 'Stive',
    age: 28,
    company: { name: 'Apple', country: 'USA' }
  }
]
test> |
```

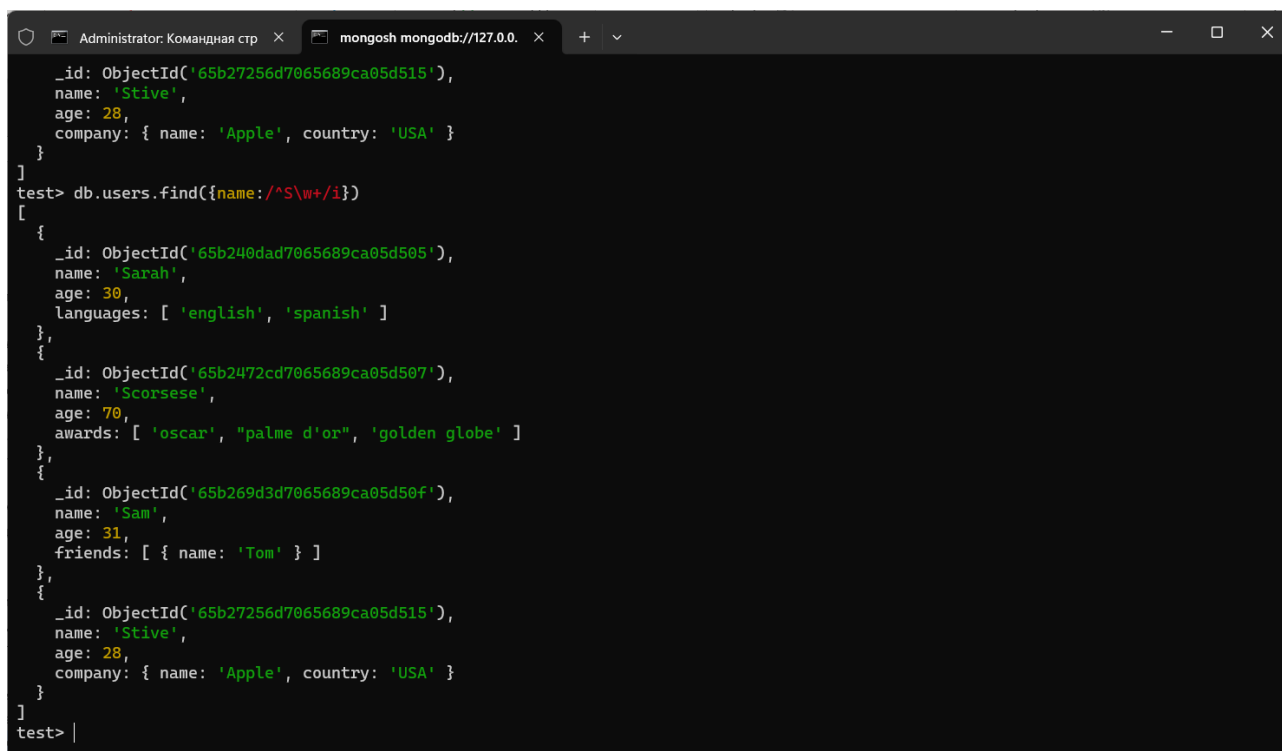
Рисунок 21 – Результат використання *JavaScript*

4.8 Використання регулярних виразів

Ще однією чудовою можливістю при побудові запитів є використання регулярних виразів. Наприклад, знайдемо всі документи, у яких значення ключа *name* починається з літери S:

```
db.users.find({name:/^S\w+/i})
```

Консольний вивід:



```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.1:27017/  ×  +  ▾
[
  {
    _id: ObjectId('65b27256d7065689ca05d515'),
    name: 'Stive',
    age: 28,
    company: { name: 'Apple', country: 'USA' }
  }
]
test> db.users.find({name:/^S\w+/i})
[
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: ObjectId('65b2472cd7065689ca05d507'),
    name: 'Scorsese',
    age: 70,
    awards: [ 'oscar', 'palme d\'or', 'golden globe' ]
  },
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Sam',
    age: 31,
    friends: [ { name: 'Tom' } ]
  },
  {
    _id: ObjectId('65b27256d7065689ca05d515'),
    name: 'Stive',
    age: 28,
    company: { name: 'Apple', country: 'USA' }
  }
]
test> |

```

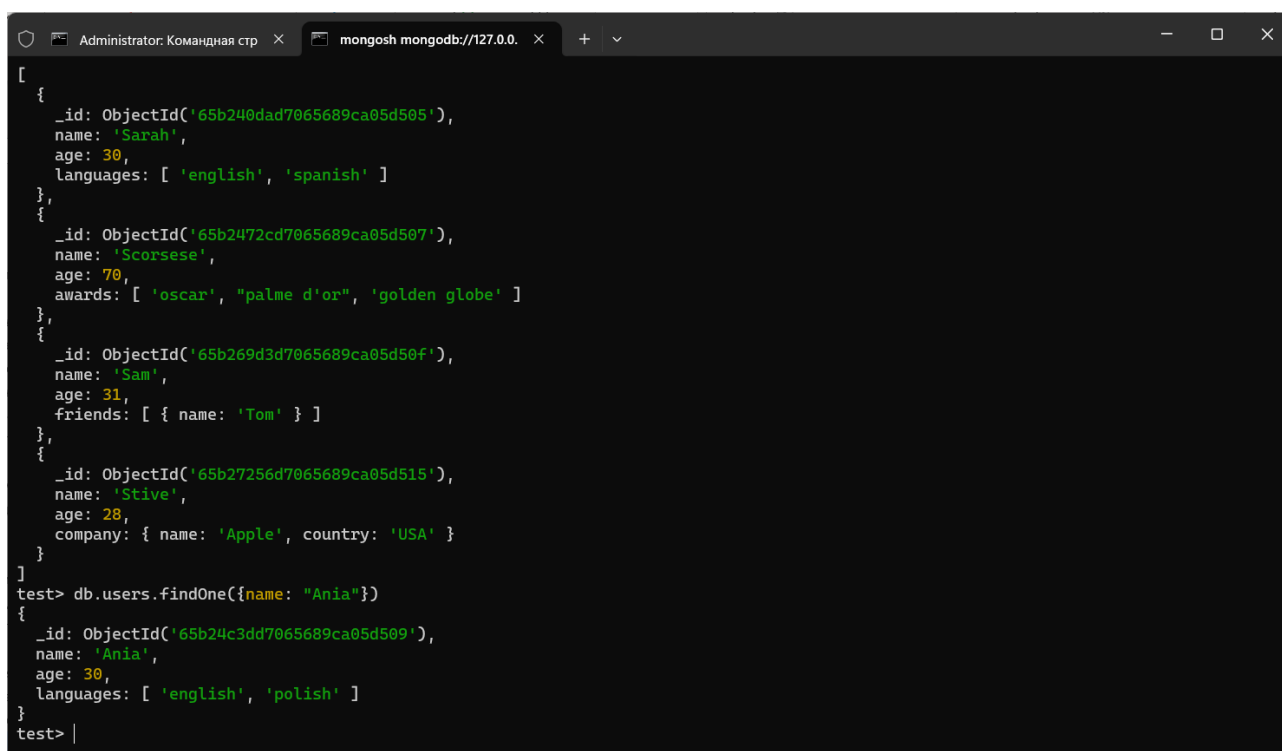
Рисунок 22 – Результат використання регулярних виразів

4.9 Пошук одиночного документа

Якщо всі документи витягуються функцією *find()*, то одиночний документ витягується функцією *findOne()*

Наприклад, виберемо один елемент з *name="Ania"*:


```
test> db.users.findOne({name: "Ania"})
```



```
[
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: ObjectId('65b2472cd7065689ca05d507'),
    name: 'Scorsese',
    age: 70,
    awards: [ 'oscar', 'palme d'or', 'golden globe' ]
  },
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Sam',
    age: 31,
    friends: [ { name: 'Tom' } ]
  },
  {
    _id: ObjectId('65b27256d7065689ca05d515'),
    name: 'Stive',
    age: 28,
    company: { name: 'Apple', country: 'USA' }
  }
]
test> db.users.findOne({name: "Ania"})
{
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Ania',
  age: 30,
  languages: [ 'english', 'polish' ]
}
test> |
```

4.10 Курсори

Результат вибірки, одержуваної за допомогою функції *find()*, називається курсором. За необхідності ми можемо передати курсор в окрему змінну:

```
var cursor = db.users.find()
```

Курсори інкапсулюють у собі набори об'єктів, які отримують із БД. Використовуючи синтаксис мови *javascript* і методи курсорів, ми можемо вивести отримані документи на екран і якось їх обробити. Наприклад:

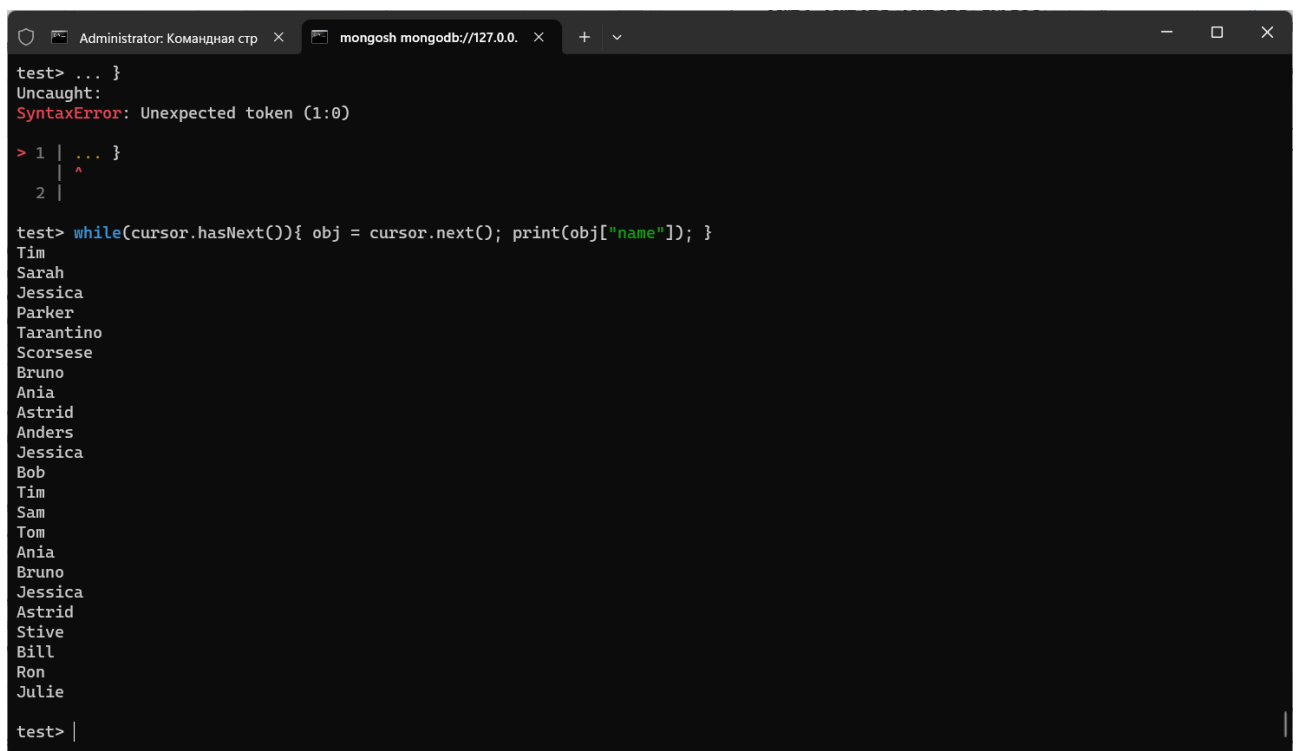
```
var cursor = db.users.find()

while(cursor.hasNext()){
  obj = cursor.next();
  print(obj["name"]);
}
```

Курсор має метод *hasNext*, який показує під час перебору, чи є ще в наборі документ. А метод *next* витягує поточний документ і переміщує курсор до наступного документа в наборі. У підсумку у змінній *obj* опиняється документ, до полів якого ми можемо отримати доступ.

```
test> var cursor = db.users.find()

test> while(cursor.hasNext()){
...   obj = cursor.next();
...   print(obj["name"]);
... }
```



The screenshot shows a terminal window titled 'mongosh mongodb://127.0.0.1'. The user enters a `while` loop to iterate over the `users` collection. The output displays a list of names: Tim, Sarah, Jessica, Parker, Tarantino, Scorsese, Bruno, Ania, Astrid, Anders, Jessica, Bob, Tim, Sam, Tom, Ania, Bruno, Jessica, Astrid, Stive, Bill, Ron, and Julie. The cursor is then reset to the beginning of the collection.

```
test> ... }
Uncaught:
SyntaxError: Unexpected token (1:0)

> 1 | ... }
    | ^
    2 |

test> while(cursor.hasNext()){ obj = cursor.next(); print(obj["name"]); }
Tim
Sarah
Jessica
Parker
Tarantino
Scorsese
Bruno
Ania
Astrid
Anders
Jessica
Bob
Tim
Sam
Tom
Ania
Bruno
Jessica
Astrid
Stive
Bill
Ron
Julie
test> |
```

Рисунок 22 – Результат роботи

Також для перебору документів у курсорі як альтернативу ми можемо використовувати конструкцію ітератора *javascript* - *forEach*:

```
var cursor = db.users.find()
cursor.forEach(function(obj){ print(obj.name); })
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1  x  +  v
Astrid
Stive
Bill
Ron
Julie

test> var cursor = db.users.find()

test> cursor.forEach(function(obj){ print(obj.name); })
Tim
Sarah
Jessica
Parker
Tarantino
Scorsese
Bruno
Ania
Astrid
Anders
Jessica
Bob
Tim
Sam
Tom
Ania
Bruno
Jessica
Astrid
Stive
Bill
Ron
Julie

test> |

```

Рисунок 23 – Результат виборки за допомогою forEach()

5 Пагінація та сортування

MongoDB представляє низку функцій, які допомагають керувати вибіркою з БД.

5.1 Функції `limit()`, `skip()`, `sort()`

Одна з них - функція `limit()`. Вона задає максимально допустиму кількість одержуваних документів. Кількість передається у вигляді числового параметра. Наприклад, обмежимо вибірку трьома документами:

```
db.users.find().limit(3)
```

У цьому разі ми отримаємо перші три документи (якщо в колекції 3 і більше документів). Але що, якщо ми хочемо зробити вибірку не спочатку, а пропустивши якусь кількість документів? У цьому нам допоможе функція `skip()`. Наприклад, пропустимо перші три записи:

```
db.users.find().skip(3)
```

Комбінуючи обидві функції, ми можемо отримати певну кількість документів, починаючи з певного документа. Наприклад, виберемо документи з 4 по 6:

```
db.users.find().skip(3).limit(3)
```

MongoDB надає можливості відсортувати отриманий з БД набір даних за допомогою функції `sort()`. Передаючи в цю функцію значення 1 або -1, ми можемо вказати в якому порядку сортувати: за зростанням (1) або за спаданням (-1). Багато в чому ця функція за дією аналогічна виразу *ORDER BY* у *SQL*. Наприклад, сортування за зростанням за полем *name*:

```
db.users.find().sort({name: 1})
```

Наприклад, виведемо з БД тільки значення поля *"name"*, відсортувавши їх за зростанням:

```
test> db.users.find({}, {name:1, _id: 0}).sort({name: 1})
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1  x  +  v
Ania
Astrid
Anders
Jessica
Bob
Tim
Sam
Tom
Ania
Bruno
Jessica
Astrid
Stive
Bill
Ron
Julie

test> db.users.find({}, {name:1, _id: 0}).sort({name: 1})
[
  { name: 'Anders' }, { name: 'Ania' },
  { name: 'Ania' }, { name: 'Astrid' },
  { name: 'Astrid' }, { name: 'Bill' },
  { name: 'Bob' }, { name: 'Bruno' },
  { name: 'Bruno' }, { name: 'Jessica' },
  { name: 'Jessica' }, { name: 'Jessica' },
  { name: 'Julie' }, { name: 'Parker' },
  { name: 'Ron' }, { name: 'Sam' },
  { name: 'Sarah' }, { name: 'Scorsese' },
  { name: 'Stive' }, { name: 'Tarantino' }
]
Type "it" for more
test> it
[ { name: 'Tim' }, { name: 'Tim' }, { name: 'Tom' } ]
test> |

```

Рисунок 24 – Результат сортування даних

5.2 Оператор \$slice

\$slice є певною мірою комбінацією функцій *limit()* і *skip()*. Але на відміну від них *\$slice* може працювати з масивами.

Оператор *\$slice* має дві форми:

- *\$slice: limit*
- *\$slice: skip, limit*

Параметр *limit* вказує на загальну кількість документів, що повертаються. Параметр *skip* вказує на зміщення щодо початку (як функція *skip()*).

Наприклад, у кожному документі визначено масив *languages* для зберігання мов, якими розмовляє людина. Їх може бути 1, 2, 3, і більше.

І ми хочемо під час виведення документів зробити так, щоб у вибірку потрапляла тільки одна мова з масиву *languages*, а не весь масив:

```
db.users.find ({name: "Anders"}, {languages: {$slice : 1}})
```

Цей запит під час вилучення документа залишить у результаті тільки першу мову з масиву *languages*, тобто в цьому випадку *norwegian*.

```

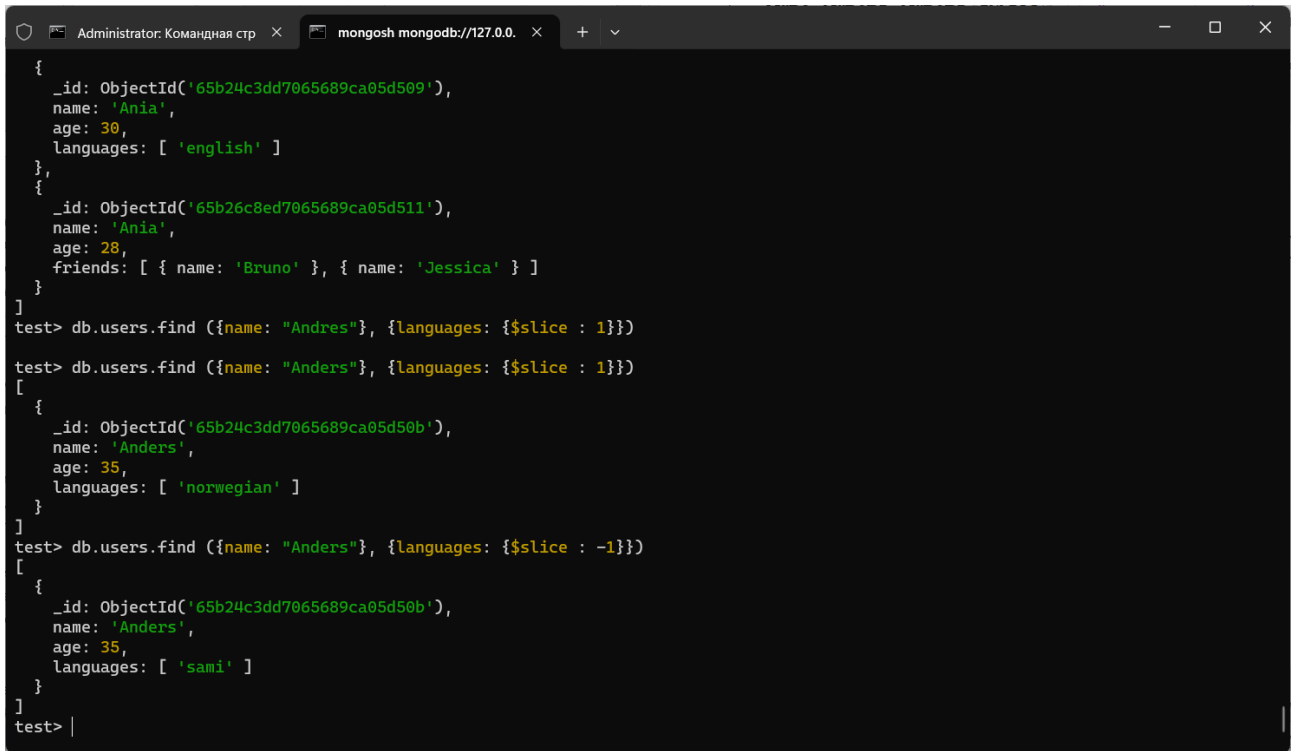
{ name: 'Ron' }, { name: 'Sam' },
{ name: 'Sarah' }, { name: 'Scorsese' },
{ name: 'Stive' }, { name: 'Tarantino' }
]
Type "it" for more
test> it
[ { name: 'Tim' }, { name: 'Tim' }, { name: 'Tom' } ]
test> db.users.find ({name: "Ania"}, {languages: {$slice : 1}})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english' ]
  },
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  }
]
test> db.users.find ({name: "Andres"}, {languages: {$slice : 1}})
test> db.users.find ({name: "Anders"}, {languages: {$slice : 1}})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d50b'),
    name: 'Anders',
    age: 35,
    languages: [ 'norwegian' ]
  }
]
test> |

```

Рисунок 25 – Результат використання оператора \$slice

Зворотна ситуація: нам треба залишити в масиві також один елемент, але не з початку, а з кінця. У цьому випадку необхідно передати в параметр від'ємне значення:

```
db.users.find ({name: "Anders"}, {languages: {$slice : -1}})
```



```

{
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Ania',
  age: 30,
  languages: [ 'english' ]
},
{
  _id: ObjectId('65b26c8ed7065689ca05d511'),
  name: 'Ania',
  age: 28,
  friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
}
]
test> db.users.find ({name: "Andres"}, {languages: {$slice : 1}})
test> db.users.find ({name: "Anders"}, {languages: {$slice : 1}})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d50b'),
    name: 'Anders',
    age: 35,
    languages: [ 'norwegian' ]
  }
]
test> db.users.find ({name: "Anders"}, {languages: {$slice : -1}})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d50b'),
    name: 'Anders',
    age: 35,
    languages: [ 'sami' ]
  }
]
test> |

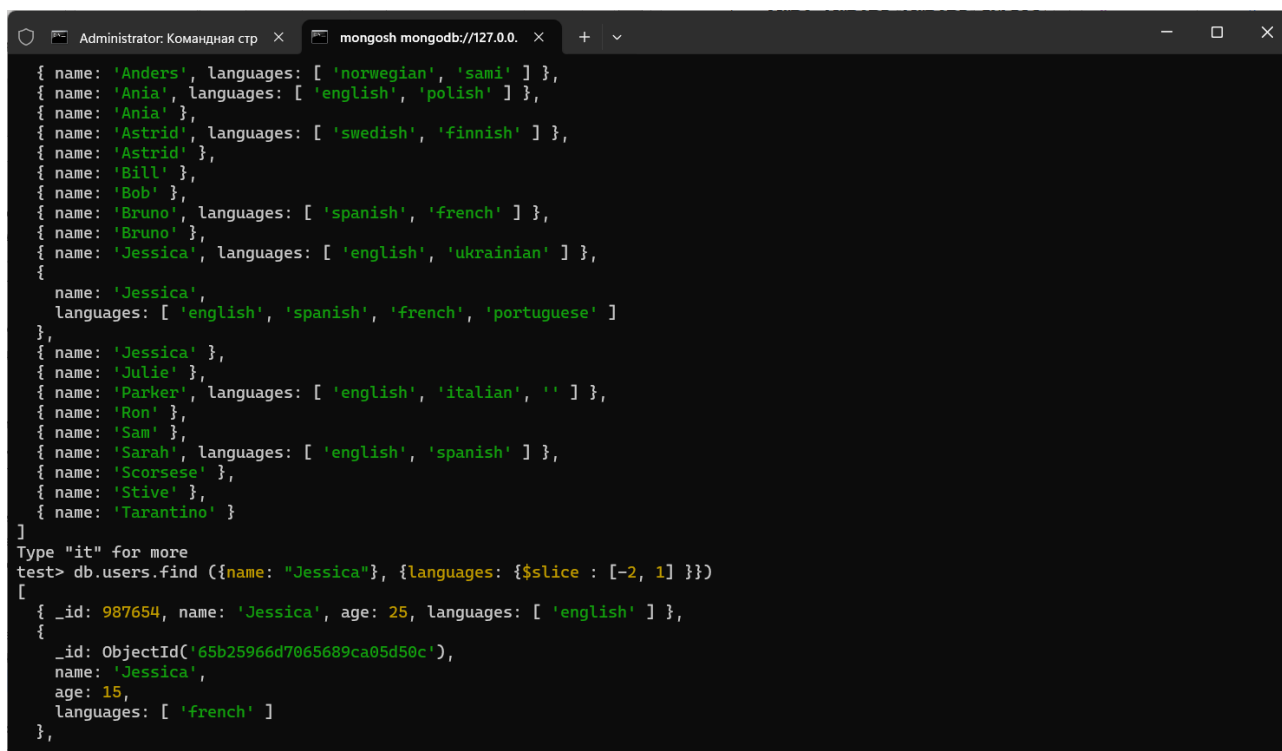
```

Рисунок 26 – Результат використання оператора \$slice

Тепер у масиві опиниться "sami", оскільки він перший з кінця в доданому елементі.

Використовуємо відразу два параметри:

```
db.users.find ({name: "Jessica"}, {languages: {$slice : [-2, 1]
}})
```



```

Administrator: Командная строка x mongosh mongodb://127.0.0.1 x + v
{ name: 'Anders', languages: [ 'norwegian', 'sami' ] },
{ name: 'Ania', languages: [ 'english', 'polish' ] },
{ name: 'Ania' },
{ name: 'Astrid', languages: [ 'swedish', 'finnish' ] },
{ name: 'Astrid' },
{ name: 'Bill' },
{ name: 'Bob' },
{ name: 'Bruno', languages: [ 'spanish', 'french' ] },
{ name: 'Bruno' },
{ name: 'Jessica', languages: [ 'english', 'ukrainian' ] },
{
  name: 'Jessica',
  languages: [ 'english', 'spanish', 'french', 'portuguese' ]
},
{ name: 'Jessica' },
{ name: 'Julie' },
{ name: 'Parker', languages: [ 'english', 'italian', '' ] },
{ name: 'Ron' },
{ name: 'Sam' },
{ name: 'Sarah', languages: [ 'english', 'spanish' ] },
{ name: 'Scorsese' },
{ name: 'Stive' },
{ name: 'Tarantino' }
]
Type "it" for more
test> db.users.find ({name: "Jessica"}, {languages: {$slice : [-2, 1] }})
[
  { _id: 987654, name: 'Jessica', age: 25, languages: [ 'english' ] },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'french' ]
  },
]

```

Рисунок 27 – Результат використання оператора \$slice

Перший параметр говорить почати вибірку елементів з кінця (оскільки від'ємне значення), тобто вибірка йде починаючи з другого елемента з кінця, а другий параметр вказує на кількість елементів масиву, що повертаються. У підсумку в масиві *language* виявиться "english" та "french" для різних документів у колекції *users*.

6 Індеси

Під час пошуку документів у невеликих колекціях ми не зазнаємо особливих проблем. Однак коли колекції містять мільйони документів, а нам треба зробити вибірку за певним полем, то пошук потрібних даних може зайняти деякий час, який може виявитися критичним для нашого завдання. У цьому випадку нам можуть допомогти індекси.

Індекси дають змогу впорядкувати дані за певним полем, що згодом прискорить пошук. Наприклад, якщо ми у своєму додатку або завданні, як правило, виконуємо пошук за полем *name*, то ми можемо індексувати колекцію за цим полем.

6.1 Створення індексу

Для створення індексу застосовується функція *createIndex()*, в яку передається об'єкт із зазначенням полів, для яких створюється індекс. Наприклад, створення індексу за полем "name":

```
db.users.createIndex({"name" : 1})
```

Під час створення індексу консоль поверне нам назву індексу:

```

{
  name: 'Jessica',
  languages: [ 'english', 'spanish', 'french', 'portuguese' ]
},
{ name: 'Jessica' },
{ name: 'Julie' },
{ name: 'Parker', languages: [ 'english', 'italian', '' ] },
{ name: 'Ron' },
{ name: 'Sam' },
{ name: 'Sarah', languages: [ 'english', 'spanish' ] },
{ name: 'Scorsese' },
{ name: 'Stive' },
{ name: 'Tarantino' }
]
Type "it" for more
test> db.users.find ({name: "Jessica"}, {languages: {$slice : [-2, 1] }})
[
  { _id: 987654, name: 'Jessica', age: 25, languages: [ 'english' ] },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'french' ]
  },
  {
    _id: ObjectId('65b26c98d7065689ca05d513'),
    name: 'Jessica',
    age: 31,
    friends: [ { name: 'Astrid' } ]
  }
]
test> db.users.createIndex({"name" : 1})
name_1
test>

```

Рисунок 28 – Результат створення індексу

Тобто в прикладі вище було створено індекс з ім'ям "name_1" за полем *name*. *MongoDB* дає змогу встановити до 64 індексів на одну колекцію.

Для створення декількох індексів застосовується функція *createIndexes()* - в неї передається масив об'єктів, які встановлюють поля для індексів:

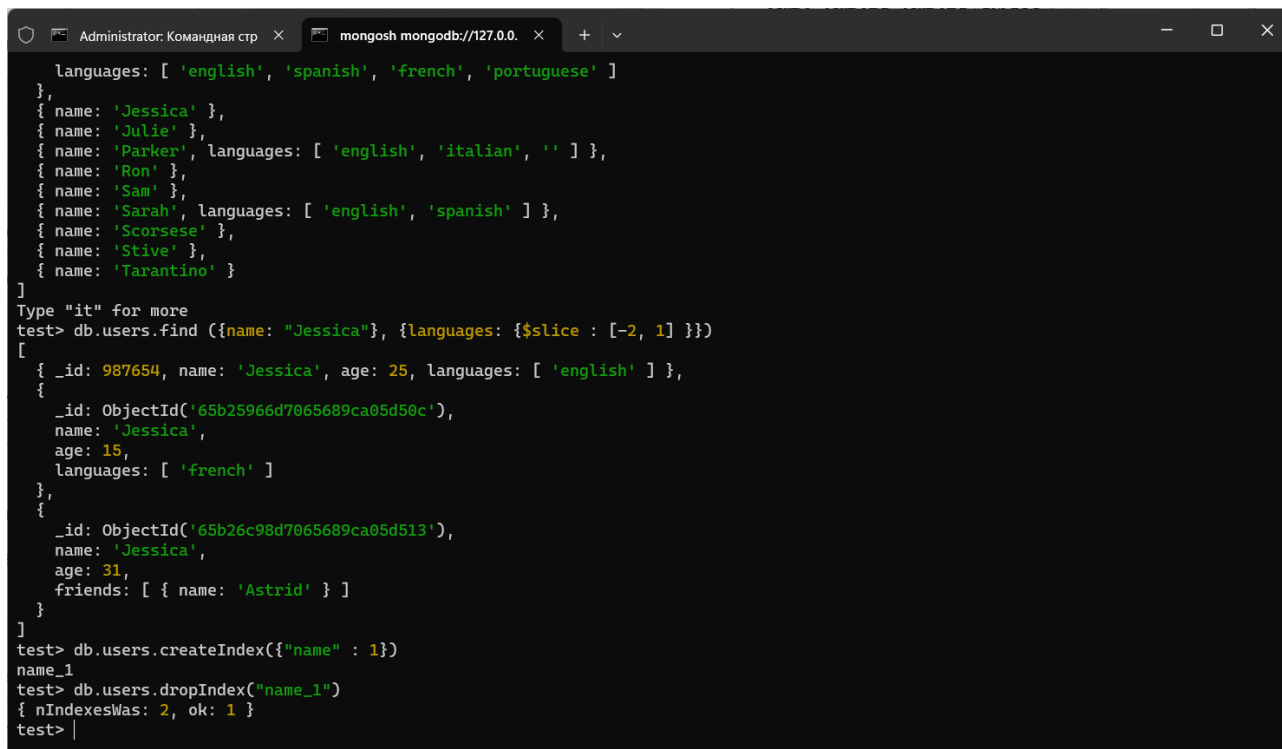
```
db.users.createIndexes([{"name": 1}, {"age": 1}])
```

У цьому випадку створюються два індекси - один для поля *name*, інший для поля *age*.

6.2 Видалення індексів

Для видалення індексів застосовується функція `dropIndex()`, в яку передається ім'я індексу. Наприклад, видалимо вище визначений індекс `"name_1"`:

```
db.users.dropIndex("name_1")
```



```

languages: [ 'english', 'spanish', 'french', 'portuguese' ]
},
{ name: 'Jessica' },
{ name: 'Julie' },
{ name: 'Parker', languages: [ 'english', 'italian', '' ] },
{ name: 'Ron' },
{ name: 'Sam' },
{ name: 'Sarah', languages: [ 'english', 'spanish' ] },
{ name: 'Scorsese' },
{ name: 'Stive' },
{ name: 'Tarantino' }
]
Type "it" for more
test> db.users.find ({name: "Jessica"}, {languages: {$slice : [-2, 1] }})
[
  { _id: 987654, name: 'Jessica', age: 25, languages: [ 'english' ] },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'french' ]
  },
  {
    _id: ObjectId('65b26c98d7065689ca05d513'),
    name: 'Jessica',
    age: 31,
    friends: [ { name: 'Astrid' } ]
  }
]
test> db.users.createIndex({"name" : 1})
name_1
test> db.users.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
test>

```

Рисунок 29 – Результат видалення індексу

6.3 Налаштування індексів

Якщо ми просто визначимо індекс для колекції, наприклад, `db.users.createIndex({"name" : 1})`, то ми все ще зможемо додавати в колекцію документи з однаковим значенням ключа `name`. Однак, якщо нам потрібно, щоб у колекцію можна було додавати документ з одним і тим самим значенням ключа тільки один раз, ми можемо встановити прапор `unique`:

```
db.users.createIndex({"name" : 1}, {"unique" : true})
```

При цьому під час додавання унікального індексу в колекції не повинно бути документів, у яких для певного поля є однакові значення.

Тепер, якщо ми спробуємо додати в колекцію два документи з одним і тим самим значенням *name*, то ми отримаємо помилку.

Водночас тут є свої тонкощі. Так, документ може не мати ключа *name*. У цьому разі для документа, що додається, автоматично створюється ключ *name* зі значенням *null*. Тому під час додавання другого документа, в якому не визначено ключ *name*, буде викинуто виняток, оскільки ключ *name* зі значенням *null* уже присутній у колекції.

Також можна задати один індекс відразу для двох полів:

```
db.users.createIndex({"name" : 1, "age" : 1})
```

Однак у цьому разі всі документи, що додаються, повинні мати унікальні значення для обох полів.

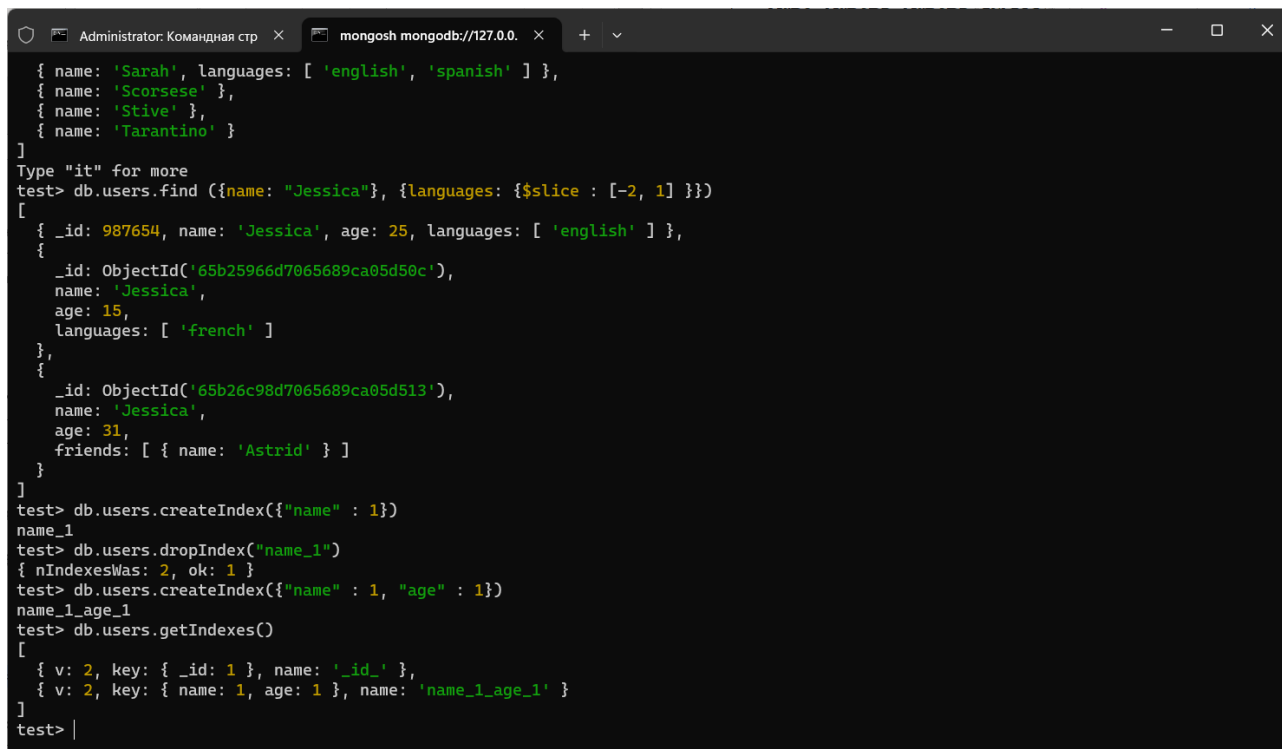
Крім того, тут є свої обмеження. Наприклад, значення поля, за яким іде індексація, не має бути більшим за 1024 байти.

6.4 Управління індексами

Усі індекси бази даних зберігаються в системній колекції *indexes*. Для звернення до неї ми можемо використовувати функцію *getIndexes*, наприклад, щоб вивести всю інформацію про індекси для конкретної колекції:

```
db.users.getIndexes()
```

Ця команда поверне виведення на кшталт такого:



```

Administrator: Командная строка  mongosh mongodb://127.0.0.1:27017/
{ name: 'Sarah', languages: [ 'english', 'spanish' ] },
{ name: 'Scorsese' },
{ name: 'Stive' },
{ name: 'Tarantino' }
]
Type "it" for more
test> db.users.find ({name: "Jessica"}, {languages: {$slice : [-2, 1] }})
[
  { _id: 987654, name: 'Jessica', age: 25, languages: [ 'english' ] },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'french' ]
  },
  {
    _id: ObjectId('65b26c98d7065689ca05d513'),
    name: 'Jessica',
    age: 31,
    friends: [ { name: 'Astrid' } ]
  }
]
test> db.users.createIndex({"name" : 1})
name_1
test> db.users.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
test> db.users.createIndex({"name" : 1, "age" : 1})
name_1_age_1
test> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1, age: 1 }, name: 'name_1_age_1' }
]
test>

```

Рисунок 30 – Перелік індексів

Як ми бачимо, тут для колекції *users* (з БД *test*) визначено 2 індекси: *id* і *name+age*. Поле *key* використовується для пошуку максимального і мінімального значень, для різних операцій, де треба застосовувати цей індекс. Поле *name* застосовується як ідентифікатор для операцій адміністрування, наприклад, для видалення індексу.

7 Агрегатні функції

7.1 Кількість елементів у колекції

За допомогою функції *countDocuments()* можна отримати загальну кількість документів у колекції:

```
db.users.countDocuments()
```

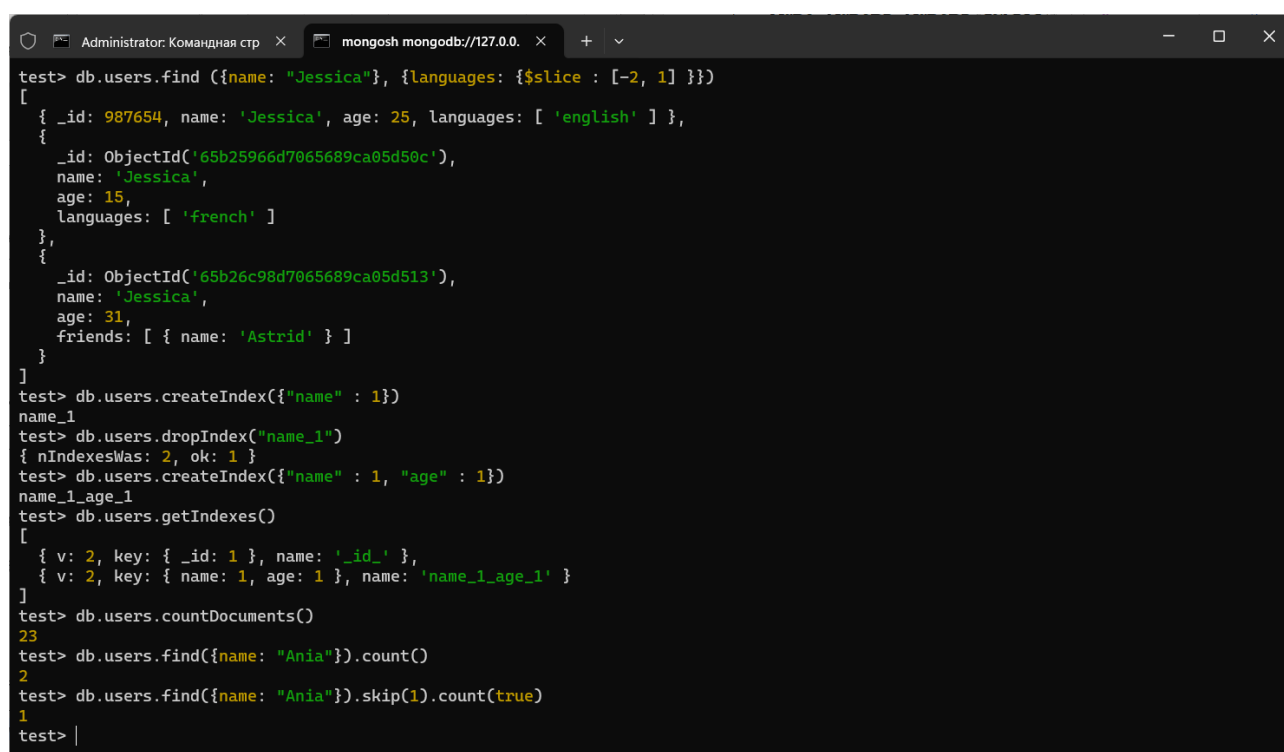
Якщо нам треба дізнатися не сукупну кількість документів у колекції, а тільки кількість документів у конкретній вибірці, то можна застосовувати функцію *count()*. Наприклад, підрахуємо кількість документів, у яких *name=Ania*:

```
db.users.find({name: "Ania"}).count()
```

Ба більше, ми можемо створювати ланцюжки функцій, щоб конкретизувати умови підрахунку:

```
db.users.find({name: "Ania"}).skip(1).count(true)
```

Тут треба зазначити, що за замовчуванням функція *count* не використовується з функціями *limit* і *skip*. Щоб їх використовувати, як у прикладі вище, у функцію *count()* треба передати булеве значення *true*.



```

Administrator: Командная стр  X  mongosh mongodb://127.0.0.  X  +  v
test> db.users.find ({name: "Jessica", {languages: {$slice : [-2, 1] }}})
[
  { _id: 987654, name: 'Jessica', age: 25, languages: [ 'english' ] },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'french' ]
  },
  {
    _id: ObjectId('65b26c98d7065689ca05d513'),
    name: 'Jessica',
    age: 31,
    friends: [ { name: 'Astrid' } ]
  }
]
test> db.users.createIndex({"name" : 1})
name_1
test> db.users.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
test> db.users.createIndex({"name" : 1, "age" : 1})
name_1_age_1
test> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1, age: 1 }, name: 'name_1_age_1' }
]
test> db.users.countDocuments()
23
test> db.users.find({name: "Ania"}).count()
2
test> db.users.find({name: "Ania"}).skip(1).count(true)
1
test> |

```

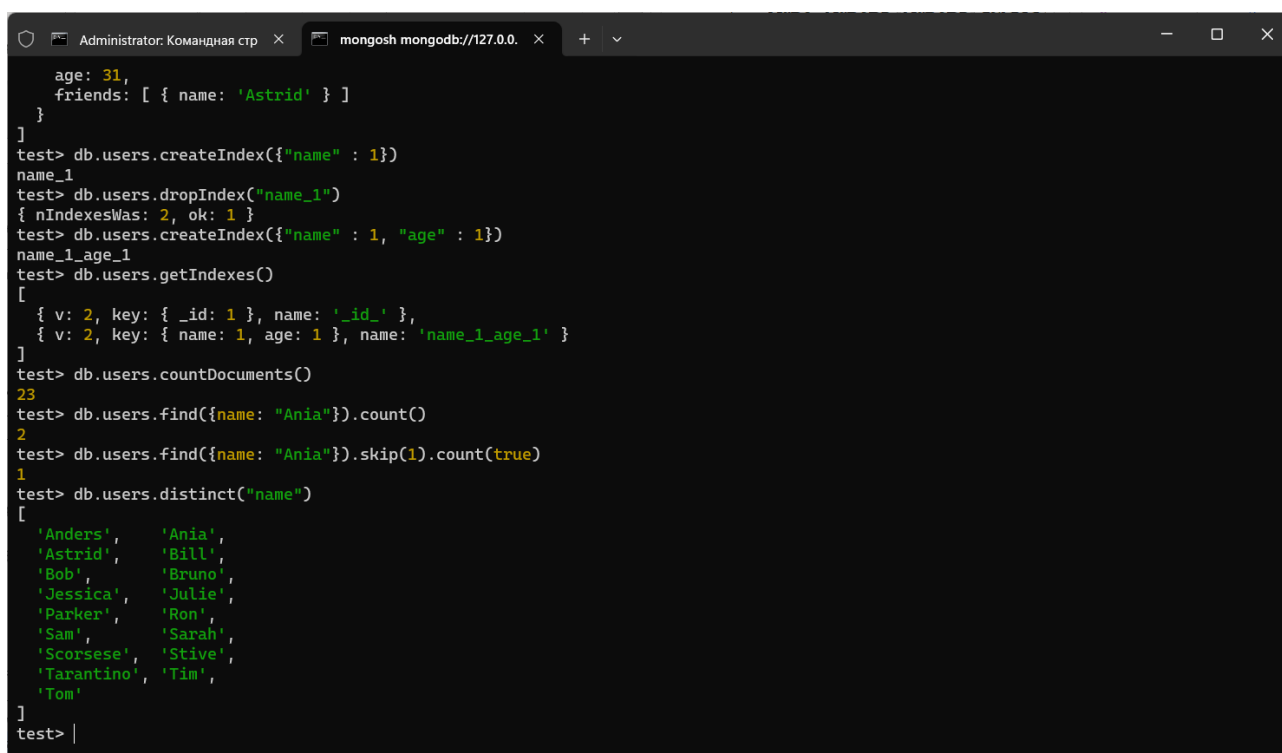
Рисунок 31 – Результат роботи функції *count()*

7.2 Функція `distinct()`

У колекції можуть бути документи, які містять однакові значення для одного або декількох полів. Наприклад, у кількох документах визначено `name: "Ania"`. І нам треба знайти тільки унікальні відмінні значення для одного з полів документа. Для цього ми можемо скористатися функцією `distinct()`. Скористаємося навчальною БД `users`:

Виведемо всі унікальні значення за полем `"name"`:

```
test> db.users.distinct("name")
```



```

age: 31,
friends: [ { name: 'Astrid' } ]
}
]
test> db.users.createIndex({"name" : 1})
name_1
test> db.users.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
test> db.users.createIndex({"name" : 1, "age" : 1})
name_1_age_1
test> db.users.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1, age: 1 }, name: 'name_1_age_1' }
]
test> db.users.countDocuments()
23
test> db.users.find({name: "Ania"}).count()
2
test> db.users.find({name: "Ania"}).skip(1).count(true)
1
test> db.users.distinct("name")
[
  'Anders',   'Ania',
  'Astrid',   'Bill',
  'Bob',      'Bruno',
  'Jessica',  'Julie',
  'Parker',   'Ron',
  'Sam',      'Sarah',
  'Scorsese', 'Stive',
  'Tarantino', 'Tim',
  'Tom'
]
test> |

```

Рисунок 32 – Результат виборки по унікальним значенням

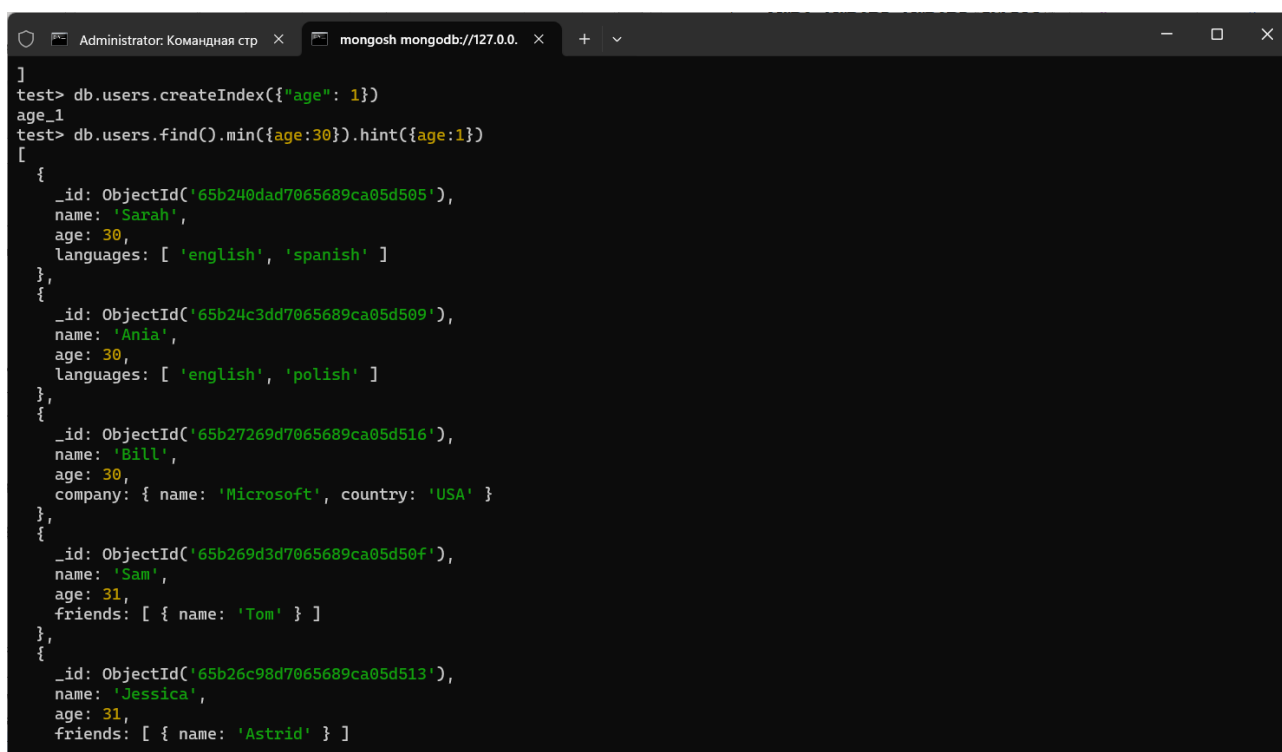
7.2 Функції `min()` і `max()`

Функції `min()` і `max()` встановлює для певного поля мінімальне значення для потрапляння у вибірку. При цьому ці функції можуть використовувати тільки ті поля, для яких встановлено індекси. Наприклад, візьмемо вище визначену колекцію `db.users` і визначимо в ній індекс для поля `age`:

```
db.users.createIndex({"age": 1})
```

Під час виконання функції також треба використовувати функцію *hint()*, в яку передається індекс. Наприклад, виберемо всі документи, в яких поле *age* більше 30:

```
test> db.users.find().min({age:30}).hint({age:1})
```



```

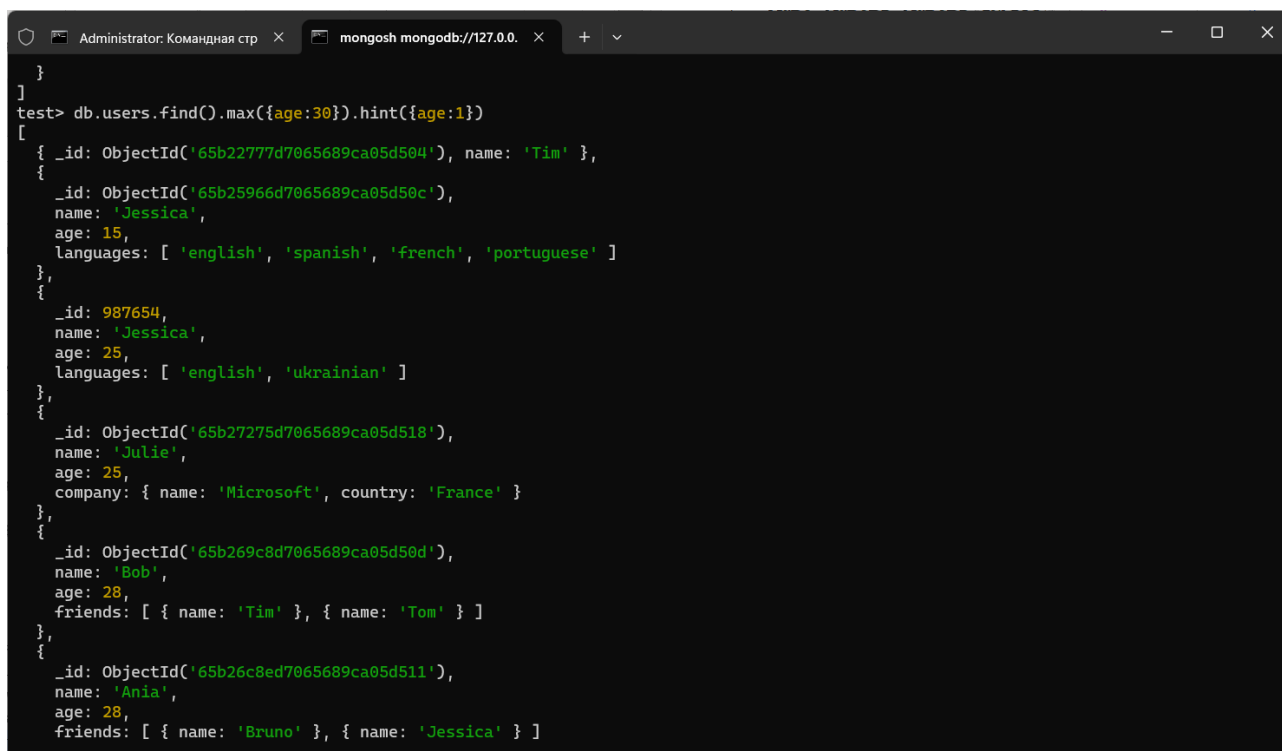
]
test> db.users.createIndex({"age": 1})
age_1
test> db.users.find().min({age:30}).hint({age:1})
[
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b27269d7065689ca05d516'),
    name: 'Bill',
    age: 30,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Sam',
    age: 31,
    friends: [ { name: 'Tom' } ]
  },
  {
    _id: ObjectId('65b26c98d7065689ca05d513'),
    name: 'Jessica',
    age: 31,
    friends: [ { name: 'Astrid' } ]
  }
]

```

Рисунок 33 – Результ застосування функцій *min()* та *hint()*

Аналогічним чином працює функція *max()*, яка встановлює максимальне значення. Наприклад, виберемо документи, де *age* менше 30:

```
test> db.users.find().max({age:30}).hint({age:1})
```



```

}
]
test> db.users.find().max({age:30}).hint({age:1})
[
  { _id: ObjectId('65b2277d7065689ca05d504'), name: 'Tim' },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  },
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  },
  {
    _id: ObjectId('65b27275d7065689ca05d518'),
    name: 'Julie',
    age: 25,
    company: { name: 'Microsoft', country: 'France' }
  },
  {
    _id: ObjectId('65b269c8d7065689ca05d50d'),
    name: 'Bob',
    age: 28,
    friends: [ { name: 'Tim' }, { name: 'Tom' } ]
  },
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  }
]

```

Рисунок 34 – Результ застосування функцій max() та hint()

8 Оператори вибірки

8.1 Умовні оператори

Умовні оператори задають умову, якій має відповідати значення поля документа:

- *\$eq* (дорівнює)
- *\$ne* (не дорівнює)
- *\$gt* (більше ніж)
- *\$lt* (менше ніж)
- *\$gte* (більше або дорівнює)
- *\$lte* (менше або дорівнює)
- *\$in* визначає масив значень, одне з яких повинно мати поле документа
- *\$nin* визначає масив значень, які не повинно мати поле документа

Наприклад, знайдемо всі документи, у яких значення ключа *age* менше 45:

```
db.users.find ({age: {$lt : 45}})
```

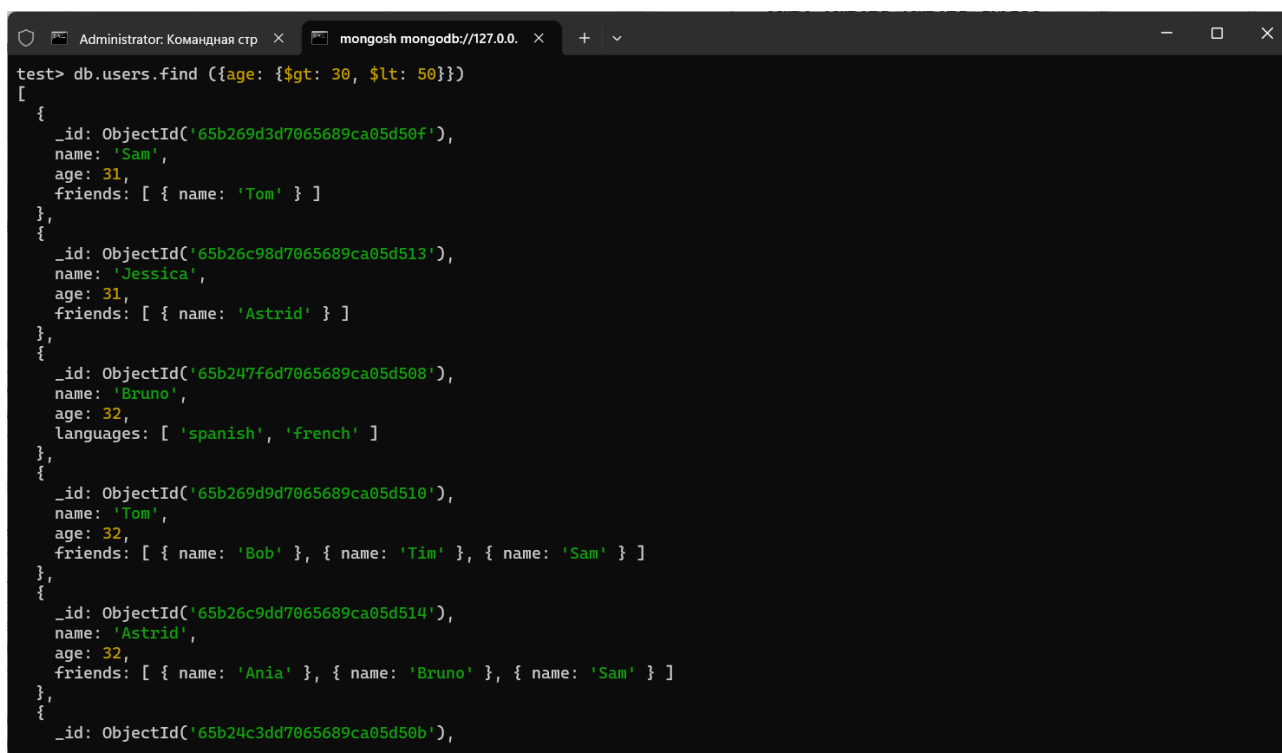

Аналогічно буде використання інших операторів порівняння. Наприклад, той самий ключ, тільки більше 45:

```
db.users.find ({age: {$gt : 45}})
```

Зверніть увагу, що порівняння тут проводиться над цілочисельними типами, а не рядками. Якщо ключ `age` представляє строкові значення, то відповідно треба проводити порівняння над рядками: `db.users.find ({age: {$gt : "45"}})`, однак результат буде тим самим.

Але уявімо ситуацію, коли нам треба знайти всі об'єкти зі значенням поля `age` більше 30, але менше 50. У цьому випадку ми можемо комбінувати два оператори:

```
db.users.find ({age: {$gt: 30, $lt: 50}})
```



```

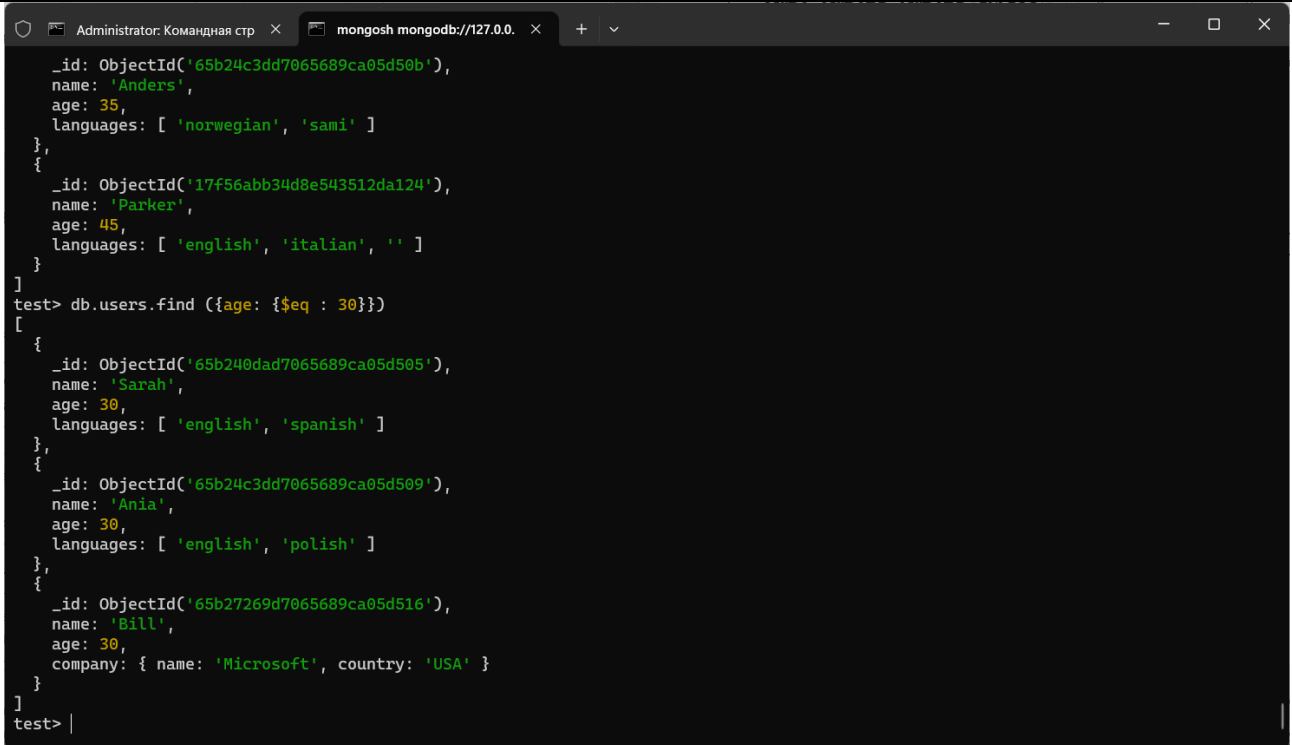
Administrator: Командная стр  ×  mongosh mongodbi/127.0.0.  ×  +  ▾
test> db.users.find ({age: {$gt: 30, $lt: 50}})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Sam',
    age: 31,
    friends: [ { name: 'Tom' } ]
  },
  {
    _id: ObjectId('65b26c98d7065689ca05d513'),
    name: 'Jessica',
    age: 31,
    friends: [ { name: 'Astrid' } ]
  },
  {
    _id: ObjectId('65b247f6d7065689ca05d508'),
    name: 'Bruno',
    age: 32,
    languages: [ 'spanish', 'french' ]
  },
  {
    _id: ObjectId('65b269d9d7065689ca05d510'),
    name: 'Tom',
    age: 32,
    friends: [ { name: 'Bob' }, { name: 'Tim' }, { name: 'Sam' } ]
  },
  {
    _id: ObjectId('65b26c9dd7065689ca05d514'),
    name: 'Astrid',
    age: 32,
    friends: [ { name: 'Ania' }, { name: 'Bruno' }, { name: 'Sam' } ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d50b'),

```

Рисунок 35 – Результат комбінування двої умовних операторів

Знайдемо користувачів, вік яких дорівнює 30:

```
db.users.find ({age: {$eq : 30}})
```



```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.1:27017/
{
  "_id": ObjectId("65b24c3dd7065689ca05d50b"),
  "name": "Anders",
  "age": 35,
  "languages": [ "norwegian", "sami" ]
},
{
  "_id": ObjectId("17f56abb34d8e543512da124"),
  "name": "Parker",
  "age": 45,
  "languages": [ "english", "italian", "" ]
}
]
test> db.users.find ({age: {$eq : 30}})
[
  {
    "_id": ObjectId("65b240dad7065689ca05d505"),
    "name": "Sarah",
    "age": 30,
    "languages": [ "english", "spanish" ]
  },
  {
    "_id": ObjectId("65b24c3dd7065689ca05d509"),
    "name": "Ania",
    "age": 30,
    "languages": [ "english", "polish" ]
  },
  {
    "_id": ObjectId("65b27269d7065689ca05d516"),
    "name": "Bill",
    "age": 30,
    "company": { name: "Microsoft", country: "USA" }
  }
]
test> |

```

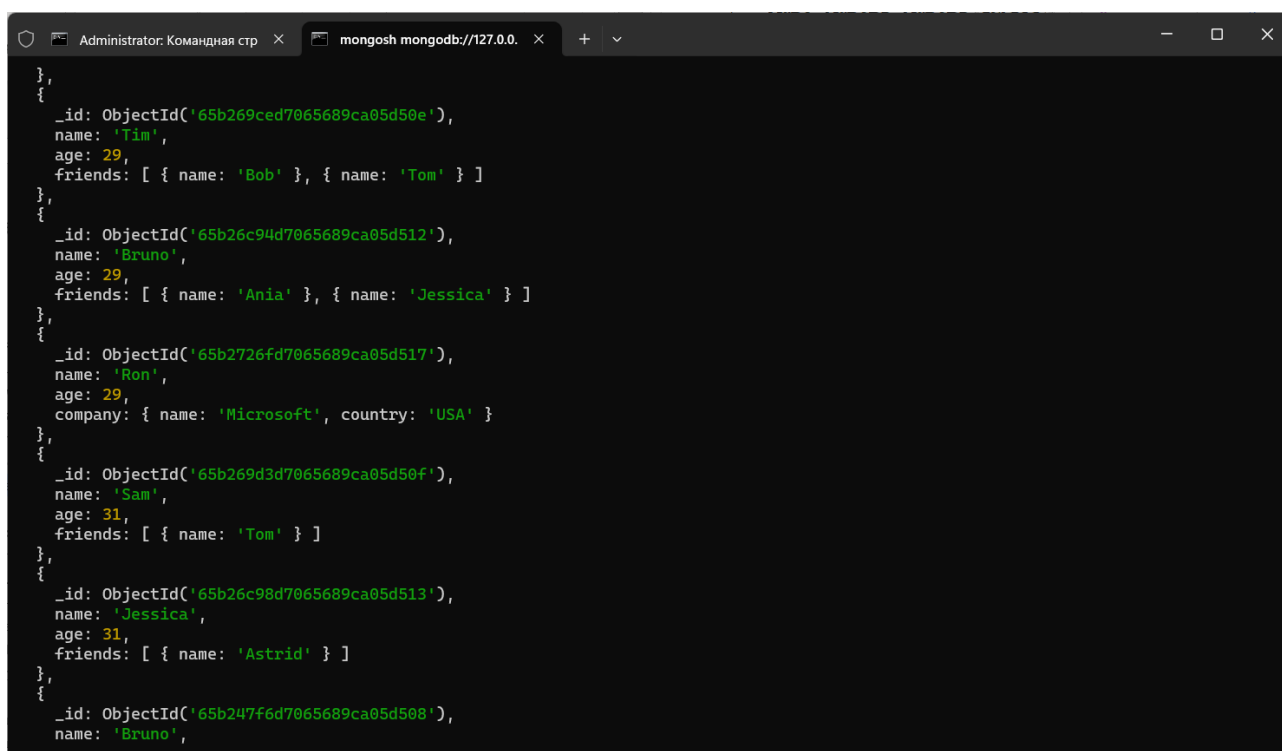
Рисунок 36 – Результат використання оператора \$eq

По суті це аналогія такого запиту:

```
db.users.find ({age: 30})
```

Зворотна операція - знайдемо користувачів, вік яких НЕ дорівнює 30:

```
db.users.find ({age: {$ne: 30}})
```



```

{
  "_id": ObjectId("65b269ced7065689ca05d50e"),
  "name": "Tim",
  "age": 29,
  "friends": [ { name: "Bob" }, { name: "Tom" } ]
},
{
  "_id": ObjectId("65b26c94d7065689ca05d512"),
  "name": "Bruno",
  "age": 29,
  "friends": [ { name: "Ania" }, { name: "Jessica" } ]
},
{
  "_id": ObjectId("65b2726fd7065689ca05d517"),
  "name": "Ron",
  "age": 29,
  "company": { name: "Microsoft", country: "USA" }
},
{
  "_id": ObjectId("65b269d3d7065689ca05d50f"),
  "name": "Sam",
  "age": 31,
  "friends": [ { name: "Tom" } ]
},
{
  "_id": ObjectId("65b26c98d7065689ca05d513"),
  "name": "Jessica",
  "age": 31,
  "friends": [ { name: "Astrid" } ]
},
{
  "_id": ObjectId("65b247f6d7065689ca05d508"),
  "name": "Bruno",
  "age": 31
}

```

Рисунок 36 – Результат використання оператора \$ne

Оператор *\$in* визначає масив можливих виразів і шукає ті ключі, значення яких є в масиві:

```
db.users.find ({age: {$in : [25, 40]}})
```

Протилежним чином діє оператор *\$nin* - він визначає масив можливих виразів і шукає ті ключі, значення яких відсутні в цьому масиві:

```
db.users.find ({age: {$nin : [22, 40]}})
```

```

Type "it" for more
test> db.users.find ({age: {$nin : [25, 40]}})
[
  {
    _id: ObjectId('65b2277d7065689ca05d504'),
    name: 'Tim',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  },
  {
    _id: ObjectId('65b269c8d7065689ca05d50d'),
    name: 'Bob',
    age: 28,
    friends: [ { name: 'Tim' }, { name: 'Tom' } ]
  },
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b27256d7065689ca05d515'),
    name: 'Stive',
    age: 28,
    company: { name: 'Apple', country: 'USA' }
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d50a'),
    name: 'Astrid',
    age: 29,
    languages: [ 'swedish', 'finnish' ]
  },
]

```

Рисунок 37 – Результат використання оператора \$nin

8.2 Логічні оператори

Логічні оператори виконуються над умовами вибірки:

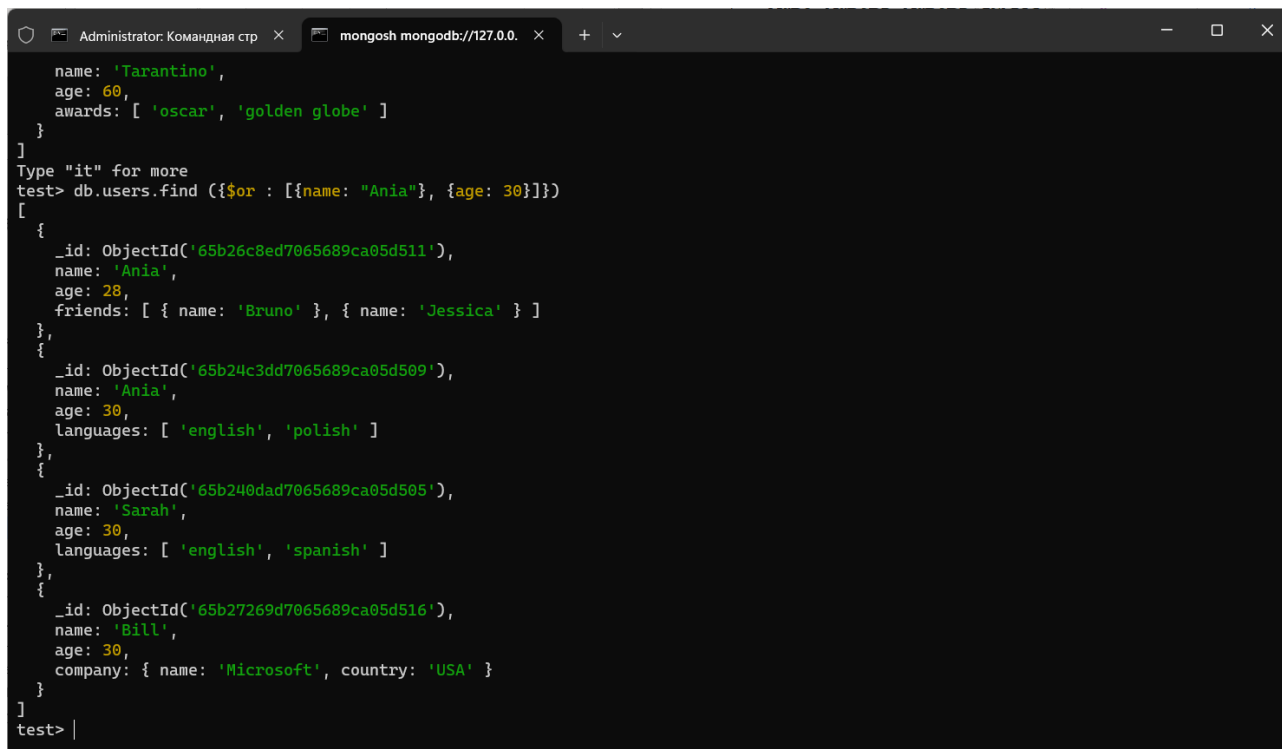
- *\$or*: з'єднує дві умови, і документ має відповідати одній із цих умов
- *\$and*: з'єднує дві умови, і документ має відповідати обом умовам
- *\$not*: документ повинен НЕ відповідати умові
- *\$nor*: з'єднує дві умови, і документ повинен НЕ відповідати обом умовам

Оператор \$or

Оператор *\$or* представляє логічну операцію *АБО* і визначає набір пар ключ-значення, які повинні бути в документі. І якщо документ має хоч одну таку пару ключ-значення, то він відповідає даному запиту і витягується з БД:

```
db.users.find ({ $or : [{ name: "Ania" }, { age: 30 }] })
```

Цей вираз поверне нам усі документи, в яких або *name=Ania*, або *age=30*.



```

name: 'Tarantino',
age: 60,
awards: [ 'oscar', 'golden globe' ]
}
]
Type "it" for more
test> db.users.find ({ $or : [{ name: "Ania"}, { age: 30 } ] })
[
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: ObjectId('65b27269d7065689ca05d516'),
    name: 'Bill',
    age: 30,
    company: { name: 'Microsoft', country: 'USA' }
  }
]
test> |

```

Рисунок 38 – Результат використання оператора \$or

Інший приклад поверне нам усі документи, в яких *name=Ania*, а *age* дорівнює або 20, або серед значень *languages* є "english":

```
db.users.find ({name: "Ania", $or : [{age: 20}, {languages: "english"}]})
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1  x  +  v
{
  _id: ObjectId('65b26c8ed7065689ca05d511'),
  name: 'Ania',
  age: 28,
  friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
},
{
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Ania',
  age: 30,
  languages: [ 'english', 'polish' ]
},
{
  _id: ObjectId('65b240dad7065689ca05d505'),
  name: 'Sarah',
  age: 30,
  languages: [ 'english', 'spanish' ]
},
{
  _id: ObjectId('65b27269d7065689ca05d516'),
  name: 'Bill',
  age: 30,
  company: { name: 'Microsoft', country: 'USA' }
}
]
test> db.users.find ({name: "Ania", $or : [{age: 20}, {languages: "english"}]})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  }
]
test> |

```

У підвиразах `or` можна застосовувати умовні оператори:

```
db.users.find ({ $or : [{ name: "Ania" }, { age: { $gte: 30 } } ] })
```

У цьому випадку ми вибираємо всі документи, де `name="Ania"` або поле `age` має значення від 30 і вище.

```

Administrator: Командная стр  x  mongosh mongod://127.0.0.  x  +  v
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Ania',
  age: 30,
  languages: [ 'english', 'polish' ]
}
]
test> db.users.find ({ $or : [{name: "Ania"}, {age: {$gte:30}}] })
[
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: ObjectId('65b27269d7065689ca05d516'),
    name: 'Bill',
    age: 30,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),

```

Рисунок 39 – Результат застосування складного логічного оператора

Оператор \$and

Оператор *\$and* представляє логічну операцію *I* (логічне множення) і визначає набір критеріїв, яким обов'язково має відповідати документ. На відміну від оператора *\$or* документ має відповідати всім зазначеним критеріям. Наприклад:

```
db.users.find ({ $and : [{name: "Ania"}, {age: 30}] })
```

Тут обрані документи обов'язково повинні мати ім'я Ania і вік 30 - обидві ці ознаки.

```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.  ×  +  ▾
{
  _id: ObjectId('65b24c3dd7065689ca05d50b'),
  name: 'Anders',
  age: 35,
  languages: [ 'norwegian', 'sami' ]
},
{
  _id: ObjectId('17f56abb34d8e543512da124'),
  name: 'Parker',
  age: 45,
  languages: [ 'english', 'italian', '' ]
},
{
  _id: ObjectId('65b2472cd7065689ca05d506'),
  name: 'Tarantino',
  age: 60,
  awards: [ 'oscar', 'golden globe' ]
},
{
  _id: ObjectId('65b2472cd7065689ca05d507'),
  name: 'Scorsese',
  age: 70,
  awards: [ 'oscar', 'palme d'or', 'golden globe' ]
}
]
test> db.users.find ({ $and : [{ name: "Ania" }, { age: 30 }] })
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  }
]
test> |

```

Рисунок 40 – Результат використання оператора \$and

8.3 Пошук за масивами

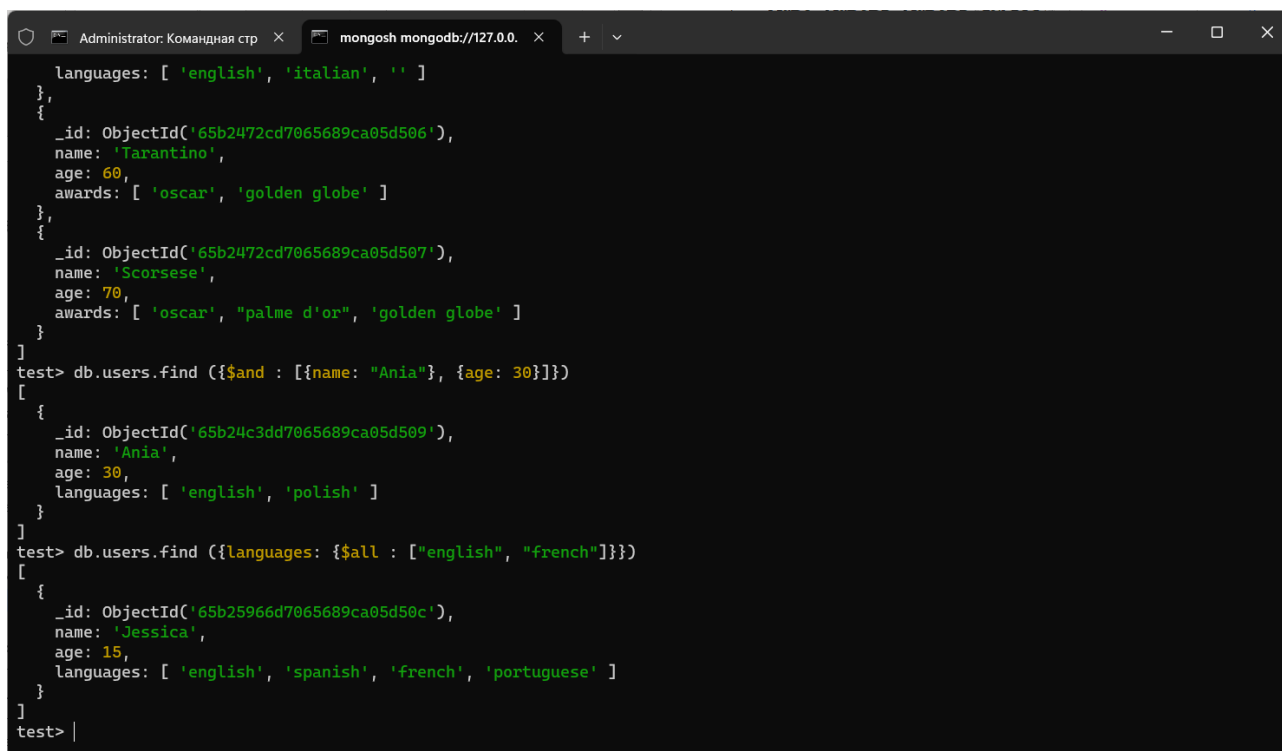
Ряд операторів призначені для роботи з масивами:

- *\$all*: визначає набір значень, які повинні бути в масиві
- *\$size*: визначає кількість елементів, які повинні бути в масиві
- *\$elemMatch*: визначає умову, яким повинні відповідати елементи в масиві

Оператор \$all

Оператор *\$all* визначає масив можливих виразів і вимагає, щоб документи мали весь визначуваний набір виразів. Відповідно він застосовується для пошуку за масивом. Наприклад, у документах є масив *languages*, що зберігає іноземні мови, якими розмовляє користувач. І щоб знайти всіх людей, які розмовляють одночасно і англійською, і французькою, ми можемо використовувати такий вираз:

```
db.users.find ({languages: {$all : ["english", "french"]}})
```

```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.  ×  +  ▾
languages: [ 'english', 'italian', '' ]
},
{
  _id: ObjectId('65b2472cd7065689ca05d506'),
  name: 'Tarantino',
  age: 60,
  awards: [ 'oscar', 'golden globe' ]
},
{
  _id: ObjectId('65b2472cd7065689ca05d507'),
  name: 'Scorsese',
  age: 70,
  awards: [ 'oscar', 'palme d'or', 'golden globe' ]
}
]
test> db.users.find ({ $and : [{name: "Ania"}, {age: 30}]})
[
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  }
]
test> db.users.find ({languages: { $all : ["english", "french"]}})
[
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> |

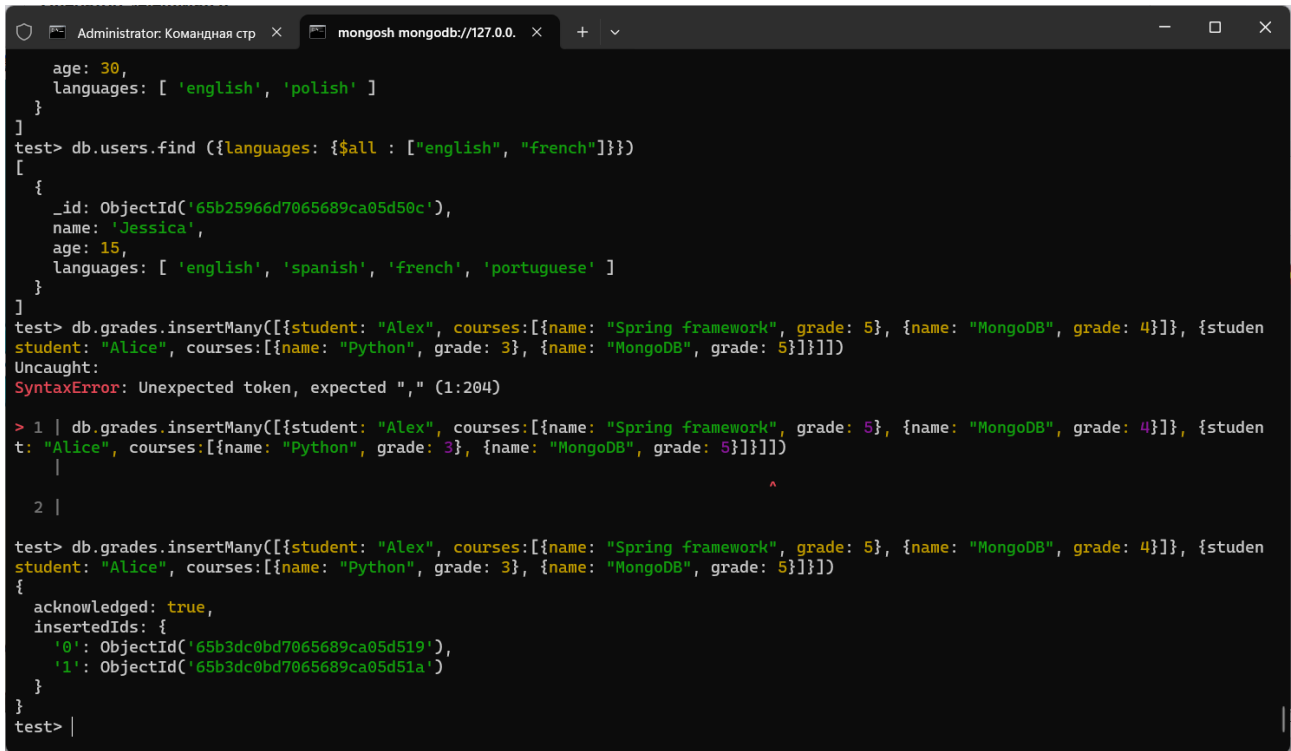
```

Рисунок 41 – Результат використання оператора \$all

Оператор \$elemMatch

Оператор \$elemMatch дає змогу вибрати документи, у яких масиви містять елементи, що потрапляють під певні умови. Наприклад, нехай у базі даних буде колекція, яка містить оцінки користувачів за певними курсами. Додамо кілька документів:

```
db.grades.insertMany([
  {student: "Alex", courses:[
    {name: "Spring framework", grade: 5},
    {name: "MongoDB", grade: 4}
  ]},
  {student: "Alice", courses:[
    {name: "Python", grade: 3},
    {name: "MongoDB", grade: 5}
  ]}
])
```



```

age: 30,
languages: [ 'english', 'polish' ]
}
]
test> db.users.find ({languages: {$all : ["english", "french"]}})
[
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  }
]
test> db.grades.insertMany([
  {student: "Alex", courses:[{name: "Spring framework", grade: 5}, {name: "MongoDB", grade: 4}]},
  {student: "Alice", courses:[{name: "Python", grade: 3}, {name: "MongoDB", grade: 5}]}])
Uncaught:
SyntaxError: Unexpected token, expected ",", (1:204)

> 1 | db.grades.insertMany([
    | {student: "Alex", courses:[{name: "Spring framework", grade: 5}, {name: "MongoDB", grade: 4}]},
    | {student: "Alice", courses:[{name: "Python", grade: 3}, {name: "MongoDB", grade: 5}]}])
    |
  2 |
test> db.grades.insertMany([
  {student: "Alex", courses:[{name: "Spring framework", grade: 5}, {name: "MongoDB", grade: 4}]},
  {student: "Alice", courses:[{name: "Python", grade: 3}, {name: "MongoDB", grade: 5}]}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65b3dc0bd7065689ca05d519'),
    '1': ObjectId('65b3dc0bd7065689ca05d51a')
  }
}
test> |

```

Рисунок 42 – Результат додавання нових документів

Кожен документ має масив *courses*, який зі свого боку складається з вкладених документів.

Тепер знайдемо студентів, які для курсу *MongoDB* мають оцінку вище 4:

```
db.grades.find({courses: {$elemMatch: {name: "MongoDB", grade: {$gt: 4}}}})
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
SyntaxError: Unexpected token, expected ",", (1:204)

> 1 | db.grades.insertMany([
  |   {student: "Alex", courses:[{name: "Spring framework", grade: 5}, {name: "MongoDB", grade: 4}]}, {student: "Alice", courses:[{name: "Python", grade: 3}, {name: "MongoDB", grade: 5}]}])
  |   ^
2 |

test> db.grades.insertMany([
{
  student: "Alex", courses:[{name: "Spring framework", grade: 5}, {name: "MongoDB", grade: 4}]}, {student: "Alice", courses:[{name: "Python", grade: 3}, {name: "MongoDB", grade: 5}]}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65b3dc0bd7065689ca05d519'),
    '1': ObjectId('65b3dc0bd7065689ca05d51a')
  }
}
test> db.grades.find({courses: {$elemMatch: {name: "MongoDB", grade: {$gt: 4}}}})
Uncaught:
SyntaxError: Unexpected token, expected ",", (1:70)

> 1 | db.grades.find({courses: {$elemMatch: {name: "MongoDB", grade: {$gt: 4}}}})
  |   ^
2 |

test> db.grades.find({courses: {$elemMatch: {name: "MongoDB", grade: {$gt: 4}}}})
[
  {
    _id: ObjectId('65b3dc0bd7065689ca05d51a'),
    student: 'Alice',
    courses: [ { name: 'Python', grade: 3 }, { name: 'MongoDB', grade: 5 } ]
  }
]
test> |

```

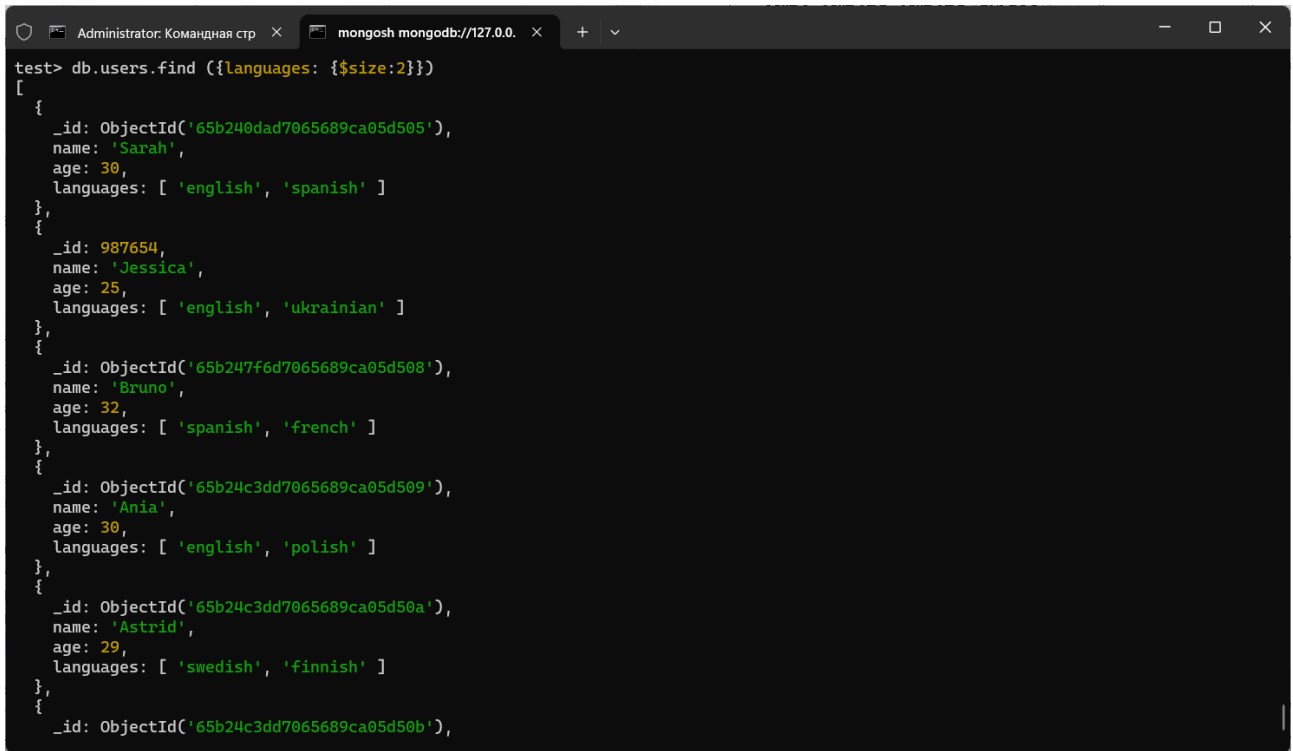
Рисунок 43 – Результат використання оператора \$elemMatch

Оператор \$size

Оператор *\$size* використовується для знаходження документів, у яких масиви мають число елементів, що дорівнює значенню *\$size*. Наприклад, витягнемо всі документи, в яких у масиві *languages* два елементи:

```
db.users.find ({languages: {$size:2}})
```

Такому запиту відповідатиме низка документів:



```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.  ×  +  ▾
test> db.users.find ({languages: {$size:2}})
[
  {
    _id: ObjectId('65b240dad7065689ca05d505'),
    name: 'Sarah',
    age: 30,
    languages: [ 'english', 'spanish' ]
  },
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  },
  {
    _id: ObjectId('65b247f6d7065689ca05d508'),
    name: 'Bruno',
    age: 32,
    languages: [ 'spanish', 'french' ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d50a'),
    name: 'Astrid',
    age: 29,
    languages: [ 'swedish', 'finnish' ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d50b'),

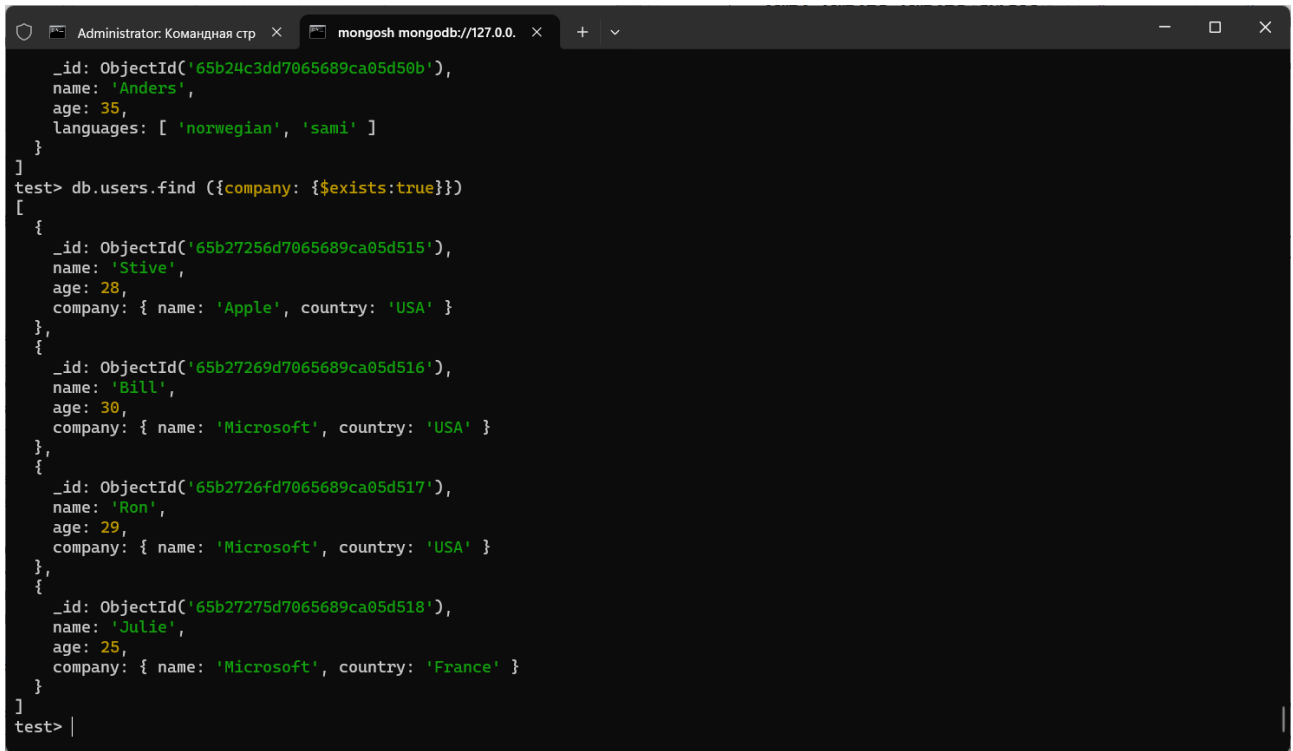
```

Рисунок 44 – Результат використання оператора \$size

Оператор \$exists

Оператор *\$exists* дає змогу витягти тільки ті документи, в яких певний ключ присутній або відсутній. Наприклад, повернемо всі документи, у яких є ключ *company*:

```
db.users.find ({company: {$exists:true}})
```



```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.1  ×  +  ▾
{
  _id: ObjectId('65b24c3dd7065689ca05d50b'),
  name: 'Anders',
  age: 35,
  languages: [ 'norwegian', 'sami' ]
}
]
test> db.users.find ({company: {$exists:true}})
[
  {
    _id: ObjectId('65b27256d7065689ca05d515'),
    name: 'Stive',
    age: 28,
    company: { name: 'Apple', country: 'USA' }
  },
  {
    _id: ObjectId('65b27269d7065689ca05d516'),
    name: 'Bill',
    age: 30,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b2726fd7065689ca05d517'),
    name: 'Ron',
    age: 29,
    company: { name: 'Microsoft', country: 'USA' }
  },
  {
    _id: ObjectId('65b27275d7065689ca05d518'),
    name: 'Julie',
    age: 25,
    company: { name: 'Microsoft', country: 'France' }
  }
]
test> |

```

Рисунок 45 – Результат використання оператора *\$exists*

Якщо ми вкажемо в оператора *\$exists* як параметр *false*, то запит поверне нам тільки ті документи, в яких не визначено ключ *company*.

Оператор *\$type*

Оператор *\$type* витягує тільки ті документи, в яких певний ключ має значення певного типу, наприклад, рядок або число:

```

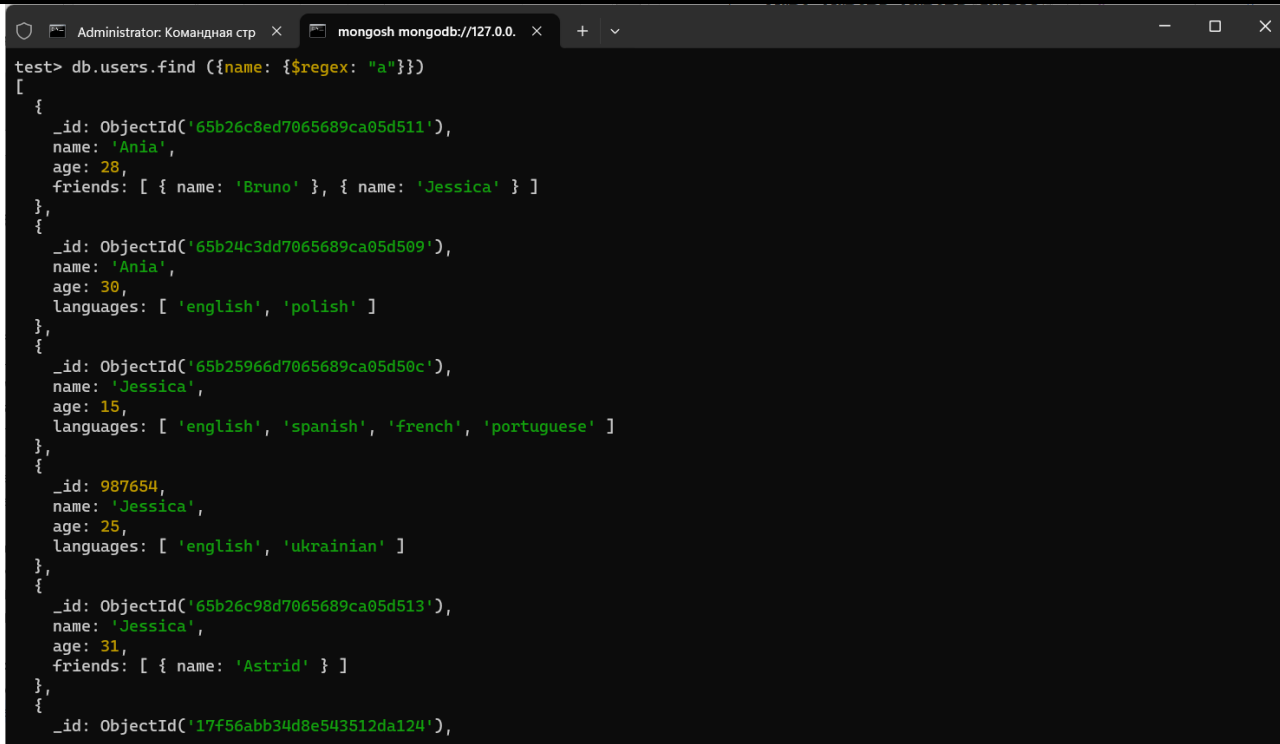
db.users.find ({age: {$type: "string"}})
db.users.find ({age: {$type: "number"}}})

```

Оператор *\$regex*

Оператор *\$regex* задає регулярний вираз, якому має відповідати значення поля. Наприклад, нехай поле *name* обов'язково має букву "b":

```
db.users.find ({name: {$regex: "a"}})
```



```
test> db.users.find ({name: {$regex: "a"}})
[
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Ania',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Ania',
    age: 30,
    languages: [ 'english', 'polish' ]
  },
  {
    _id: ObjectId('65b25966d7065689ca05d50c'),
    name: 'Jessica',
    age: 15,
    languages: [ 'english', 'spanish', 'french', 'portuguese' ]
  },
  {
    _id: 987654,
    name: 'Jessica',
    age: 25,
    languages: [ 'english', 'ukrainian' ]
  },
  {
    _id: ObjectId('65b26c98d7065689ca05d513'),
    name: 'Jessica',
    age: 31,
    friends: [ { name: 'Astrid' } ]
  },
  {
    _id: ObjectId('17f56abb34d8e543512da124'),
```

Рисунок 45 – Результат використання регулярного виразу

Важливо розуміти, що *\$regex* приймає не просто рядки, а саме регулярні вирази, наприклад: *name: {\$regex: "ia\$"}* - значення *name* має закінчуватися на "ia".

9 Оновлення даних

Як і інші системи керування базами даних MongoDB надає можливість оновлення даних. Для цього доступно низку функцій.

9.1 Функція `replaceOne()`

Якщо нам треба повністю замінити один документ іншим, також може використовуватися функція *replaceOne()*:

```
db.collection.replaceOne(filter, update, options)
```

– *filter*: приймає запит на вибірку документа, який треба оновити

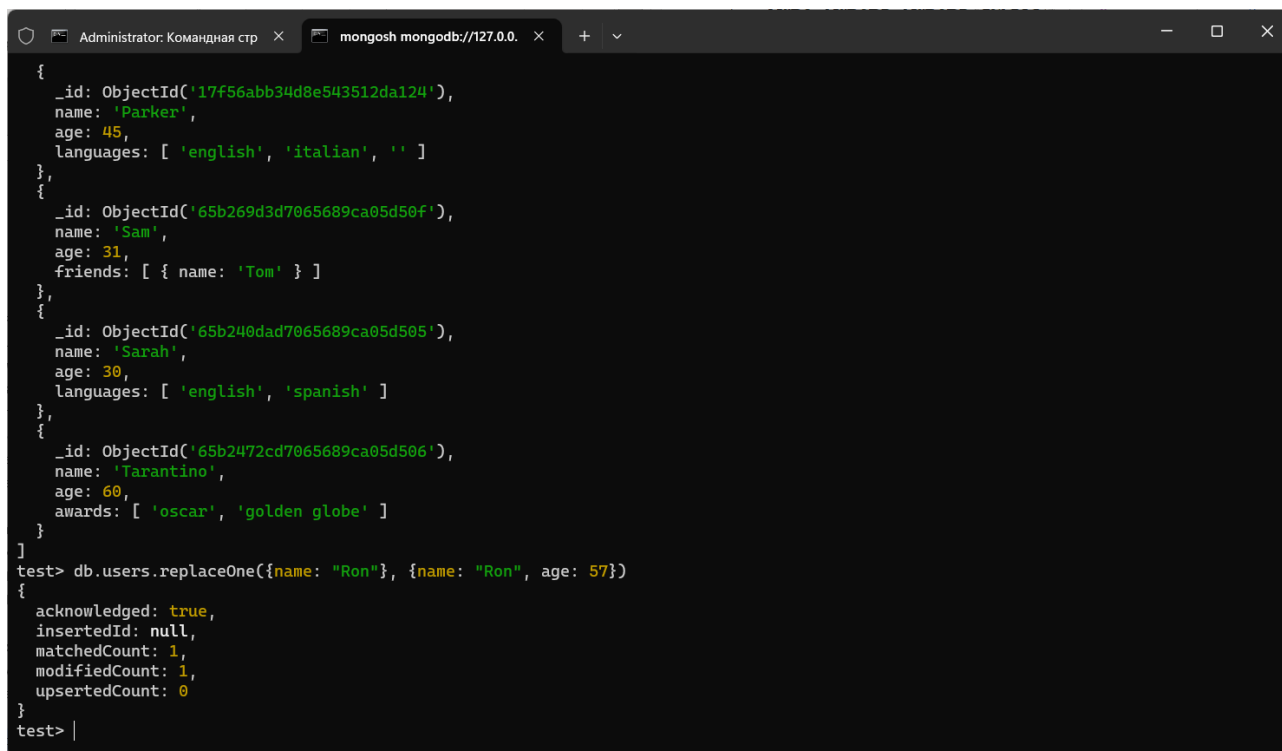
- *update*: представляє новий документ, який замінить старий під час оновлення
- *options*: визначає додаткові параметри під час оновлення документів, основним з яких є параметр *upsert*.

Якщо параметр *upsert* має значення *true*, то *mongodb* буде оновлювати документ, якщо він знайдений, і створювати новий, якщо такого документа немає. Якщо ж він має значення *false*, то *mongodb* не буде створювати новий документ, якщо запит на вибірку не знайде жодного документа.

```
db.users.replaceOne({name: "Ron"}, {name: "Ron", age: 57})
```

У цьому випадку знаходимо документ, у якому `name = "Ron"`, і замінюємо його документом `{name: "Ron", age: 57}`

Після виконання операції консоль поверне на результат оновлення:



```

{
  _id: ObjectId('17f56abb34d8e543512da124'),
  name: 'Parker',
  age: 45,
  languages: [ 'english', 'italian', '' ]
},
{
  _id: ObjectId('65b269d3d7065689ca05d50f'),
  name: 'Sam',
  age: 31,
  friends: [ { name: 'Tom' } ]
},
{
  _id: ObjectId('65b240dad7065689ca05d505'),
  name: 'Sarah',
  age: 30,
  languages: [ 'english', 'spanish' ]
},
{
  _id: ObjectId('65b2472cd7065689ca05d506'),
  name: 'Tarantino',
  age: 60,
  awards: [ 'oscar', 'golden globe' ]
}
]
test> db.users.replaceOne({name: "Ron"}, {name: "Ron", age: 57})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> |

```

Рисунок 46 – Результат виконання функції `replaceOne()`

В отриманому результаті параметр *matchedCount* вказує на кількість документів, які відповідають запиту. А параметр *modifiedCount* вказує на кількість змінених документів. Тобто в даному випадку відповідає запиту 1 документ і він же був змінений.

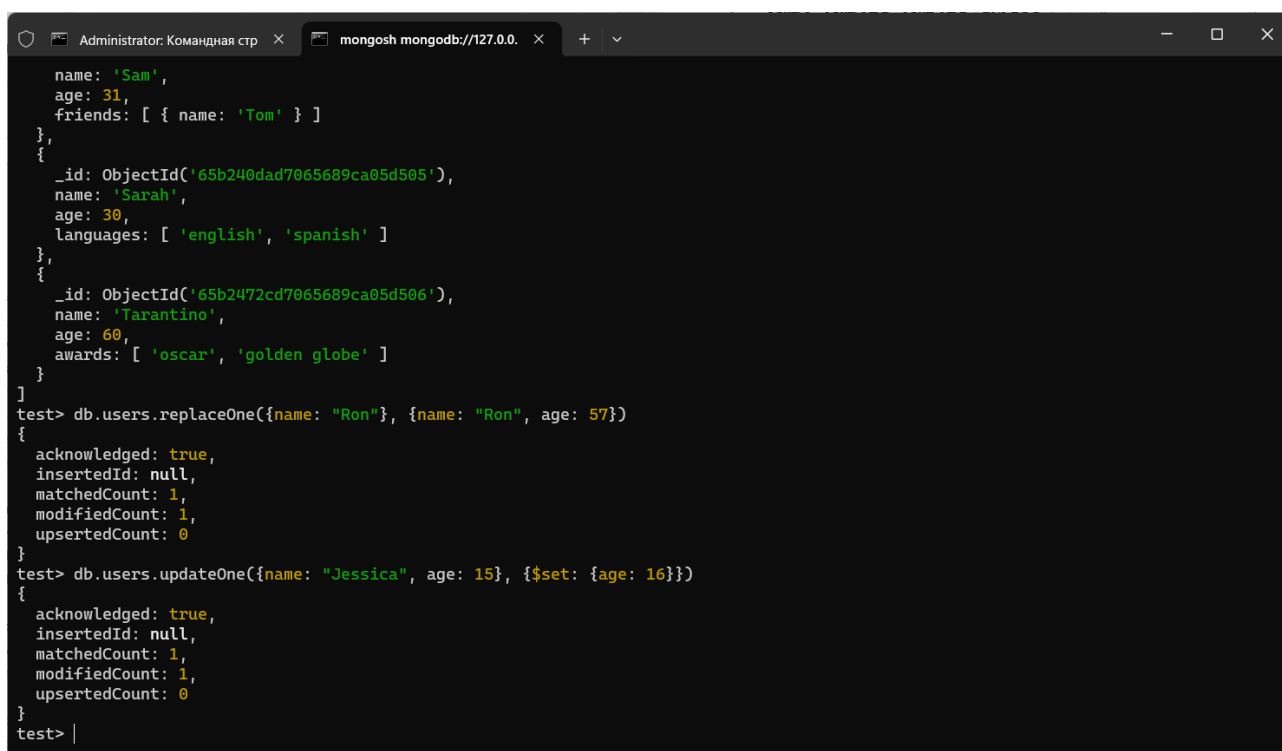
9.2 Функції `updateOne()` і `updateMany()`

Часто не потрібно оновлювати весь документ, а тільки значення однієї або декількох його властивостей. Для цього застосовуються функції `updateOne()` (оновлює тільки один документ) і `updateMany()` (оновлює безліч документів).

Для оновлення окремих полів у цих функціях застосовується оператор `$set`. Якщо документ не містить оновлюваного поля, то воно створюється.

```
db.users.updateOne({name: "Jessica", age: 15}, {$set: {age: 16}})
```

Тут ми шукаємо документ з `name="Jessica"` і `age=15` і встановлюємо для його властивості `age` значення 16.



```

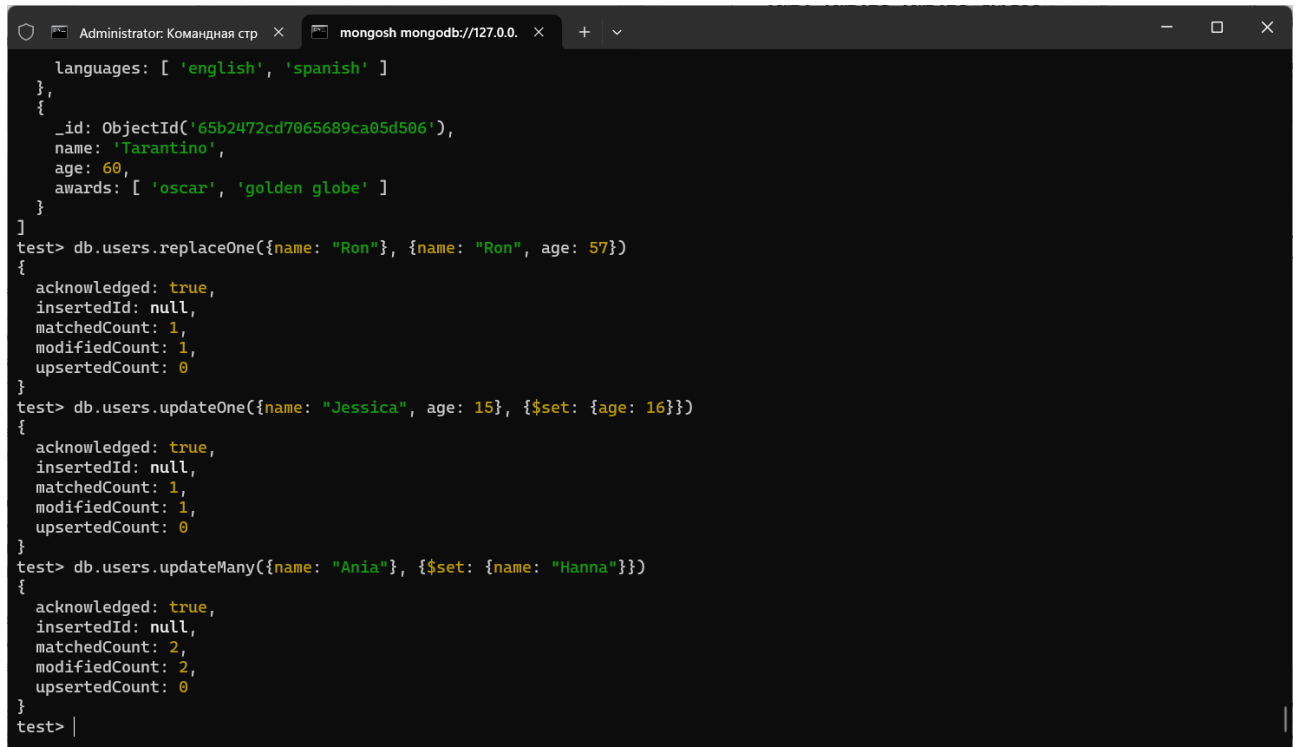
name: 'Sam',
age: 31,
friends: [ { name: 'Tom' } ]
},
{
  _id: ObjectId('65b240dad7065689ca05d505'),
  name: 'Sarah',
  age: 30,
  languages: [ 'english', 'spanish' ]
},
{
  _id: ObjectId('65b2472cd7065689ca05d506'),
  name: 'Tarantino',
  age: 60,
  awards: [ 'oscar', 'golden globe' ]
}
]
test> db.users.replaceOne({name: "Ron"}, {name: "Ron", age: 57})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.updateOne({name: "Jessica", age: 15}, {$set: {age: 16}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> |

```

Рисунок 47 – Результат виконання функції `updateOne()`

Якщо необхідно оновити всі документи, що відповідають деякому критерію, то застосовується функція *updateMany()*:

```
db.users.updateMany({name: "Ania"}, {$set: {name: "Hanna"}})
```



```
Administrator: Командная строка  mongosh mongodb://127.0.0.1:27021/
{
  languages: [ 'english', 'spanish' ]
},
{
  _id: ObjectId('65b2472cd7065689ca05d506'),
  name: 'Tarantino',
  age: 60,
  awards: [ 'oscar', 'golden globe' ]
}
]
test> db.users.replaceOne({name: "Ron"}, {name: "Ron", age: 57})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.updateOne({name: "Jessica", age: 15}, {$set: {age: 16}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.updateMany({name: "Ania"}, {$set: {name: "Hanna"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
test> |
```

Рисунок 48 – Результат виконання функції *updateMany()*

Якщо оновлюваного поля в документі немає, то воно додається:

```
db.users.updateOne({name : "Hanna", age: 28}, {$set: {university:
“oxford”}})
```

```

Administrator: Командная стр  ×  mongosh mongodb://127.0.0.  ×  +  ▾
{
  friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
},
{
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Hanna',
  age: 30,
  languages: [ 'english', 'polish' ]
}
]
test> db.users.updateOne({name : "Hanna", age: 28}, {$set: {university: "oxford"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name : "Hanna"})
[
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Hanna',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ],
    university: 'oxford'
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Hanna',
    age: 30,
    languages: [ 'english', 'polish' ]
  }
]
test> |

```

Результат 4.49 – Додавання нового поля

Якщо треба оновити значення декількох полів, то вони передаються оператору `$set` через кому:

```
db.users.updateOne({name : "Sam"}, {$set: {name: "Samuel", age : 45}})
```

Для простого збільшення значення числового поля на певну кількість одиниць застосовується оператор `$inc`. Якщо документ не містить оновлюваного поля, то воно створюється. Цей оператор застосовується тільки до числових значень.

```
db.users.updateOne({name: "Samuel"}, {$inc: {age:2}})
```

```

}
{
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Hanna',
  age: 30,
  languages: [ 'english', 'polish' ]
}
]
test> db.users.updateOne({name : "Sam"}, {$set: {name: "Samuel", age : 45}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.updateOne({name: "Samuel"}, {$inc: {age:2}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name: "Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ]
  }
]
test> |

```

9.3 Видалення поля

Для видалення окремого ключа використовується оператор \$unset:

```
db.users.updateOne({name : "Hanna"}, {$unset: { university: 1}})
```

```

}
test> db.users.find({name: "Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ]
  }
]
test> db.users.updateOne({name : "Hanna"}, {$unset: { university: 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name: "Hanna"})
[
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Hanna',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Hanna',
    age: 30,
    languages: [ 'english', 'polish' ]
  }
]
test> |

```

Рисунок 50 – Результат видалення ключа

Якщо раптом подібного ключа в документі не існує, то оператор не має жодного впливу. Також можна видаляти відразу кілька полів:

```
db.users.updateOne({name : "Samuel"}, {$unset: {friends: 1, age: 1}})
```

9.4 Оновлення масивів

Оператор \$push

Оператор *\$push* дає змогу додати ще одне значення до вже наявного. Наприклад, якщо ключ як значення зберігає масив:

```
db.users.updateOne({name : "Hanna"}, {$push: {languages: "ukrainian"}})
```

Вище використовувалася функція `updateOne`, але цей оператор також застосовується і в функції `updateMany`

```
db.users.updateMany({name : "Hanna"}, {$push: {languages: "portuguese"}})
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Hanna',
    age: 30,
    languages: [ 'english', 'polish' ]
  }
]
test> db.users.updateMany({name : "Hanna"}, {$push: {languages: "portuguese"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
test> db.users.find({name: "Hanna"})
[
  {
    _id: ObjectId('65b26c8ed7065689ca05d511'),
    name: 'Hanna',
    age: 28,
    friends: [ { name: 'Bruno' }, { name: 'Jessica' } ],
    languages: [ 'ukrainian', 'portuguese' ]
  },
  {
    _id: ObjectId('65b24c3dd7065689ca05d509'),
    name: 'Hanna',
    age: 30,
    languages: [ 'english', 'polish', 'portuguese' ]
  }
]
test> |

```

Якщо ключ, для якого ми хочемо додати значення, не представляє масив, то ми отримаємо помилку `Cannot apply $push/$pushAll modifier to non-array`.

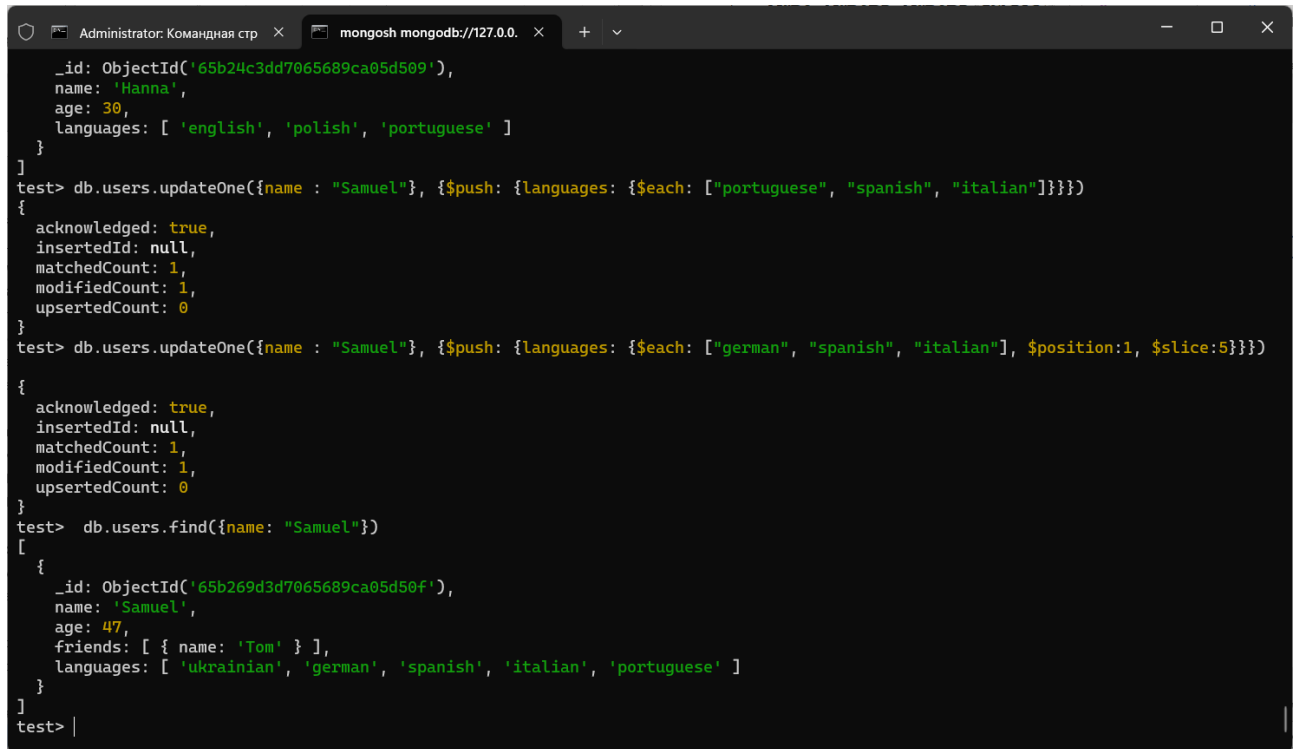
Використовуючи оператор *\$each*, можна додати відразу кілька значень:

```
db.users.updateOne({name : "Samuel"}, {$push: {languages: {$each: ["portuguese", "spanish", "italian"]}}})
```

Ще пара операторів дає змогу налаштувати вставку. Оператор *\$position* задає позицію в масиві для вставки елементів, а оператор *\$slice* вказує, скільки елементів залишити в масиві після вставки.

```
db.users.updateOne({name : "Samuel"}, {$push: {languages: {$each: ["german", "spanish", "italian"], $position:1, $slice:5}}})
```

У цьому разі елементи `["german", "spanish", "italian"]` будуть вставлятися в масив *languages* з 1-го індексу, і після вставки, в масиві залишаться тільки 5 перших елементів.



```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
  _id: ObjectId('65b24c3dd7065689ca05d509'),
  name: 'Hanna',
  age: 30,
  languages: [ 'english', 'polish', 'portuguese' ]
}
]
test> db.users.updateOne({name : "Samuel"}, {$push: {languages: {$each: ["portuguese", "spanish", "italian"]}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.updateOne({name : "Samuel"}, {$push: {languages: {$each: ["german", "spanish", "italian"], $position:1, $slice:5}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name: "Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'ukrainian', 'german', 'spanish', 'italian', 'portuguese' ]
  }
]
test> |

```

Рисунок 51 – Результат оновлення масиву

Оператор \$addToSet

Оператор *\$addToSet* подібно до оператора *\$push* додає об'єкти в масив. Відмінність полягає в тому, що *\$addToSet* додає дані, якщо їх ще немає в масиві (при додаванні через *\$push* дані дублюються, якщо додаються елементи, які вже є в масиві):

```
db.users.updateOne({name : "Bruno"}, {$addToSet: {languages: "ukrainian"}})
```

```

[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'ukrainian', 'german', 'spanish', 'italian', 'portuguese' ]
  }
]
test> db.users.updateOne({name : "Bruno"}, {$addToSet: {languages: "ukrainian"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name:"Bruno"})
[
  {
    _id: ObjectId('65b26c94d7065689ca05d512'),
    name: 'Bruno',
    age: 29,
    friends: [ { name: 'Ania' }, { name: 'Jessica' } ],
    languages: [ 'ukrainian' ]
  },
  {
    _id: ObjectId('65b247f6d7065689ca05d508'),
    name: 'Bruno',
    age: 32,
    languages: [ 'spanish', 'french' ]
  }
]
test> |

```

Рисунок 52 – Результат застосування оператора \$addToSet

10 Видалення елемента з масиву

Оператор *\$pop* дозволяє видаляти елемент із масиву:

```
db.users.updateOne({name : "Samuel"}, {$pop: {languages: 1}})
```

Вказуючи для ключа *languages* значення 1, ми видаляємо перший елемент з кінця.

Щоб видалити перший елемент спочатку масиву, треба передати від'ємне значення:

```
db.users.updateOne({name: "Samuel"}, {$pop: {languages: -1}})
```

```

insertedId: null,
matchedCount: 1,
modifiedCount: 1,
upsertedCount: 0
}
test> db.users.find({name:"Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'ukrainian', 'german', 'spanish', 'italian' ]
  }
]
test> db.users.updateOne({name : "Samuel"}, {$pop: {languages: -1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name:"Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'german', 'spanish', 'italian' ]
  }
]
test> |

```

Рисунок 53 – Результат видалення елемента масиву

Дещо іншу дію передбачає оператор *\$pull*. Він видаляє кожне входження елемента в масив. Наприклад, через оператор *\$push* ми можемо додати одне й те саме значення в масив кілька разів. І тепер за допомогою *\$pull* видалимо його:

```
db.users.updateOne({name: "Samuel"}, {$pull: {languages: "english"}})
```

А якщо ми хочемо видалити не одне значення, а відразу кілька, тоді ми можемо застосувати оператор *\$pullAll*:

```
db.users.updateOne({name: "Samuel"}, {$pullAll: {languages: ["english", "german", "french"]}})
```



```

insertedId: null,
matchedCount: 1,
modifiedCount: 1,
upsertedCount: 0
}
test> db.users.find({name:"Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'german', 'spanish', 'italian' ]
  }
]
test> db.users.updateOne({name: "Samuel"}, {$pullAll: {languages: ["english", "german", "french"]}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name:"Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'spanish', 'italian' ]
  }
]
test> |

```

Рисунок 54 – Результат видалення елементів масиву

11 Видалення даних

Для видалення документів у *MongoDB* передбачено функції *deleteOne()* - видаляє один документ і *deleteMany()* - дає змогу видалити кілька документів. Як параметр у ці функції передається фільтр документів, що видаляються.

11.1 Видалення документа

Наприклад, видалимо документ, у якому `name="Hanna"`:

```
db.users.deleteOne({name : "Hanna"})
```

У підсумку перший знайдений документ з `name= Hanna` буде видалено. Після видалення консоль відображає нам об'єкт, у якому параметр `deletedCount` вказує на кількість видалених документів:

```

modifiedCount: 1,
upsertedCount: 0
}
test> db.users.find({name:"Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'german', 'spanish', 'italian' ]
  }
]
test> db.users.updateOne({name: "Samuel"}, {$pullAll: {languages: ["english", "german", "french"]}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.users.find({name:"Samuel"})
[
  {
    _id: ObjectId('65b269d3d7065689ca05d50f'),
    name: 'Samuel',
    age: 47,
    friends: [ { name: 'Tom' } ],
    languages: [ 'spanish', 'italian' ]
  }
]
test> db.users.deleteOne({name : "Hanna"})
{ acknowledged: true, deletedCount: 1 }
test>

```

Рисунок 55 – Результат видалення документа

Для видалення всіх документів, які відповідають фільтру, застосовується функція *deleteMany()*:

```
db.users.deleteMany({name : "Hanna"})
```

Причому, як і у випадку з *find()*, ми можемо задавати умови вибірки для видалення різними способами (у вигляді регулярних виразів, у вигляді умовних конструкцій тощо):

```
db.users.deleteOne({name : /^T\w+/i})
db.users.deleteOne({age: {$lt : 30}})
```

Щоб видалити разом усі документи з колекції, треба залишити порожнім параметр запиту:

```
db.users.deleteMany({})
```

11.2 Видалення колекцій і баз даних

Ми можемо видаляти не тільки документи, а й колекції та бази даних. Для видалення колекцій використовується функція *drop()*:

```
db.users.drop()
```

І якщо видалення колекції пройде успішно, то консоль виведе:

```
true
```

Щоб видалити всю базу даних, треба скористатися функцією *dropDatabase()*:

```
db.dropDatabase()
```

12 Встановлення посилань у БД

У реляційних базах даних можна встановлювати зовнішні ключі, коли поля з однієї таблиці посилаються на поля в іншій таблиці. У *MongoDB* також можна встановлювати посилання. Далі ми розглянемо встановлення посилань між колекцією *users*, яка зберігає користувачів, і колекцією *companies*, яка зберігає список компаній, у яких працюють користувачі.

12.1 Ручне встановлення посилань

Ручне встановлення посилань зводиться до присвоєння значення поля *_id* одного документа полю іншого документа. Припустимо, у нас можуть бути колекції, що представляють компанії та працівників, які працюють у цих компаніях. Отже, спочатку додамо в колекцію *companies* документ, що представляє компанію:

```
db.companies.insertOne({"_id" : "apple", head: "Steve Jobs", year: 1976})
```

Тепер додамо в колекцію *clients* документ, який представляє працівника. У цьому документі буде поле *company*, що представляє компанію, де працює працівник. І дуже важливо, що як значення для цього поля ми встановлюємо не об'єкт *company*, а значення ключа *_id* доданого вище документа:

```
db.clients.insertOne({name: "Sam", age: 30, company: "apple"})
```

Тепер отримаємо документ із колекції *clients*:

```
client = db.clients.findOne()
```

У цьому разі мається на увазі, що вище доданий елемент буде єдиним у колекції. Інакше треба зробити вибірку за *_id* останнього доданого документа.

Після цього консоль виводить отриманий документ. І тепер знайдемо посилання на його компанію в колекції *companies*:

```
db.companies.findOne({"_id": client.company})
```

І якщо документ із таким ідентифікатором виявлено, він відображається на консолі:

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
test> db.clients.find()
[
  { _id: ObjectId('65b40c0ed7065689ca05d51c'), name: 'John', age: 25 },
  { _id: ObjectId('65b40c21d7065689ca05d51d'), name: 'Jack', age: 30 },
  {
    _id: ObjectId('65b40c8ad7065689ca05d51e'),
    name: 'Sam',
    age: 30,
    company: 'apple'
  }
]
test> db.clients.deleteOne({name: 'John'})
{ acknowledged: true, deletedCount: 1 }
test> db.clients.deleteOne({name: 'Jack'})
{ acknowledged: true, deletedCount: 1 }
test> db.clients.find()
[
  {
    _id: ObjectId('65b40c8ad7065689ca05d51e'),
    name: 'Sam',
    age: 30,
    company: 'apple'
  }
]
test> client = db.clients.findOne()
{
  _id: ObjectId('65b40c8ad7065689ca05d51e'),
  name: 'Sam',
  age: 30,
  company: 'apple'
}
test> db.companies.findOne({_id: client.company})
{ _id: 'apple', head: 'Steve Jobs', year: 1976 }
test>

```

12.2 Автоматичне зв'язування

Використовуючи функціональність DBRef, ми можемо встановити автоматичне зв'язування між документами. Подивимося на прикладі застосування цієї функціональності. Спочатку додамо новий документ у колекцію *companies*:

```

microsoft = db.companies.insertOne({name : "microsoft", head:
"Bill Gates", year: 1974})

```

Зверніть увагу, що ми отримуємо результат додавання у змінну *microsoft*. При додаванні нового документа генерує *_id*, який ми можемо отримати за допомогою властивості *insertedId* результату функції.

Тепер створимо новий документ для колекції *clients*, у якого ключ *company* зв'яжемо зі щойно доданим документом *microsoft*:

```

bran = ({name: "Bran", age: 25, company: { "$ref" : "companies",
"$id" : microsoft.insertedId}})
db.clients.insertOne(bran)

```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
{
  _id: ObjectId('65b40c8ad7065689ca05d51e'),
  name: 'Sam',
  age: 30,
  company: 'apple'
}
test> db.companies.findOne({_id: client.company})
{ _id: 'apple', head: 'Steve Jobs', year: 1976 }
test> db.companies.insertOne({name : "google", head: "Larry Page", year: 1988})
{
  acknowledged: true,
  insertedId: ObjectId('65b40f61d7065689ca05d51f')
}
test> google = db.companies.insertOne({name : "google", head: "Larry Page", year: 1988})
{
  acknowledged: true,
  insertedId: ObjectId('65b40fadd7065689ca05d520')
}
test> microsoft = db.companies.insertOne({name : "microsoft", head: "Bill Gates", year: 1974})
{
  acknowledged: true,
  insertedId: ObjectId('65b4102cd7065689ca05d521')
}
test> bran = ({name: "Bran", age: 25, company: { "$ref" : "companies", "$id" : microsoft.insertedId}})
{
  name: 'Bran',
  age: 25,
  company: { '$ref': 'companies', '$id': ObjectId('65b4102cd7065689ca05d521') }
}
test> db.clients.insertOne(bran)
{
  acknowledged: true,
  insertedId: ObjectId('65b410ddd7065689ca05d522')
}
test> |

```

Рисунок 56 – Результати автоматичного зв'язування документу

І ми можемо протестувати:

```
db.companies.findOne({_id: bran.company.$id})
```

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.  x  +  v
test> db.companies.insertOne({name : "google", head: "Larry Page", year: 1988})
{
  acknowledged: true,
  insertedId: ObjectId('65b40f61d7065689ca05d51f')
}
test> google = db.companies.insertOne({name : "google", head: "Larry Page", year: 1988})
{
  acknowledged: true,
  insertedId: ObjectId('65b40fadd7065689ca05d520')
}
test> microsoft = db.companies.insertOne({name : "microsoft", head: "Bill Gates", year: 1974})
{
  acknowledged: true,
  insertedId: ObjectId('65b4102cd7065689ca05d521')
}
test> bran = ({name: "Bran", age: 25, company: { "$ref" : "companies", "$id" : microsoft.insertedId}})
{
  name: 'Bran',
  age: 25,
  company: { '$ref': 'companies', '$id': ObjectId('65b4102cd7065689ca05d521') }
}
test> db.clients.insertOne(bran)
{
  acknowledged: true,
  insertedId: ObjectId('65b410ddd7065689ca05d522')
}
test> db.companies.findOne({_id: bran.company.$id})
{
  _id: ObjectId('65b4102cd7065689ca05d521'),
  name: 'microsoft',
  head: 'Bill Gates',
  year: 1974
}
test> |

```

Подивившись на прикладі, тепер розберемо організацію посилань між документами. Для зв'язування з документом *microsoft* використовувався такий вираз `{ "$ref" : "companies", "$id" : microsoft.insertedId }`. Формальний синтаксис DBRef такий:

```
{ "$ref" : назва_колекції, "$id": значення [, "$db" : назва_бд ] }
```

Перший параметр `$ref` вказує на колекцію, де зберігається пов'язаний документ. Другий параметр вказує на значення, яке і буде представляти щось типу зовнішнього ключа. Третій необов'язковий параметр вказує на базу даних.

Під час тестування як запит на вибірку вказується вираз `_id: bran.company.$id`. Оскільки *bran.company* представляє тепер об'єкт `{ "$ref" : "companies", "$id" : microsoft.insertedId }`, то нам треба конкретизувати параметр *bran.company.\$id*

13 Керування колекцією

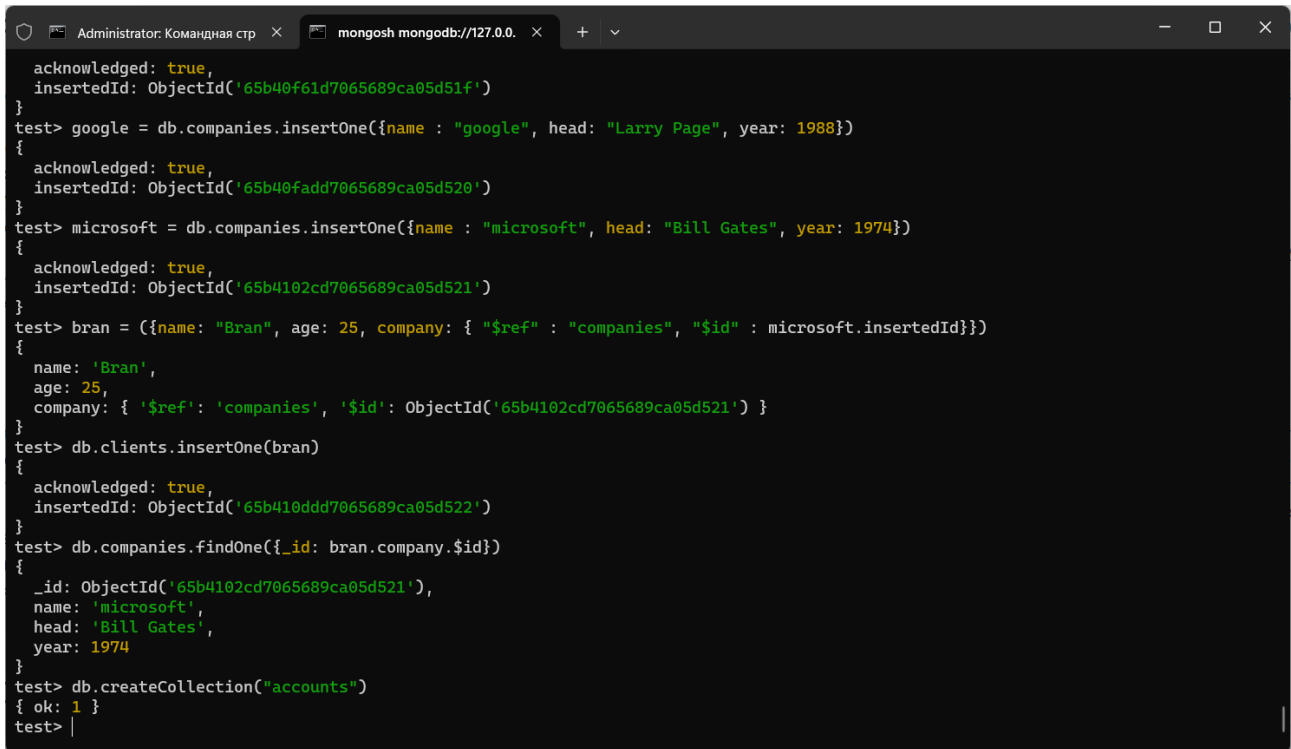
13.1 Явне створення колекції

У попередніх темах колекція створювалася неявно автоматично під час додавання в неї перших даних. Але ми також можемо створити її явним чином, застосувавши метод `db.createCollection(name, options)`, де *name* - назва колекції, а *options* - необов'язковий об'єкт із додатковими налаштуваннями ініціалізації.

Наприклад:

```
db.createCollection("accounts")
```

Таким чином, створюється колекція `accounts`.



```

Administrator: Командная строка x mongosh mongod://127.0.0.1 x
{
  acknowledged: true,
  insertedId: ObjectId('65b40f61d7065689ca05d51f')
}
test> google = db.companies.insertOne({name : "google", head: "Larry Page", year: 1988})
{
  acknowledged: true,
  insertedId: ObjectId('65b40fadd7065689ca05d520')
}
test> microsoft = db.companies.insertOne({name : "microsoft", head: "Bill Gates", year: 1974})
{
  acknowledged: true,
  insertedId: ObjectId('65b4102cd7065689ca05d521')
}
test> bran = ({name: "Bran", age: 25, company: { "$ref" : "companies", "$id" : microsoft.insertedId}})
{
  name: 'Bran',
  age: 25,
  company: { '$ref': 'companies', '$id': ObjectId('65b4102cd7065689ca05d521') }
}
test> db.clients.insertOne(bran)
{
  acknowledged: true,
  insertedId: ObjectId('65b410ddd7065689ca05d522')
}
test> db.companies.findOne({_id: bran.company.$id})
{
  _id: ObjectId('65b4102cd7065689ca05d521'),
  name: 'microsoft',
  head: 'Bill Gates',
  year: 1974
}
test> db.createCollection("accounts")
{ ok: 1 }
test>

```

Рисунок 57 – Результат створення нової колекції

13.2 Перейменування колекції

У процесі роботи, можливо, буде потрібно змінити назву колекції. Наприклад, якщо під час першого додавання даних у її назві була помилка. І щоб не видаляти і потім перестворювати колекцію, слід використовувати функцію *renameCollection()*:

```
db.users.renameCollection("нова_назва")
```

І якщо перейменування пройде вдало, то консоль відобразить:


```

test> google = db.companies.insertOne({name : "google", head: "Larry Page", year: 1988})
{
  acknowledged: true,
  insertedId: ObjectId('65b40fadd7065689ca05d520')
}
test> microsoft = db.companies.insertOne({name : "microsoft", head: "Bill Gates", year: 1974})
{
  acknowledged: true,
  insertedId: ObjectId('65b4102cd7065689ca05d521')
}
test> bran = ({name: "Bran", age: 25, company: { "$ref" : "companies", "$id" : microsoft.insertedId}})
{
  name: 'Bran',
  age: 25,
  company: { '$ref': 'companies', '$id': ObjectId('65b4102cd7065689ca05d521') }
}
test> db.clients.insertOne(bran)
{
  acknowledged: true,
  insertedId: ObjectId('65b410ddd7065689ca05d522')
}
test> db.companies.findOne({_id: bran.company.$id})
{
  _id: ObjectId('65b4102cd7065689ca05d521'),
  name: 'microsoft',
  head: 'Bill Gates',
  year: 1974
}
test> db.createCollection("accounts")
{ ok: 1 }
test> db.accounts.renameCollection("students")
{ ok: 1 }
test> |

```

Рисунок 58 – Результат переіменування колекції

13.3 Обмежені колекції

Коли ми відправляємо запит до БД на вибірку, то *MongoDB* повертає нам документи в тому порядку, як правило, в якому вони були додані. Однак такий порядок не завжди гарантується, оскільки дані можуть бути видалені, переміщені, змінені. Тому в *MongoDB* існує поняття обмеженої колекції (*capped collection*). Подібна колекція гарантує, що документи розташовуватимуться в тому самому порядку, в якому їх додавали в колекцію. Обмежені колекції мають фіксований розмір. І коли в колекції вже немає місця, найстаріші документи видаляються, і в кінець додаються нові дані.

На відміну від звичайних колекцій обмежені ми можемо задати явним чином. Наприклад, створимо обмежену колекцію з назвою *profiles* і задамо для неї розмір у 9500 байт:

```
db.createCollection("profiles", {capped:true, size:9500})
```

І після вдалого створення колекції консоль виведе:

```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1  x  +  v
{
  acknowledged: true,
  insertedId: ObjectId('65b40fadd7065689ca05d520')
}
test> microsoft = db.companies.insertOne({name : "microsoft", head: "Bill Gates", year: 1974})
{
  acknowledged: true,
  insertedId: ObjectId('65b4102cd7065689ca05d521')
}
test> bran = ({name: "Bran", age: 25, company: { "$ref" : "companies", "$id" : microsoft.insertedId}})
{
  name: 'Bran',
  age: 25,
  company: { '$ref': 'companies', '$id': ObjectId('65b4102cd7065689ca05d521') }
}
test> db.clients.insertOne(bran)
{
  acknowledged: true,
  insertedId: ObjectId('65b410ddd7065689ca05d522')
}
test> db.companies.findOne({_id: bran.company.$id})
{
  _id: ObjectId('65b4102cd7065689ca05d521'),
  name: 'microsoft',
  head: 'Bill Gates',
  year: 1974
}
test> db.createCollection("accounts")
{ ok: 1 }
test> db.accounts.renameCollection("students")
{ ok: 1 }
test> db.createCollection("profiles", {capped:true, size:9500})
{ ok: 1 }
test> |

```

Також можна обмежити кількість документів у колекції, вказавши її в параметрі max:

```
db.createCollection("profiles", {capped:true, size:9500, max:150})
```

Однак за такого способу створення колекції слід враховувати, що якщо все місце під колекцію заповнене (наприклад, виділені нами 9500 байтів), а кількість документів ще не досягла максимуму, в даному випадку 150, то в цьому випадку під час додавання нового документа найстаріший документ видалятиметься, а на його місце вставлятиметься новий документ.

```

Administrator: Командная строка x mongosh mongodb://127.0.0.1:27021
insertedId: ObjectId('65b40fadd7065689ca05d520')
}
test> microsoft = db.companies.insertOne({name : "microsoft", head: "Bill Gates", year: 1974})
{
  acknowledged: true,
  insertedId: ObjectId('65b4102cd7065689ca05d521')
}
test> bran = ({name: "Bran", age: 25, company: { "$ref" : "companies", "$id" : microsoft.insertedId}})
{
  name: 'Bran',
  age: 25,
  company: { '$ref': 'companies', '$id': ObjectId('65b4102cd7065689ca05d521') }
}
test> db.clients.insertOne(bran)
{
  acknowledged: true,
  insertedId: ObjectId('65b410ddd7065689ca05d522')
}
test> db.companies.findOne({_id: bran.company.$id})
{
  _id: ObjectId('65b4102cd7065689ca05d521'),
  name: 'microsoft',
  head: 'Bill Gates',
  year: 1974
}
test> db.createCollection("accounts")
{ ok: 1 }
test> db.accounts.renameCollection("students")
{ ok: 1 }
test> db.createCollection("profiles", {capped:true, size:9500})
{ ok: 1 }
test> db.createCollection("owners", {capped:true, size:9500, max: 150})
{ ok: 1 }
test>

```

Під час оновлення документів у таких колекціях треба пам'ятати, що документи не повинні зростати в розмірах, інакше оновлення не вдасться зробити.

Також не можна видаляти документи з подібних колекцій, можна тільки видалити всю колекцію.

13.4 Підколекції

Для спрощення організації даних у колекціях ми можемо використовувати підколекції. Наприклад, дані з колекції `users` треба розмежувати на профілі та облікові дані. І ми можемо використовувати створити колекції `db.users.profiles` і `db.users.accounts`. При цьому вони не будуть ніяк пов'язані з колекцією `users`. Тобто в підсумку будуть три різні колекції, проте в плані логічної організації зберігання даних, можливо, для когось так буде простіше.

14 Робота з БД MongoDB у Python

Веб-додатки, як і багато інших програмних додатків, живляться від даних. Організація і зберігання цих даних є важливими, оскільки вони визначають, як ми

взаємодіємо з різними додатками, що перебувають у нашому розпорядженні. Вид оброблюваних даних також може впливати на те, як ми робимо цей процес.

Бази даних дають нам змогу організовувати та зберігати ці дані, а також контролювати, як ми зберігаємо, отримуємо доступ і захищаємо інформацію.

14.1 Бази даних NoSQL

Існує два основних типи баз даних - реляційні та нереляційні бази даних.

Реляційні бази даних дають нам змогу зберігати, отримувати доступ і маніпулювати даними стосовно іншого фрагмента даних у базі даних. Дані зберігаються в організованих таблицях із рядками та стовпчиками зі зв'язками, що зв'язують інформацію між таблицями. Для роботи з цими базами даних ми використовуємо мову структурованих запитів (SQL), наприклад MySQL і PostgreSQL.

Нереляційні бази даних зберігають дані ні в реляційній, ні в табличній формі, як у реляційних базах даних. Вони також називаються базами даних NoSQL, оскільки ми не використовуємо SQL для взаємодії з ними.

14.2 MongoDB і коли його використовувати

MongoDB - це сховище документів і нереляційна база даних. Це дозволяє нам зберігати дані в колекціях, які складені з документів. У MongoDB документ являє собою просто JSON-подібний двійковий формат серіалізації, званий BSON або Binary-JSON, і має максимальний розмір 16 мегабайт. Це обмеження розміру використовується для забезпечення ефективного використання пам'яті та смуги пропускання під час передачі.

MongoDB також надає специфікацію GridFS у разі необхідності зберігати файли, розмір яких перевищує встановлене обмеження.

Документи складаються з пар ключ-значення, як у звичайних даних JSON. Однак цей формат BSON також може містити більше типів даних, таких як Date і Binary Data. BSON був розроблений, щоб бути легким, легко прохідним і ефективним під час кодування і декодування даних в і з BSON.

Будучи сховищем даних NoSQL, MongoDB дозволяє нам користуватися перевагами використання нереляційної бази даних порівняно з реляційною. Однією з переваг є те, що він забезпечує високу масштабованість за рахунок ефективного масштабування по горизонталі за рахунок поділу даних і розміщення їх на декількох комп'ютерах.

MongoDB також дає нам змогу зберігати великі обсяги структурованих, напівструктурованих і неструктурованих даних без необхідності підтримувати відносини між ними.

Як і в будь-якого іншого рішення, у MongoDB є свої недоліки. Перший полягає в тому, що він не підтримує відносини між збереженими даними. Через це важко виконувати транзакції ACID, які забезпечують узгодженість. Складність збільшується при спробі підтримки транзакцій ACID. MongoDB, як і інші сховища даних NoSQL, не настільки розвинений, як реляційні бази даних, і це може ускладнити пошук експертів.

Нереляційна природа MongoDB робить його ідеальним для зберігання даних у певних ситуаціях порівняно з його реляційними аналогами. Наприклад, сценарій, де MongoDB є більш підходящим, ніж реляційна база даних, - це коли формат даних є гнучким і не має відносин. З гнучкими / нереляційними даними нам не потрібно підтримувати властивості ACID під час зберігання даних, на відміну від реляційних баз даних. MongoDB також дозволяє нам легко масштабувати дані в нові вузли.

Однак, незважаючи на всі свої переваги, MongoDB не ідеальний, коли наші дані мають реляційний характер. Наприклад, якщо ми зберігаємо записи клієнтів та їхні замовлення. У цій ситуації нам знадобиться реляційна база даних для підтримання відносин між нашими даними, які є важливими. Також не підходить використовувати MongoDB, якщо нам потрібно дотримуватися властивостей ACID.

14.3 Взаємодія з MongoDB через Mongo Shell

Для роботи з MongoDB нам потрібно встановити MongoDB Server, який можна завантажити з офіційної домашньої сторінки <https://www.mongodb.com/download->

[center/community](https://www.mongodb.com/center/community). Для цієї демонстрації ми будемо використовувати безкоштовний сервер спільної роботи.

MongoDB Server поставляється з оболонкою Mongo, яку ми можемо використовувати для взаємодії з сервером через термінал.

Після інсталяції треба створити на жорсткому диску каталог, у якому розміщуватимуться бази даних *MongoDB*.

В ОС Windows за замовчуванням *MongoDB* зберігає бази даних за шляхом *C:\data\db*, тому, якщо ви використовуєте Windows, вам треба створити відповідний каталог. Якщо ж виникла потреба використовувати якийсь інший шлях до файлів, то його можна передати під час запуску *MongoDB* у прапорі *--dbpath*.

Отже, після створення каталогу для зберігання БД можна запустити сервер *MongoDB*. Сервер представляє додаток *mongod*, який знаходиться в каталозі *bin* у папці сервера. Для цього запустимо термінал/командний рядок і там введемо відповідні команди. Для ОС Windows це матиме такий вигляд:

```

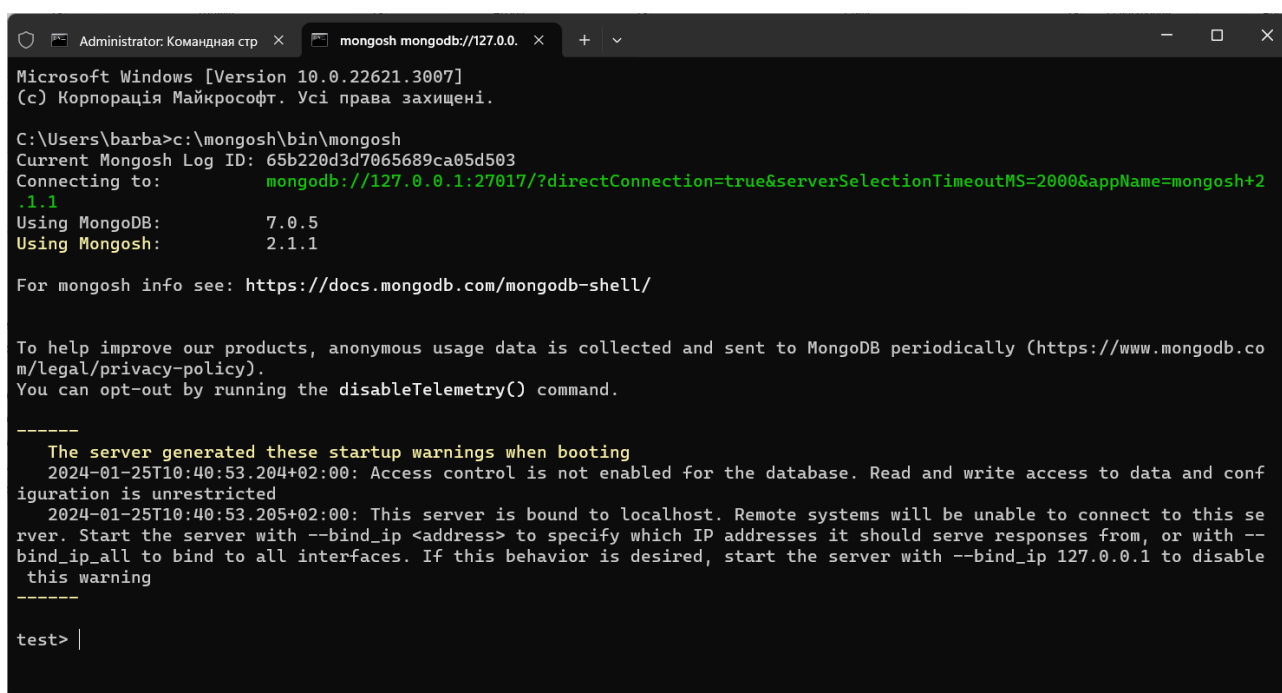
", "msg": "createCollection", "attr": {"namespace": "config.system.sessions", "uuidDisposition": "generated", "uuid": {"$
uuid": "35a87cee-b69e-4237-ad45-dc37cd7e2044"}}, "options": {}}
{"t": {"$date": "2024-01-25T10:40:54.551+02:00"}, "s": "I", "c": "NETWORK", "id": 23016, "ctx": "listener", "msg": "Waiting f
or connections", "attr": {"port": 27017, "ssl": "off"}}
{"t": {"$date": "2024-01-25T10:40:54.551+02:00"}, "s": "I", "c": "CONTROL", "id": 8423403, "ctx": "initandlisten", "msg": "mong
od startup complete", "attr": {"Summary of time elapsed": {"Startup from clean shutdown?": true, "Statistics": {"Transport lay
er setup": "0 ms", "Run initial syncer crash recovery": "0 ms", "Create storage engine lock file in the data directory": "0 m
s", "Get metadata describing storage engine": "0 ms", "Create storage engine": "60 ms", "Write current PID to file": "0 ms", "W
rite a new metadata for storage engine": "10 ms", "Initialize FCV before rebuilding indexes": "1 ms", "Drop abandoned idents
and get back indexes that need to be rebuilt or builds that need to be restarted": "0 ms", "Rebuild indexes for collectio
ns": "0 ms", "Load cluster parameters from disk for a standalone": "1 ms", "Build user and roles graph": "0 ms", "Set up the b
ackground thread pool responsible for waiting for opTimes to be majority committed": "1 ms", "Initialize information neede
d to make a mongod instance shard aware": "0 ms", "Start up the replication coordinator": "0 ms", "Start transport layer": "2
ms", "_initAndlisten total elapsed time": "1421 ms"}}}}
{"t": {"$date": "2024-01-25T10:40:54.558+02:00"}, "s": "I", "c": "REPL", "id": 7360102, "ctx": "LogicalSessionCacheRefresh
", "msg": "Added oplog entry for create to transaction", "attr": {"namespace": "config.$cmd", "uuid": {"$uuid": {"$uuid": "35a87ce
e-b69e-4237-ad45-dc37cd7e2044"}}, "object": {"create": "system.sessions", "idIndex": {"v": 2, "key": {"_id": 1}, "name": "_id_"}},
{"t": {"$date": "2024-01-25T10:40:54.558+02:00"}, "s": "I", "c": "REPL", "id": 7360100, "ctx": "LogicalSessionCacheRefresh
", "msg": "Added oplog entry for createIndexes to transaction", "attr": {"namespace": "config.$cmd", "uuid": {"$uuid": {"$uuid": "
35a87cee-b69e-4237-ad45-dc37cd7e2044"}}, "object": {"createIndexes": "system.sessions", "v": 2, "key": {"lastUse": 1}, "name": "ls
idTTLIndex", "expireAfterSeconds": 1800}}
{"t": {"$date": "2024-01-25T10:40:54.562+02:00"}, "s": "I", "c": "INDEX", "id": 20345, "ctx": "LogicalSessionCacheRefresh
", "msg": "Index build: done building", "attr": {"buildUUID": null, "collectionUUID": {"$uuid": {"$uuid": "35a87cee-b69e-4237-ad45
-dc37cd7e2044"}}, "namespace": "config.system.sessions", "index": "_id_", "ident": "index-5--5509988953087594779", "collectionI
dent": "collection-4--5509988953087594779", "commitTimestamp": null}}
{"t": {"$date": "2024-01-25T10:40:54.562+02:00"}, "s": "I", "c": "INDEX", "id": 20345, "ctx": "LogicalSessionCacheRefresh
", "msg": "Index build: done building", "attr": {"buildUUID": null, "collectionUUID": {"$uuid": {"$uuid": "35a87cee-b69e-4237-ad45
-dc37cd7e2044"}}, "namespace": "config.system.sessions", "index": "lsidTTLIndex", "ident": "index-6--5509988953087594779", "col
lectionIdent": "collection-4--5509988953087594779", "commitTimestamp": null}}

```

Рисунок 59 – Запуск сервера MongoDB mongod

Командний рядок відобразить нам низку службової інформації, наприклад, що сервер запускається на *localhost* на порту *27017*. І після вдалого запуску сервера ми зможемо проводити операції з БД через клієнт.

Використовуємо встановлений клієнтський додаток *mongosh* для взаємодії з сервером *mongodb*. (Під час роботи з *mongosh* не варто забувати, що у нас має бути запущений сервер *mongod*). Отже, запустимо файл *mongosh*, який розташовується у вище розглянутій папці встановлення:



```

Microsoft Windows [Version 10.0.22621.3007]
(c) Корпорація Майкрософт. Усі права захищені.

C:\Users\barba>c:\mongosh\bin\mongosh
Current Mongosh Log ID: 65b220d3d7065689ca05d503
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.1.1
Using MongoDB:      7.0.5
Using Mongosh:       2.1.1

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2024-01-25T10:40:53.204+02:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2024-01-25T10:40:53.205+02:00: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> |
  
```

Рисунок 60 – Запуск оболонки MongoDB Shell mongosh

Наприклад, наш сервер працює на:

```
mongodb://127.0.0.1:27017
```

У MongoDB база даних використовується для зберігання колекцій, що містять документи. Через оболонку Mongo ми можемо створити нову базу даних або переключитися на наявну, використовуючи команду *use*:

```
use SeriesDB
```

Кожна операція, яку ми виконуємо після цього, буде виконуватися в нашій базі даних SeriesDB. У базі даних ми будемо зберігати колекції, які схожі на таблиці в реляційних базах даних.

Наприклад, для цілей цього заняття додамо кілька документів у колекцію:

```
db.series.insertMany([ { name: "Game of Thrones", year: 2012}, {
name: "House of Cards", year: 2013 }, { name: "Suits", year:
2011}])
```

І в нас вийде:

```
Administrator: Командная строка x mongosh mongodb://127.0.0.1 x
Using MongoDB: 7.0.5
Using Mongosh: 2.1.1

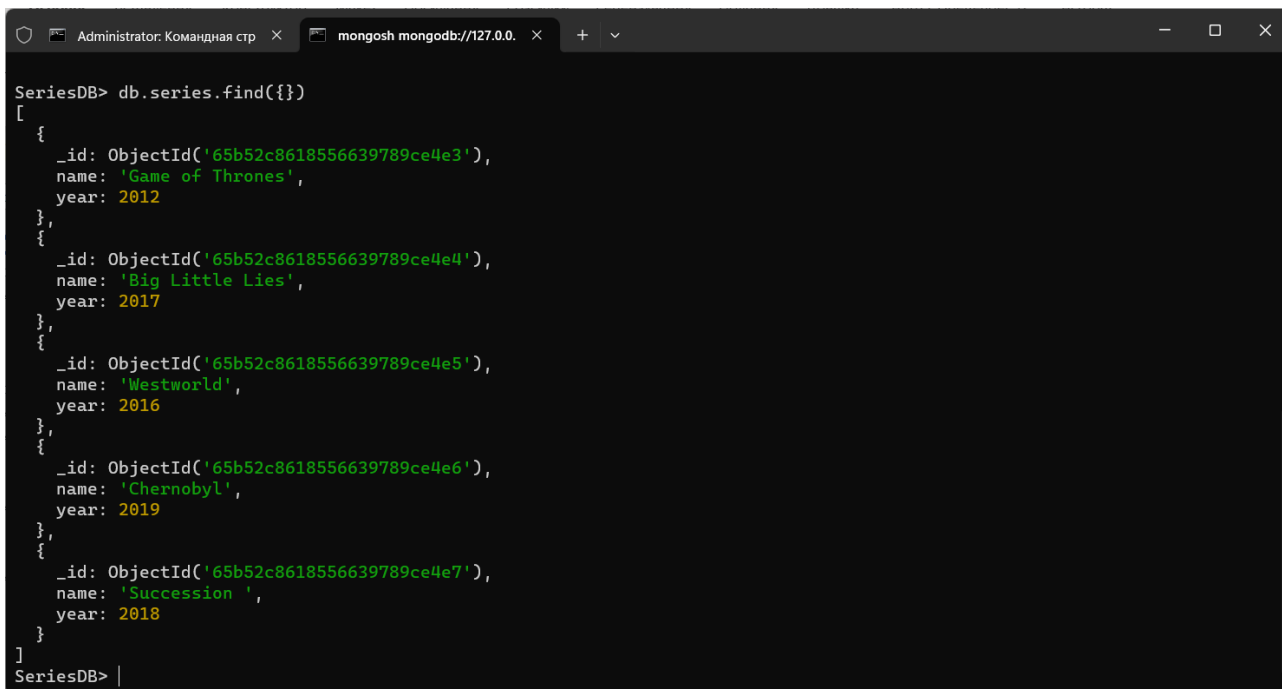
For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
The server generated these startup warnings when booting
2024-01-27T17:30:53.523+02:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2024-01-27T17:30:53.523+02:00: This server is bound to localhost. Remote systems will be unable to connect to this server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses from, or with --bind_ip_all to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1 to disable this warning
-----

test> use SeriesDB
switched to db SeriesDB
SeriesDB> db.series.insertMany([ { name: "Game of Thrones", year: 2012}, { name: "Big Little Lies", year: 2017 }, { name: "Westworld", year: 2016}, { name: "Chernobyl", year: 2019}, { name: "Succession ", year: 2018}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65b52c8618556639789ce4e3'),
    '1': ObjectId('65b52c8618556639789ce4e4'),
    '2': ObjectId('65b52c8618556639789ce4e5'),
    '3': ObjectId('65b52c8618556639789ce4e6'),
    '4': ObjectId('65b52c8618556639789ce4e7')
  }
}
SeriesDB> |
```

Рисунок 61 – Результат додавання документів

Для вилучення всіх документів, що зберігаються в нашій колекції series, ми використовуємо `db.series.find({})`, еквівалент `SQL SELECT * FROM series`. Передача порожнього запиту (тобто `{}`) поверне всі документи:



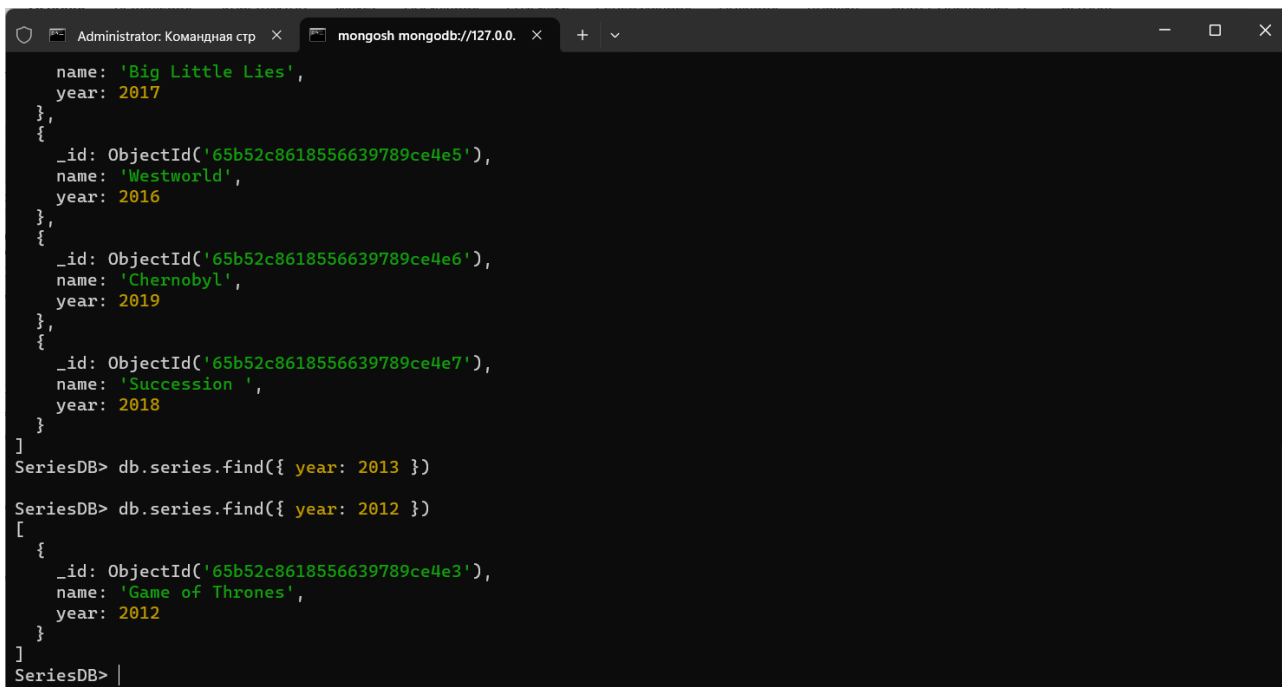
```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1  x  +  v
SeriesDB> db.series.find({})
[
  {
    _id: ObjectId('65b52c8618556639789ce4e3'),
    name: 'Game of Thrones',
    year: 2012
  },
  {
    _id: ObjectId('65b52c8618556639789ce4e4'),
    name: 'Big Little Lies',
    year: 2017
  },
  {
    _id: ObjectId('65b52c8618556639789ce4e5'),
    name: 'Westworld',
    year: 2016
  },
  {
    _id: ObjectId('65b52c8618556639789ce4e6'),
    name: 'Chernobyl',
    year: 2019
  },
  {
    _id: ObjectId('65b52c8618556639789ce4e7'),
    name: 'Succession ',
    year: 2018
  }
]
SeriesDB> |

```

Рисунок 62 – Результат отримання колекції документів

Командою `db.series.find({ year: 2012 })` також можемо запросити дані, використовуючи умову рівності, наприклад, щоб повернути всі серіали, які були показані в 2012 році:



```

Administrator: Командная стр  x  mongosh mongodb://127.0.0.1  x  +  v
name: 'Big Little Lies',
year: 2017
},
{
  _id: ObjectId('65b52c8618556639789ce4e5'),
  name: 'Westworld',
  year: 2016
},
{
  _id: ObjectId('65b52c8618556639789ce4e6'),
  name: 'Chernobyl',
  year: 2019
},
{
  _id: ObjectId('65b52c8618556639789ce4e7'),
  name: 'Succession ',
  year: 2018
}
]
SeriesDB> db.series.find({ year: 2013 })
SeriesDB> db.series.find({ year: 2012 })
[
  {
    _id: ObjectId('65b52c8618556639789ce4e3'),
    name: 'Game of Thrones',
    year: 2012
  }
]
SeriesDB> |

```

Рисунок 63 – Відображення документа за умовою

SQL-еквівалент буде `SELECT * FROM series WHERE year=2012`.

MongoDB також дозволяє нам оновлювати окремі документи за допомогою `db.collection.UpdateOne()` або виконувати пакетне оновлення за допомогою `db.collection.UpdateMany()`. Наприклад, щоб оновити рік випуску для "Big Little Lies":

```

year: 2018
}
]
SeriesDB> db.series.find({ year: 2013 })
SeriesDB> db.series.find({ year: 2012 })
[
  {
    _id: ObjectId('65b52c8618556639789ce4e3'),
    name: 'Game of Thrones',
    year: 2012
  }
]
SeriesDB> db.series.updateOne({ name: "Big Little Lies" }, {$set: { year: 2016 }})
Uncaught:
SyntaxError: Unexpected token, expected ",", (1:71)

> 1 | db.series.updateOne({ name: "Big Little Lies" }, {$set: { year: 2016 }})
    |                                     ^
    2 |

SeriesDB> db.series.updateOne({ name: "Big Little Lies" }, {$set: { year: 2016 }})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
SeriesDB>

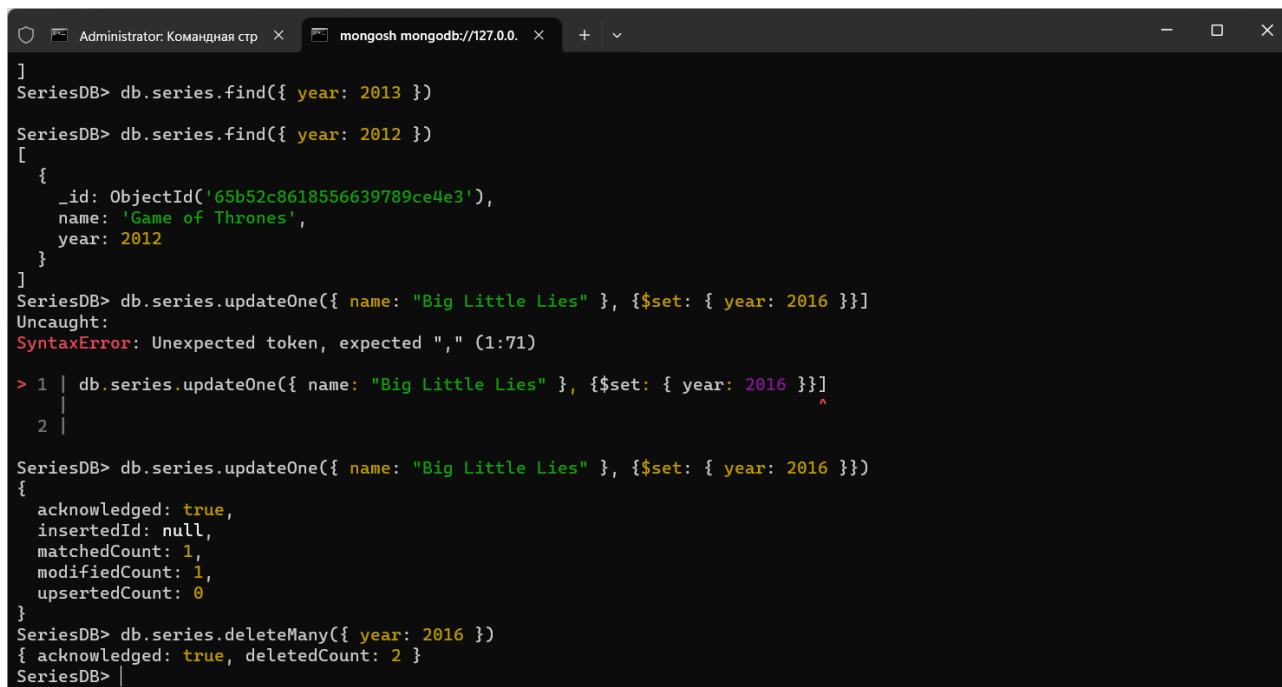
```

Рисунок 64 – Оновлення даних

Для видалення документів, *Mongo Shell* пропонує функції `db.collection.deleteOne()` і `db.collection.deleteMany()`.

Наприклад, щоб видалити всі прем'єри 2016 року, ми запустили:

```
db.series.deleteMany({ year: 2016 })
```



```

]
SeriesDB> db.series.find({ year: 2013 })

SeriesDB> db.series.find({ year: 2012 })
[
  {
    _id: ObjectId('65b52c8618556639789ce4e3'),
    name: 'Game of Thrones',
    year: 2012
  }
]
SeriesDB> db.series.updateOne({ name: "Big Little Lies" }, {$set: { year: 2016 }}]
Uncaught:
SyntaxError: Unexpected token, expected ",", (1:71)

> 1 | db.series.updateOne({ name: "Big Little Lies" }, {$set: { year: 2016 }}]
    |
    2 |

SeriesDB> db.series.updateOne({ name: "Big Little Lies" }, {$set: { year: 2016 }})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
SeriesDB> db.series.deleteMany({ year: 2016 })
{ acknowledged: true, deletedCount: 2 }
SeriesDB>

```

Рисунок 65 – Результат видалення документів з колекції

Додаткову інформацію про операції CRUD над MongoDB можна знайти в онлайн-довідці за посиланням <https://docs.mongodb.com/manual/crud/>, включно з додатковими прикладами, виконанням операцій з умовами, атомарністю і зіставленням понять SQL з поняттями і термінологією MongoDB.

14.4 Інтеграція Python з MongoDB

MongoDB – популярна NoSQL база даних, яка використовує документо-орієнтований підхід. Для роботи з MongoDB в Python можна використовувати бібліотеку *pyMongo*. *PyMongo* є офіційним драйвером MongoDB для *Python*, і ми будемо використовувати його для створення простого скрипта, який ми будемо використовувати для маніпулювання даними, що зберігаються в нашій базі даних *SeriesDB*. Для початку роботи необхідно встановити *pyMongo* та підключитися до сервера MongoDB:

```
pip install pymongo
```

```

Administrator: Командная строка
pip                23.3.2
psycpg2            2.9.9
pydantic           2.5.3
pydantic_core      2.14.6
python-dotenv      1.0.0
python-multipart   0.0.6
PyYAML             6.0.1
setuptools         69.0.3
sniffio            1.3.0
SQLAlchemy         2.0.25
starlette          0.35.1
typing_extensions  4.9.0
uvicorn            0.26.0
watchfiles         0.21.0
websockets         12.0
wheel              0.42.0

C:\Users\barba>pip install pymongo
Collecting pymongo
  Downloading pymongo-4.6.1-cp312-cp312-win_amd64.whl.metadata (22 kB)
Collecting dnspython<3.0.0,>=1.16.0 (from pymongo)
  Downloading dnspython-2.5.0-py3-none-any.whl.metadata (5.8 kB)
Download pymongo-4.6.1-cp312-cp312-win_amd64.whl (472 kB)
  472.8/472.8 kB 4.9 MB/s eta 0:00:00
Download dnspython-2.5.0-py3-none-any.whl (305 kB)
  305.4/305.4 kB 9.5 MB/s eta 0:00:00
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.5.0 pymongo-4.6.1

C:\Users\barba>

```

Рисунок 66 – Результат встановлення *pymongo*

14.5 Підключення до MongoDB

Спочатку ми імпортуємо *pymongo* в *main.py* і створюємо клієнт, підключений до нашого локально працюючого екземпляра *MongoDB*:

```

from pymongo import MongoClient

# Створюємо клієнт
client = MongoClient('localhost', 27017)

# Підключаємося до нашої бази даних
db = client['SeriesDB']

# Отримайте нашу колекцію серій
collection = db['series']

```

Ми створили клієнт, який підключається до нашого сервера MongoDB, і використовували його для отримання нашої бази даних 'SeriesDB'. Потім ми отримуємо нашу колекцію 'series' і зберігаємо її у змінній.

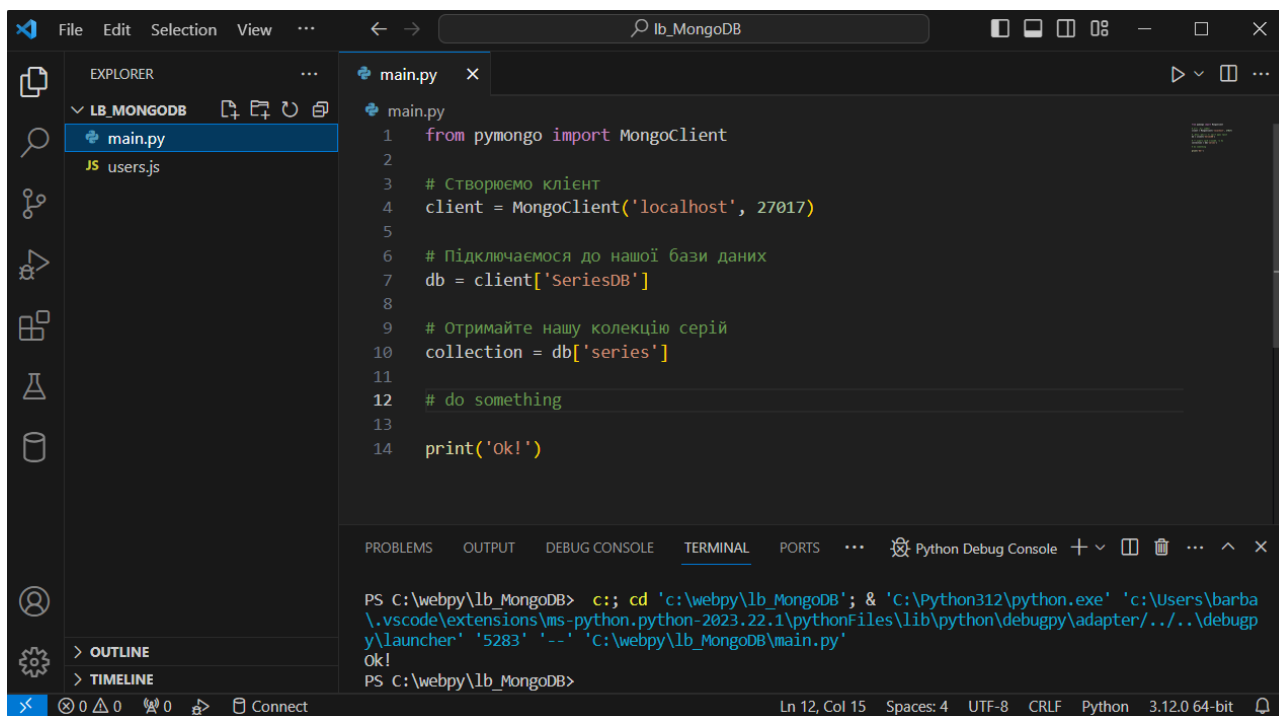


Рисунок 67 – Результат підключення до БД

14.6 Створення документів

Щоб зробити наш сценарій зручнішим, ми напишемо функції, які дозволяють нам легко маніпулювати даними з PyMongo. Ми будемо використовувати Python словники для представлення документів і передамо ці словники нашим функціям. По-перше, давайте створимо функцію для вставки даних у нашу колекцію 'series':

```

def insert_document(collection, data):
    """ Function to insert a document into a collection and
    return the document's id.
    """
    return collection.insert_one(data).inserted_id
```

Ця функція отримує колекцію та словник даних і вставляє дані в надану колекцію. Потім функція повертає ідентифікатор, який ми можемо використовувати для точного запиту окремого об'єкта з бази даних.

Слід також зазначити, що MongoDB додає додатковий ключ `_id` до наших документів, коли вони не надаються, під час створення даних.

Тепер давайте спробуємо додати шоу, використовуючи нашу функцію:

```
new_show = {
    "name": "FRIENDS",
    "year": 1994
}
print(insert_document(collection, new_show))
```

Коли ми запускаємо наш скрипт, `_id` нового запису роздруковується в терміналі, і ми можемо використовувати цей ідентифікатор для отримання запису пізніше.

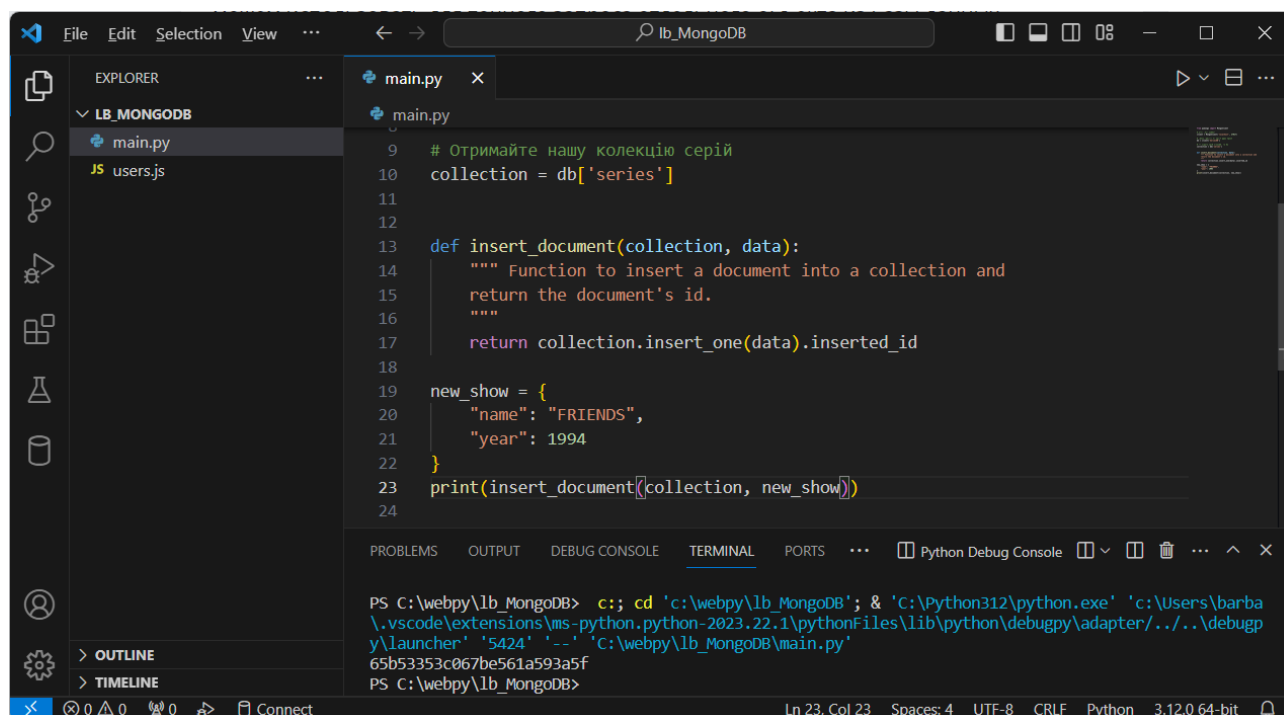


Рисунок 68– Результат додавання нового документу

Ми можемо надати значення `_id` замість того, щоб призначати його автоматично, що ми надали б у словнику:

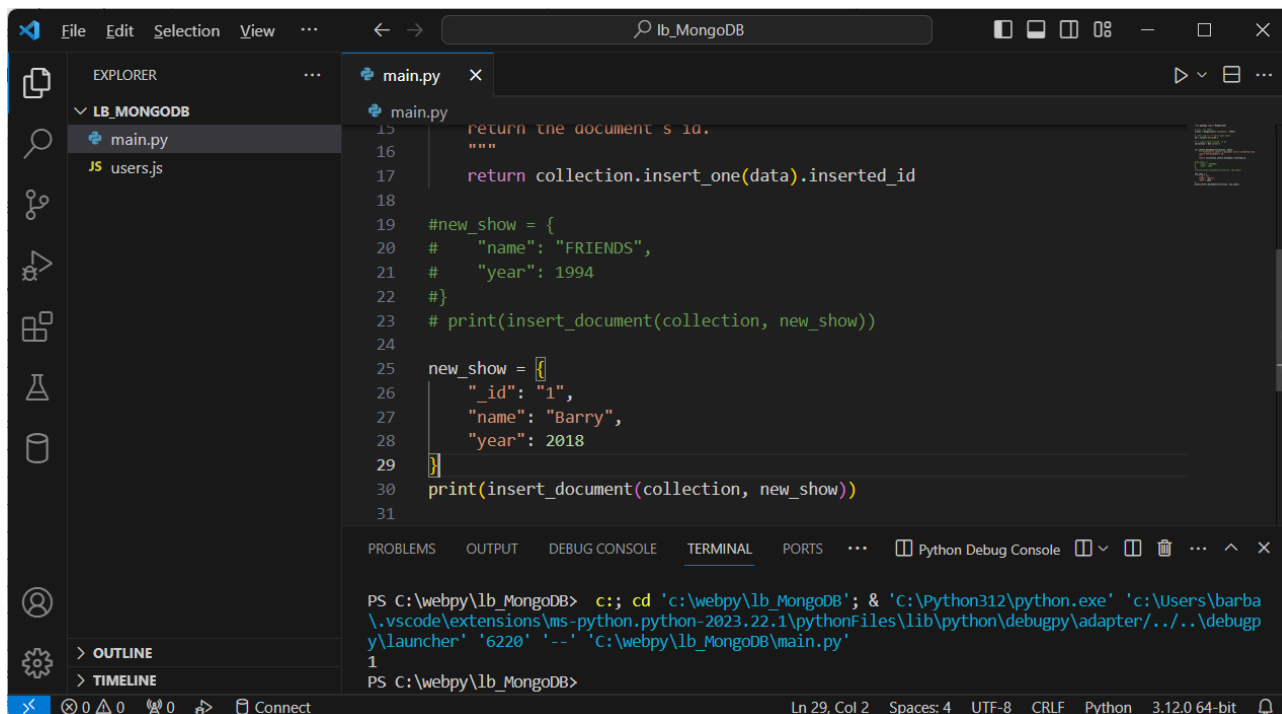


Рисунок 69 – Результат додавання нового документу з встановленням значення `_id`

14.7 Отримання документів

Для вилучення документів із бази даних ми використовуватимемо функцію `find_document()`, яка запитує нашу колекцію для одного або кількох документів. Наша функція отримає словник, який містить елементи, за якими ми хочемо фільтрувати, і необов'язковий аргумент, щоб вказати, чи хочемо ми один документ, чи кілька документів:

```

def find_document(collection, elements, multiple=False):
    """ Function to retrieve single or multiple documents from a provided
    Collection using a dictionary containing a document's elements.
    """
    if multiple:
        results = collection.find(elements)
        return [r for r in results]
    else:
        return collection.find_one(elements)

```

Використаємо цю функцію, щоб знайти деякі документи:

```

result = find_document(collection, {'year': 2018}, True)
print(result)

```

Під час виконання нашої функції якщо не надали параметр *multiple*, то в результаті ми отримаємо один документ. Коли параметр *multiple* надано, результатом є список усіх документів у нашій колекції, для яких встановлено атрибут *year* як 2018.

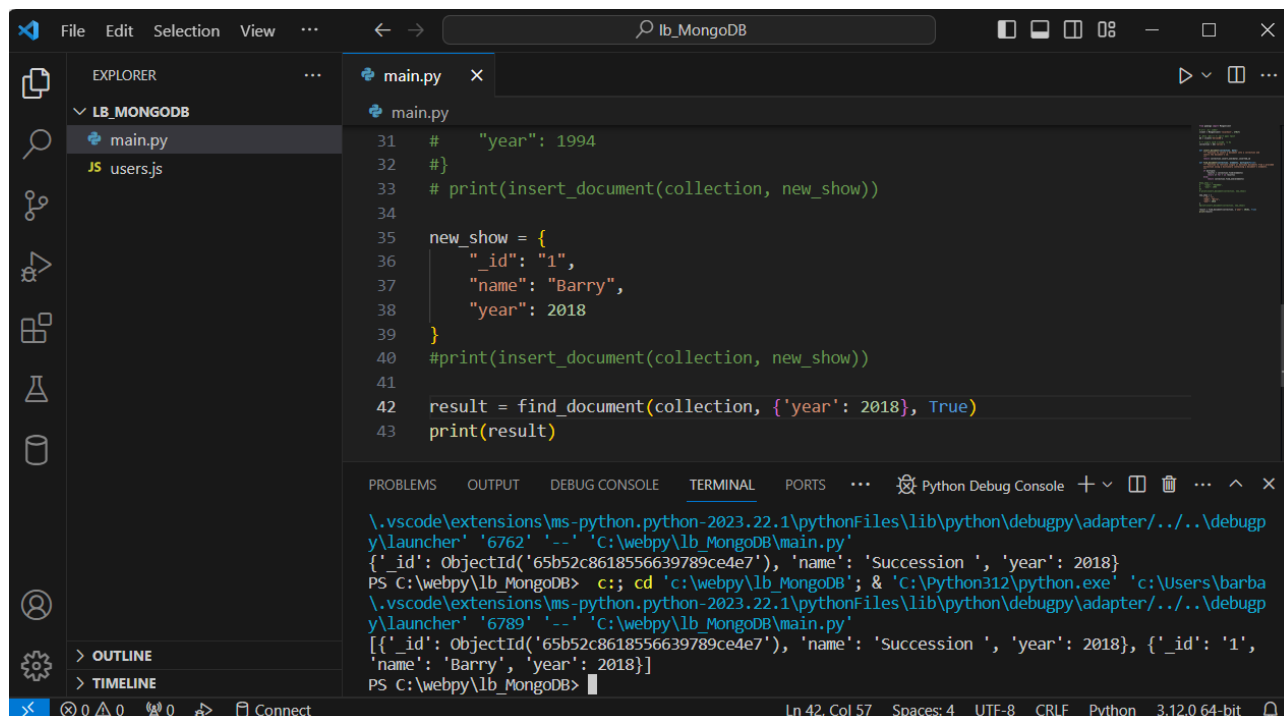


Рисунок 70 – Результат пошуку документів

14.8 Оновлення документів

Наша наступна функція *update_document()*, буде використовуватися для оновлення одного конкретного документа. Ми будемо використовувати *_id* документа і колекцію, до якої він належить, під час його пошуку:

```
def update_document(collection, query_elements, new_values):
    "Function to update a single document in a collection."
    collection.update_one(query_elements, {'$set': new_values})
```

Тепер давайте вставимо документ:

```
new_show = {
    "name": "Detective",
    "year": 2014
}
```



```
id_ = insert_document(collection, new_show)
```

Зробивши це, давайте оновимо документ, використовуючи повернутий `id_`:

```
update_document(collection, {'_id': id_}, {'name': 'True Detective'})
```

І, нарешті, давайте подивимося на нього, щоб переконатися, що нове значення було встановлено, і надрукуємо результат:

```
result = find_document(collection, {'_id': id_})
print(result)
```

Коли ми виконуємо наш скрипт, ми побачимо, що наш документ оновлено:

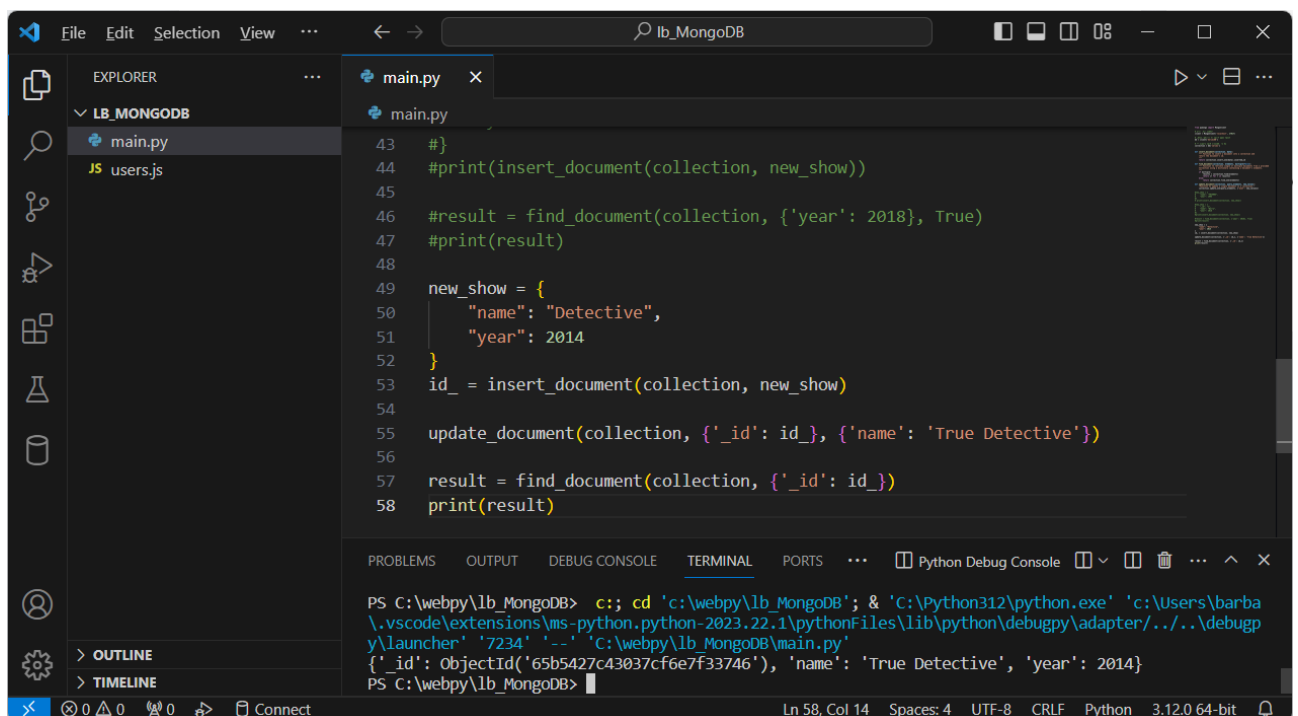


Рисунок 71 – Результат оновлення документу

14.9 Видалення документів

Напишемо функцію для видалення документів:

```
def delete_document(collection, query):
    "Function to delete a single document from a collection."
    collection.delete_one(query)
```

Оскільки ми використовуємо метод `delete_one`, тільки один документ можна видалити за виклик, навіть якщо запит відповідає кільком документам.

Тепер давайте використаємо функцію для видалення запису:

```
delete_document(collection, {'_id': id_})
```

Якщо ми спробуємо отримати той самий документ:

```
result = find_document(series_collection, {'_id': id_})
print(result)
```

Результатом буде:

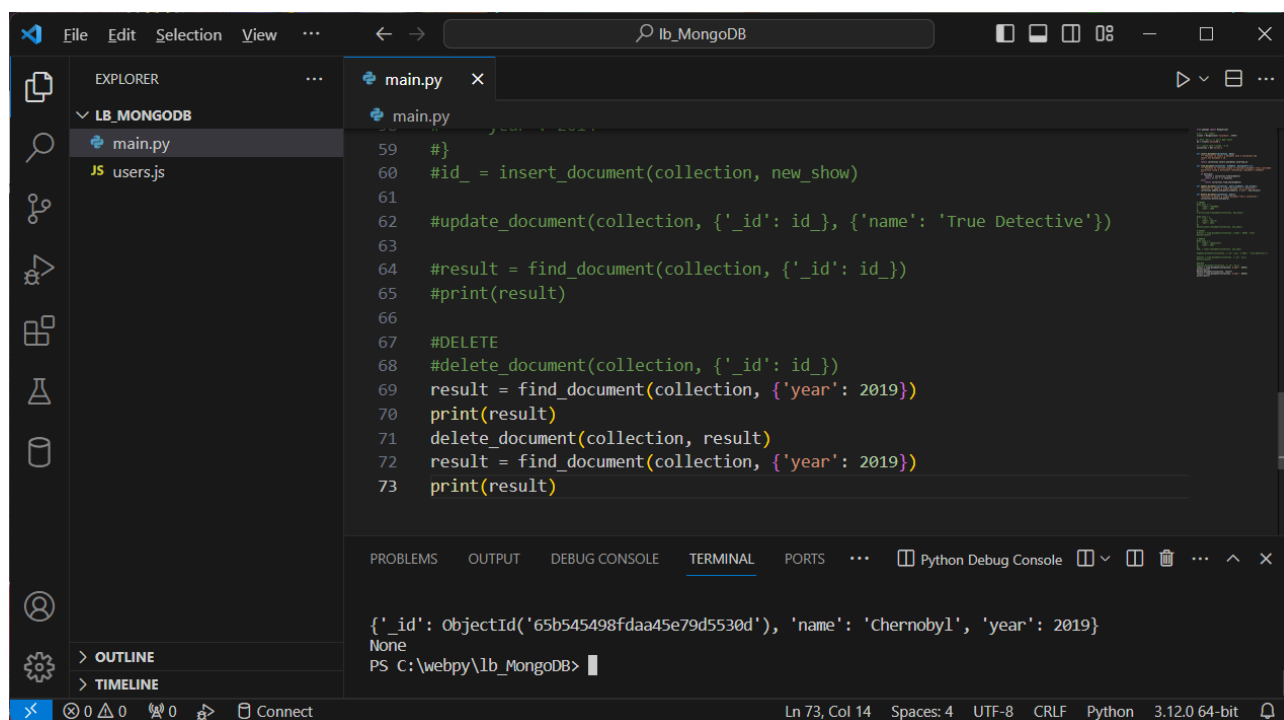


Рисунок 72 – Результат видалення документу